DSA03 - NATURAL LANGUAGE PROCESSING

ASSIGNMENT 3 CO3

Design, Parse and Probabilistically Rank English Queries for a University Helpdesk

Using Context-Free Grammars, Feature Structures and Earley Parsing

Name	**R Sai Hasnika**
Register Number	**192424178**
Course Code	**DSA0308**
Course	**Natural Language Processing**
Slot	**D**

Contents

Overview, Method and Summary of Results

Task 1 - Design a Context-Free Grammar for English Syntax (5 marks)

Task 2 - Context-Free Rules, Derivations and Parse Trees (3 marks)

Task 3 - Sentence-Level Constructions (4 marks)

Task 4 - Agreement (3 marks)

Task 5 - Subcategorization (4 marks)

Task 6 - Top-Down Parsing (4 marks)

Task 7 - Earley Parsing (5 marks)

Task 8 - Feature Structures and Unification (4 marks)

Task 9 - Probabilistic Context-Free Grammars (5 marks)

Task 10 - Implement and Test the Parsers (3 marks)

Task 11 - Compare, Recommend and Defend the Parsing Design (2 marks)

Task 12 - Day-4 Design Challenge: Re-evaluate and Adapt the Design (8 marks)

Appendix A - Complete Source Code

Appendix B - Query Set, Treebank and Effort Log

References

Overview, Method and Summary of Results

This report answers all twelve tasks of Assignment 3 for a text-based university helpdesk assistant. The assistant must give every student or staff query a reliable syntactic structure before any meaning can be extracted. Instead of describing the techniques only in the abstract, the work follows a "derive by hand, then verify by program" method: every grammar, rule table, derivation, chart, probability and count in this document was either produced by, or cross-checked against, the Python implementation listed in Appendix A. The numbers quoted in the text are therefore reproducible by running the supplied code.

Design decisions that run through the whole report

- **One master grammar, several views.** The helpdesk fragment is written once as a feature-augmented context-free grammar G_F with 52 phrase-structure rules and a lexicon of 165 word forms. The plain Chomsky-style CFG G required by Tasks 1 to 3 is derived from G_F by splitting the verb category by subcategorization frame (V-intr, V-trans, ...). The agreement-multiplied grammars of Task 4 are derived mechanically by a category-splitting program, so their rule counts are exact rather than estimated.
- **Three parsers on the same grammar.** A backtracking top-down parser (Task 6), an Earley chart parser with back-pointers and unification (Tasks 7, 8, 10) and a probabilistic CKY/Viterbi parser over the binarised grammar (Task 9) all read the same rules and lexicon, so their outputs can be compared sentence by sentence.
- **A hand-annotated mini-treebank.** Twenty sentences were annotated by selection (the annotator chose the correct tree among the candidates produced by the Earley parser). These trees give the maximum-likelihood rule probabilities with add-0.1 smoothing.
- **Notation.** Phrase categories are written S, NP, VP, PP, AdjP and so on; Nom is the nominal below the determiner; X/NP is a category X missing a noun phrase (a gap); features are written in capitals (AGR, CASE, VFORM, SUBCAT, PFORM). The symbol "=>" denotes one derivation step. Sentences are shown with their original capitalisation and punctuation, while the parsers work on lower-cased, punctuation-free tokens.

Headline results

Quantity	Value
Phrase-structure rules in G_F (and in plain CFG G)	52
Lexicon: word forms / (word, category) pairs in G_F	165 / 211 (in G)

Quantity	Value
Query set / ambiguous queries (two or more parses)	30 / 8
Queries parsed by the Earley, filtered top-down and CKY parsers	30 of 30 (same number of trees in 30 of 30)
Ungrammatical test sentences rejected by the feature grammar	9 of 9
Rules needed to enforce the same constraints without features	299 (number, case and verb form) and 3469 (adding preposition form), against 52 rules with features
Mini-treebank: sentences / nodes / rule applications	20 / 385 / 265
Earley chart states for the 18-word PP-chain sentence (132 parses)	659
Top-down steps for the 15-word sentence in the same family	more than 6,000,001 (budget exhausted)
Day-4 messages parsed completely: original grammar / extended grammar	7 of 20 / 17 of 20

Table 0.1 - Headline results of the study

The recommended design (Task 11) is a feature-based CFG parsed by an Earley parser whose packed forest is ranked by a PCFG, with a robust fragment-parsing fall-back added for the conversational input of Task 12. The report is organised task by task; each task states its guiding enquiry, gives the evidence expected and ends with a short interpretation.

Task 1 - Design a Context-Free Grammar for English Syntax

Guiding enquiry. How can the core structure of English be captured by a finite set of context-free rules, and what design decisions make the grammar both expressive and manageable?

1.1 Formal definition of the grammar

The helpdesk fragment is the context-free grammar G = (N, Sigma, R, S) where each component is fixed as follows.

- **N (non-terminals)** consists of 14 phrasal categories, namely S, SQ, SQ/NP, S/NP, VP/NP, NP, Nom, WhNP, VP, PP, AdjP, RelCl, CP, SubCl, and 23 lexical (part-of-speech) categories, namely Det, N, PN, Pron, WhPron, WhDet, WhAdv, Adj, Adv, P, Conj, Comp, Sconj, Aux, V-intr, V-trans, V-trpp, V-comp, V-ditrans, V-trcomp, V-pp, V-adj, V-pass. In total |N| = 37.
- **Sigma (terminals)** is the lexicon of 165 distinct word forms of the helpdesk domain (timetables, examinations, hostels, fees, library services and placements), listed by category in Section 1.3.
- **R (rules)** consists of 52 phrase-structure rules (Section 1.2) and 211 lexical rules of the form Category -> word. The grammar proper and the lexicon are kept apart, as the task requires.
- **S (start symbol)** is S, the sentence.

The grammar is epsilon-free (no rule has an empty right-hand side) and contains no cycles of unit rules, which is important later: every constituent covers at least one word, so the length of a derivation is bounded by the sentence length plus the number of unit rules.

1.2 The phrase-structure rules

Table 1.1 lists all 52 phrase-structure rules in Chomsky-style notation. Verb categories are already split by subcategorization frame (V-intr for intransitive verbs, V-trans for transitive verbs and so on); the reasons and the cost of this choice are analysed in Task 5. Agreement is deliberately not encoded in G; this is the baseline that Task 4 improves on.

Table 1.1 - Phrase-structure rules of G

No.	Rule	Construction
1	S → NP VP	declarative clause
2	S → VP	imperative clause
3	S → SQ	yes/no question

No.	Rule	Construction
4	S → WhNP VP	subject wh-question
5	S → WhNP SQ/NP	object wh-question (gap)
6	S → WhAdv SQ	adjunct wh-question
7	S → S Conj S	clause coordination (recursive)
8	SQ → Aux NP VP	auxiliary inversion
9	SQ → V-adj NP AdjP	copular inversion
10	SQ → V-pass NP V-trans	passive question
11	SQ → V-pp NP PP	copula with PP
12	SQ/NP → Aux NP VP/NP	inverted clause with gap
13	S/NP → NP VP/NP	clause with object gap
14	VP/NP → V-trans	object gap
15	VP/NP → V-ditrans NP	direct-object gap
16	VP/NP → V-trpp PP	object gap before PP
17	VP/NP → Aux VP/NP	gap under auxiliary
18	NP → Det Nom	determiner + nominal
19	NP → Nom	bare plural
20	NP → Pron	pronoun
21	NP → PN	proper noun
22	NP → NP PP	PP modifier (recursive)
23	NP → NP Conj NP	NP coordination (recursive)
24	NP → NP RelCl	relative clause (recursive)
25	Nom → N	simple nominal
26	Nom → N Nom	noun compound
27	Nom → Adj Nom	adjective modifier
28	Nom → Nom Conj Nom	nominal coordination
29	WhNP → WhPron	who, what
30	WhNP → WhDet Nom	which students
31	VP → V-intr	intransitive frame
32	VP → V-trans NP	transitive frame
33	VP → V-ditrans NP NP	ditransitive frame
34	VP → V-comp CP	sentential complement
35	VP → V-trcomp NP CP	object + clause
36	VP → V-pp PP	PP complement
37	VP → V-trpp NP PP	object + PP complement
38	VP → V-adj AdjP	copula + adjective
39	VP → V-pass V-trans	passive
40	VP → Aux VP	auxiliary

No.	Rule	Construction
41	VP → VP PP	PP adjunct (recursive)
42	VP → VP SubCl	adverbial clause (recursive)
43	VP → VP Conj VP	VP coordination (recursive)
44	PP → P NP	prepositional phrase
45	AdjP → Adj	adjective phrase
46	AdjP → Adv Adj	modified adjective
47	AdjP → Adj PP	adjective + PP
48	AdjP → AdjP Conj AdjP	adjective coordination
49	RelCl → Comp VP	subject relative
50	RelCl → Comp S/NP	object relative
51	CP → Comp S	that-clause
52	SubCl → Sconj S	subordinate clause

Recursive rules and unbounded structure. Nine rules are left-recursive or otherwise recursive on the same category: NP -> NP PP, NP -> NP Conj NP, NP -> NP RelCl, VP -> VP PP, VP -> VP SubCl, VP -> VP Conj VP, S -> S Conj S, Nom -> Nom Conj Nom and AdjP -> AdjP Conj AdjP. Each lets a constituent of a category contain another constituent of the same category, so a finite rule set generates an unbounded set of trees. For example, the rule NP -> NP PP together with PP -> P NP gives the family the form, the form in the office, the form in the office near the library, and so on without limit. Figure 1.1 shows the sentence "Submit the form in the office near the library", where the object noun phrase contains a PP that contains a noun phrase that again contains a PP (or, in the other analyses, the PPs attach to the verb phrase). The depth of nesting is limited only by the length of the input, not by the grammar. Because the recursion is on the left edge of the rule, it also creates the parsing difficulties analysed in Task 6.

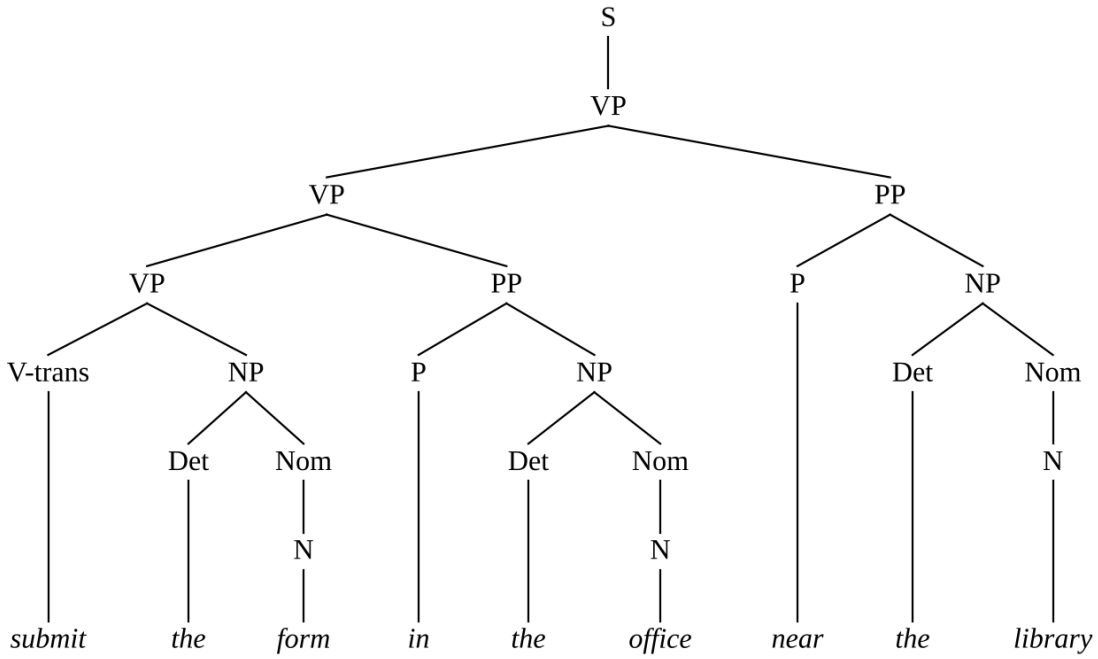


Figure 1.1 - One of the five analyses of "Submit the form in the office near the library" (recursion in NP -> NP PP)

1.3 The lexicon and the part-of-speech inventory

Table 1.2 gives the lexicon of G. Verb forms are listed in every verb category in which they can occur, so a word such as close appears under both V-intr (The library closes) and V-trans (Close the portal), and the copula forms appear under V-adj, V-trans, V-pp and V-pass.

Table 1.2 - Lexicon of G by category

Category	Role	Words
Det	determiner	the, a, this, these, those, my, our, every, some
N	common noun	student, students, exam, exams, form, forms, fee, fees, hostel, hostels, room, rooms, library, libraries, timetable, timetables, receipt, receipts, notice, notices, seminar, seminars, placement, placements, portal, portals, deadline, deadlines, book, books, charge, charges, result, results, course, courses, office, offices, certificate, certificates, hall, halls, staff
PN	proper noun	friday, monday
Pron	pronoun	i, you, he, she, it, we, they, me, him, us, them
WhPron	wh-pronoun	who, what
WhDet	wh-determiner	which, what
WhAdv	wh-adverb	when, where, how, why
Adj	adjective	due, late, available, pending, new, final, open

Category	Role	Words
Adv	degree adverb	very, already
P	preposition	in, on, near, for, to, with, at, by, from, after, before
Conj	coordinator	and, or
Comp	complementizer	that
Sconj	subordinator	if, because, when, unless
Aux	auxiliary	will, can, should, did, do, does
V-intr	verb, frame intr	crash, crashes, crashed, start, starts, started, close, closes, closed, pay, pays, paid
V-trans	verb, frame trans	close, closes, closed, submit, submits, submitted, check, checks, checked, pay, pays, paid, send, sends, sent, show, shows, showed, shown, tell, tells, told, book, books, booked, postpone, postpones, postponed, collect, collects, collected, renew, renews, renewed, hold, holds, held, publish, publishes, published, be, is, are, was, were
V-trpp	verb, frame trpp	submit, submits, submitted, send, sends, sent
V-comp	verb, frame comp	check, checks, checked, say, says, said
V-ditrans	verb, frame ditrans	send, sends, sent, show, shows, showed, book, books, booked
V-trcomp	verb, frame trcomp	tell, tells, told
V-pp	verb, frame pp	apply, applies, applied, register, registers, registered, wait, waits, waited, be, is, are, was, were
V-adj	verb, frame adj	be, is, are, was, were
V-pass	verb, frame pass	be, is, are, was, were

Justification of the granularity. The category inventory sits between two extremes. A coarse tag set (noun, verb, adjective, function word) would make the grammar tiny, but it could not state which verbs take which complements, so The portal crashed the form and Submit would be accepted. A very fine tag set (separate categories for every number, person, case and tense) would reject these, but it multiplies the rules, as Task 4 quantifies. The chosen inventory therefore makes the distinctions that decide the shape of the tree and leaves distinctions of form to features:

- Pronouns, proper nouns, determiners, wh-words and coordinators are separate categories because each heads or introduces a different phrase structure.

- Auxiliaries are separate from main verbs because they invert in questions (Did the student submit the form?) and take a verb phrase complement.
- Main verbs are split into nine frames because the frame determines the number and category of the sister constituents inside the verb phrase.
- Complementizer (that) and subordinator (if, because, when, unless) are different categories, since the first introduces a complement clause and the second an adjunct clause.
- Number, person, case and verb form are not part of the category names; they are handled informally in Task 4 and formally in Task 8.

1.4 Constituency, head and phrasal category

A **constituent** is a group of words that behaves as a unit; in a tree it is the set of words dominated by one node. A **phrasal category** (NP, VP, PP, AdjP) names the type of constituent. The **head** of a phrase is the word that determines its category and its main properties: the noun in a noun phrase (students in the late students), the verb in a verb phrase, the preposition in a prepositional phrase, and the adjective in an adjective phrase. The head projects its category upward (a noun heads Nom and NP, a verb heads VP) and selects the complements; in the feature grammar of Task 8 this is expressed by letting the head pass its features to the phrase. Three standard tests identify constituents; they are applied below to three helpdesk sentences.

Sentence (a): The student submitted the form on Friday.

- Substitution. The string the form can be replaced by a pronoun: The student submitted it on Friday. Likewise the student is replaced by he and submitted the form by did so (The student submitted the form on Friday and the staff did so on Monday). The non-constituent form on cannot be replaced in this way: The student submitted the it Friday is ill-formed.
- Movement. The prepositional phrase moves as a unit to the front: On Friday, the student submitted the form. The noun phrase moves in the passive: The form was submitted on Friday. The string the form on cannot move: *Form on, the student submitted the Friday.
- Coordination. Constituents of the same category coordinate: The student submitted the form and the receipt on Friday (NP and NP), The student submitted the form and paid the fee (VP and VP). The non-constituent pair is bad: *The student submitted the form on and the receipt at Monday.

Sentence (b): Send me the receipt.

- Substitution. The two objects are separate constituents: Send me it and Send him the receipt are fine, but the sequence me the receipt cannot be replaced by one pronoun (*Send it, which means something else and drops the recipient), so it is not a single NP.
- Movement. The second object can be questioned and moved: What did you send me? The pair cannot move together: *Me the receipt, send.
- Coordination. Send me the receipt and the form coordinates the second NP, and Send me the receipt and give him the form coordinates two whole VPs.

Sentence (c): The library closes after the seminar.

- Substitution. after the seminar is replaced by the adverb then or later, so it is a PP constituent: The library closes then.
- Movement. After the seminar, the library closes. is acceptable, while *The seminar, the library closes after is not.
- Coordination. The library closes after the seminar and before the exam coordinates two PPs.

All three tests agree with the bracketing produced by G in each case (VP -> VP PP for the adjunct in (a) and (c), VP -> V-ditrans NP NP in (b)). The head test also explains the agreement and case facts of Task 4: the noun students determines the number of the whole NP, the verb determines the form and frame of the whole VP.

1.5 G is context-free by construction

Every rule of G has the form A -> alpha where A is a single non-terminal and alpha is a string over N and Sigma. A rule may therefore be applied to a non-terminal wherever it occurs, regardless of the symbols to its left or right; this is exactly the definition of a context-free grammar. By construction G has no rule of the context-sensitive form alpha A beta -> alpha gamma beta (with gamma non-empty) and no rule that rewrites more than one symbol at a time. The constraints that Task 4 and Task 5 add as feature equations are not contextual rewriting rules: the equations only restrict which trees are admissible and they range over a finite set of values (two values of AGR, two of CASE), so the language remains context-free; the feature-augmented grammar is a compact abbreviation of a (much larger) pure CFG, as the expansion programme of Task 4 demonstrates.

A phenomenon that strains the context-free assumption

English contains constructions that are not context-free in general. The classical case is **respectively-type cross-serial dependencies**: The students, the office and the portal submitted, approved and crashed, respectively. The k-th subject must be matched with the k-th verb, which gives the copy-like pattern a1 a2 ... an b1 b2 ... bn with the pairing of a_i and b_i; such a language of crossing dependencies cannot be generated by any context-free grammar (it needs the power of a mildly context-sensitive formalism such as tree-adjoining grammar). A more modest practical strain is **unbounded agreement and coreference**: The student who submitted the forms that the office returned last week has paid, where the verb has must agree with student across an arbitrarily long relative clause. A pure CFG can enforce this only by multiplying the categories (Task 4), while feature structures handle it directly (Task 8). For the helpdesk domain such constructions are rare, and the context-free fragment suffices.

Evidence summary for Task 1. G has 52 rules, 211 lexical rules over 165 word forms, nine recursive rules, and a justified category inventory; constituency tests were applied to three sentences; and the context-free character and its limit were discussed.

Task 2 - Context-Free Rules, Derivations and Parse Trees

Guiding enquiry. How are sentences generated from the grammar, and how are trees used to represent and compare syntactic analyses?

This task works with five sentences of the query set (their identifiers are those of Appendix B): a declarative (Q1), an imperative with a ditransitive verb (Q11), a declarative with a PP adjunct (Q3), a yes/no question (Q21) and a coordinated declarative (Q8). All derivations use the plain grammar G of Task 1; lexical rules such as N -> student are applied as ordinary steps, so each derivation ends in the word string. A leftmost derivation always expands the leftmost non-terminal of the current sentential form, and a rightmost derivation the rightmost one.

2.1 Sentence Q1: "The student submitted the form."

Bracketed notation. (S (NP (Det the) (Nom (N student))) (VP (V-trans submitted) (NP (Det the) (Nom (N form)))))

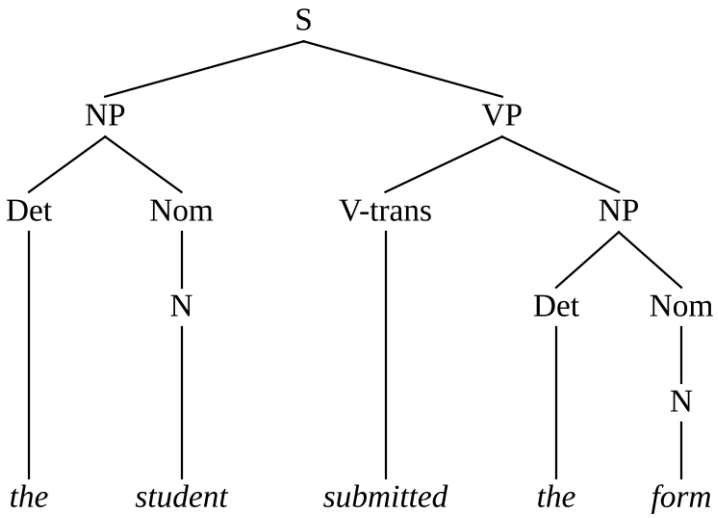


Figure 2.1 - Parse tree of "The student submitted the form."

Leftmost derivation (11 steps). S => NP VP => Det Nom VP => the Nom VP => the N VP => the student VP => the student V-trans NP => the student submitted NP => the student submitted Det Nom => the student submitted the Nom => the student submitted the N => the student submitted the form

Rightmost derivation (11 steps). S => NP VP => NP V-trans NP => NP V-trans Det Nom => NP V-trans Det N => NP V-trans Det form => NP V-trans the form => NP submitted the form

=> Det Nom submitted the form => Det N submitted the form => Det student submitted the form => the student submitted the form

The tree has 11 rule applications and 9,450 different derivations; the leftmost and rightmost derivations shown are two of them.

2.2 Sentence Q11: "Send me the receipt."

Bracketed notation. (S (VP (V-ditrans send) (NP (Pron me)) (NP (Det the) (Nom (N receipt)))))

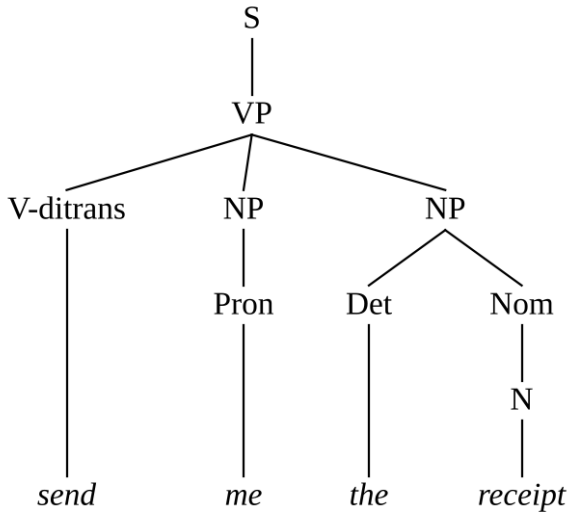


Figure 2.2 - Parse tree of "Send me the receipt."

Leftmost derivation (9 steps). S => VP => V-ditrans NP NP => send NP NP => send Pron NP => send me NP => send me Det Nom => send me the Nom => send me the N => send me the receipt

Rightmost derivation (9 steps). S => VP => V-ditrans NP NP => V-ditrans NP Det Nom => V-ditrans NP Det N => V-ditrans NP Det receipt => V-ditrans NP the receipt => V-ditrans Pron the receipt => V-ditrans me the receipt => send me the receipt

The tree has 9 rule applications and 315 different derivations; the leftmost and rightmost derivations shown are two of them.

2.3 Sentence Q3: "The exams start on Friday."

Bracketed notation. (S (NP (Det the) (Nom (N exams))) (VP (VP (V-intr start)) (PP (P on) (NP (PN friday)))))

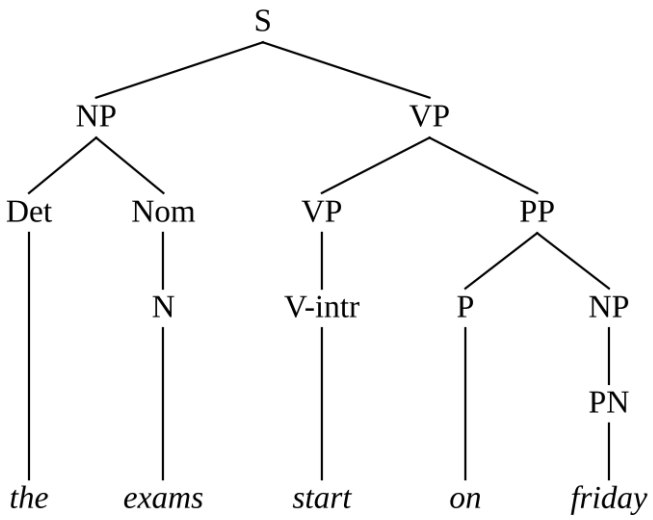


Figure 2.3 - Parse tree of "The exams start on Friday."

Leftmost derivation (12 steps). S => NP VP => Det Nom VP => the Nom VP => the N VP => the exams VP => the exams VP PP => the exams V-intr PP => the exams start PP => the exams start P NP => the exams start on NP => the exams start on PN => the exams start on friday

Rightmost derivation (12 steps). S => NP VP => NP VP PP => NP VP P NP => NP VP P PN => NP VP P friday => NP VP on friday => NP V-intr on friday => NP start on friday => Det Nom start on friday => Det N start on friday => Det exams start on friday => the exams start on friday

The tree has 12 rule applications and 44,550 different derivations; the leftmost and rightmost derivations shown are two of them.

2.4 Sentence Q21: "Did the student submit the form?"

Bracketed notation. (S (SQ (Aux did) (NP (Det the) (Nom (N student))) (VP (V-trans submit) (NP (Det the) (Nom (N form))))))

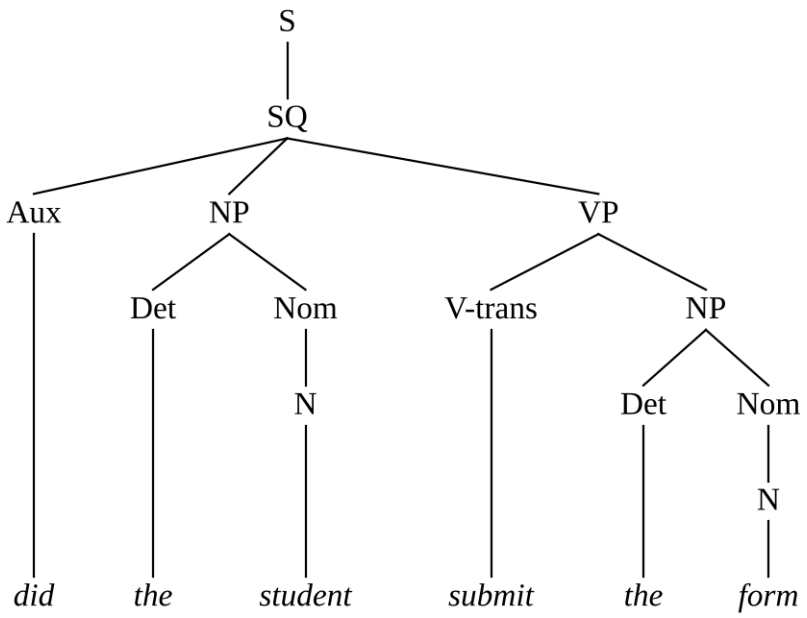


Figure 2.4 - Parse tree of "Did the student submit the form?"

Leftmost derivation (13 steps). S => SQ => Aux NP VP => did NP VP => did Det Nom VP => did the Nom VP => did the N VP => did the student VP => did the student V-trans NP => did the student submit NP => did the student submit Det Nom => did the student submit the Nom => did the student submit the N => did the student submit the form

Rightmost derivation (13 steps). S => SQ => Aux NP VP => Aux NP V-trans NP => Aux NP V-trans Det Nom => Aux NP V-trans Det N => Aux NP V-trans Det form => Aux NP V-trans the form => Aux NP submit the form => Aux Det Nom submit the form => Aux Det N submit the form => Aux Det student submit the form => Aux the student submit the form => did the student submit the form

The tree has 13 rule applications and 103,950 different derivations; the leftmost and rightmost derivations shown are two of them.

2.5 Sentence Q8: "They submitted the forms and the fees."

Bracketed notation. (S (NP (Pron they)) (VP (V-trans submitted) (NP (NP (Det the) (Nom (N forms))) (Conj and) (NP (Det the) (Nom (N fees))))))

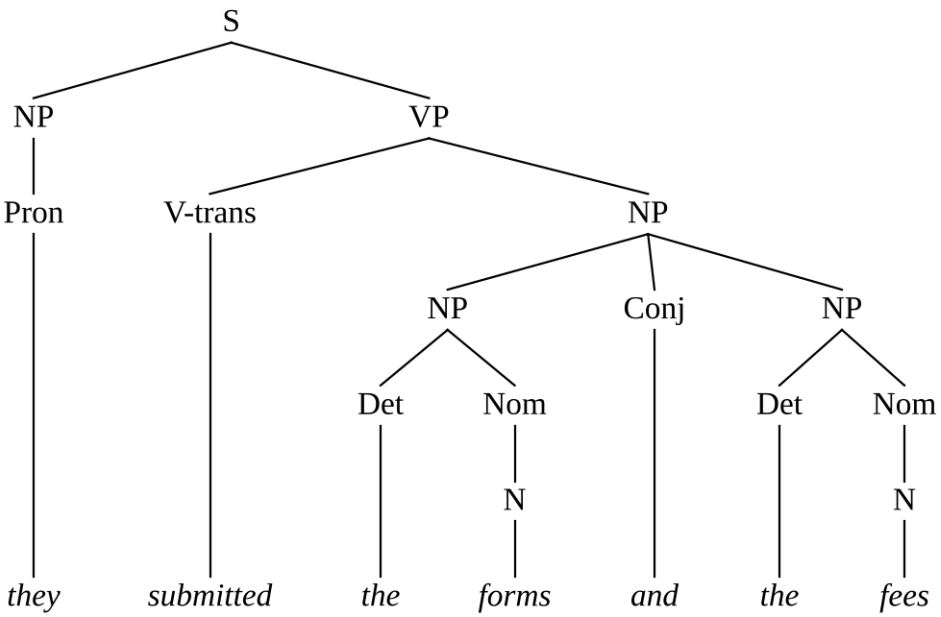


Figure 2.5 - Parse tree of "They submitted the forms and the fees."

Leftmost derivation (15 steps). S => NP VP => Pron VP => they VP => they V-trans NP => they submitted NP => they submitted NP Conj NP => they submitted Det Nom Conj NP => they submitted the Nom Conj NP => they submitted the N Conj NP => they submitted the forms Conj NP => they submitted the forms and NP => they submitted the forms and Det Nom => they submitted the forms and the Nom => they submitted the forms and the N => they submitted the forms and the fees

Rightmost derivation (15 steps). S => NP VP => NP V-trans NP => NP V-trans NP Conj NP => NP V-trans NP Conj Det Nom => NP V-trans NP Conj Det N => NP V-trans NP Conj Det fees => NP V-trans NP Conj the fees => NP V-trans NP and the fees => NP V-trans Det Nom and the fees => NP V-trans Det N and the fees => NP V-trans Det forms and the fees => NP V-trans the forms and the fees => NP submitted the forms and the fees => Pron submitted the forms and the fees => they submitted the forms and the fees

The tree has 15 rule applications and 5,675,670 different derivations; the leftmost and rightmost derivations shown are two of them.

2.6 Tree measurements

For each tree, the **height** is the number of edges on the longest path from the root to a word, the **number of nodes** counts every labelled node and every word, and the **number of rule**

applications counts the internal nodes plus the lexical rules (it equals the length of every derivation of the tree).

Table 2.1 - Measurements of the five parse trees

Id	Sentence	Words	Height	Nodes	Rule applications	Derivations
Q1	The student submitted the form.	5	5	16	11	9,450
Q11	Send me the receipt.	4	5	13	9	315
Q3	The exams start on Friday.	5	5	17	12	44,550
Q21	Did the student submit the form?	6	6	19	13	103,950
Q8	They submitted the forms and the fees.	7	6	22	15	5,675,670

Counting convention: a tree with w words has w preterminal nodes; if it has q phrasal nodes, the number of rule applications is q + w and the number of nodes is q + 2w. For Q1, w = 5 and q = 6 (S, NP, Nom, VP, NP, Nom), so there are 11 rule applications and 16 nodes, which agrees with Table 2.1.

2.7 Demonstrating structural ambiguity

Ambiguity 1: Q16 "Show the students with the late fee notice."

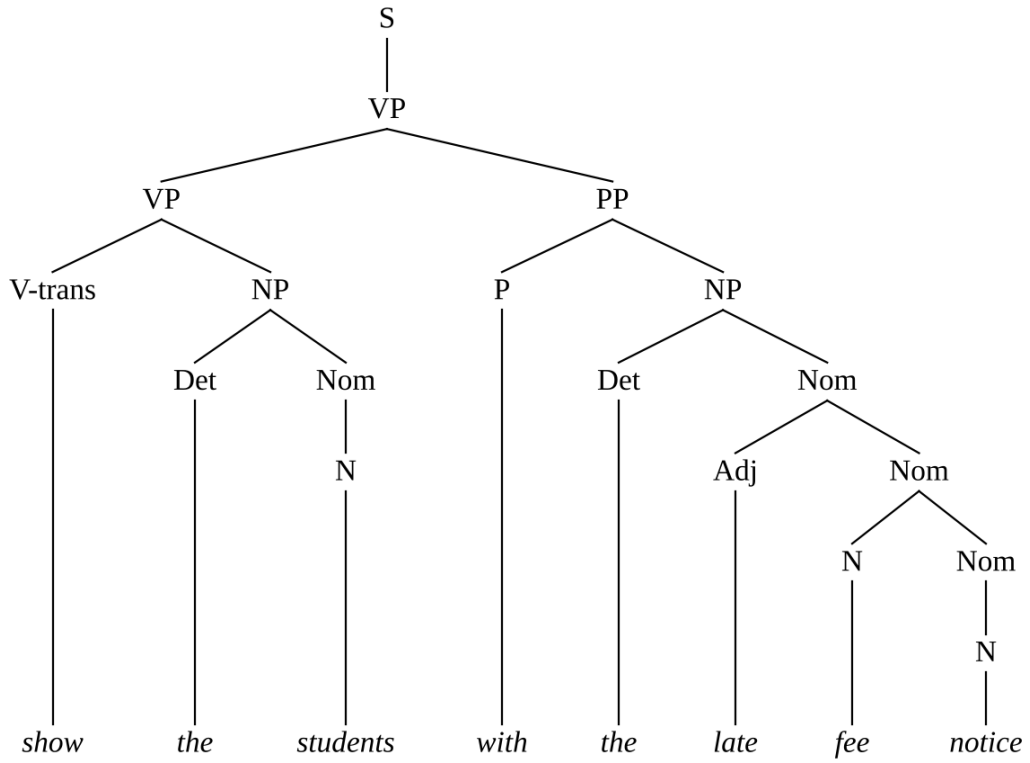


Figure 2.6 - First analysis of "Show the students with the late fee notice."

Bracketing: (S (VP (VP (V-trans show) (NP (Det the) (Nom (N students)))) (PP (P with) (NP (Det the) (Nom (Adj late) (Nom (N fee) (Nom (N notice))))))))

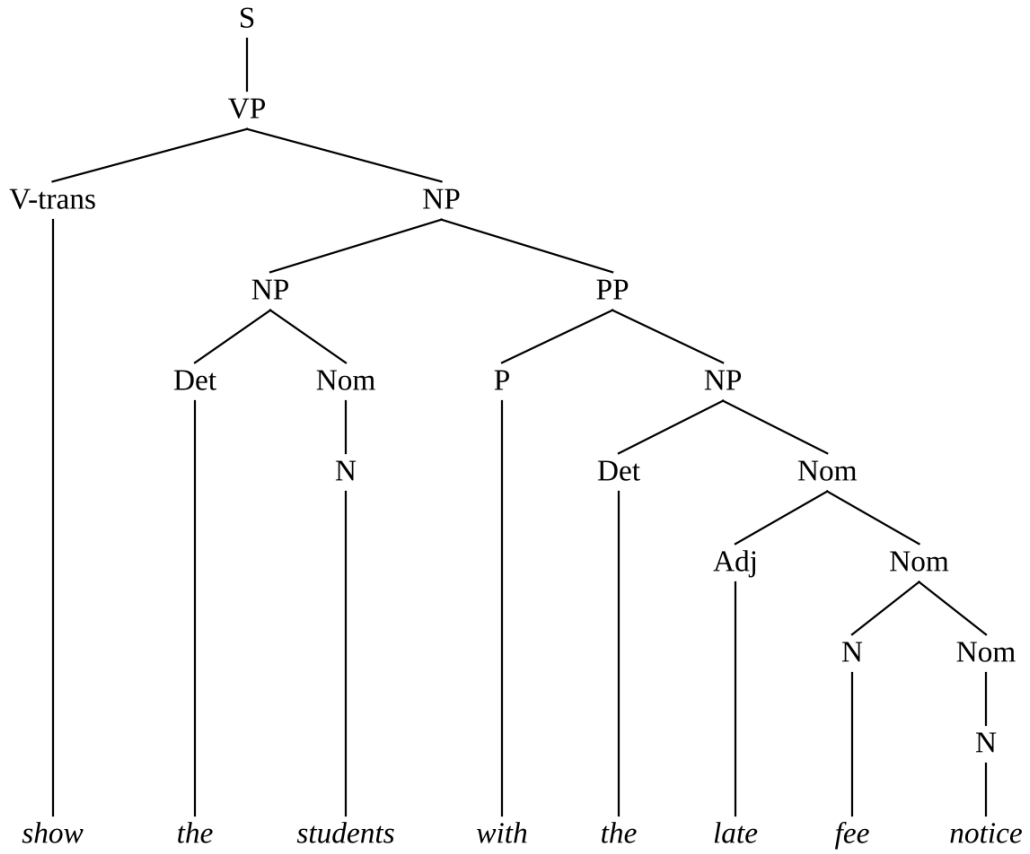


Figure 2.7 - Second analysis of "Show the students with the late fee notice."

Bracketing: (S (VP (V-trans show) (NP (NP (Det the) (Nom (N students))) (PP (P with) (NP (Det the) (Nom (Adj late) (Nom (N fee) (Nom (N notice)))))))))

Ambiguity 2: Q19 "Check the fees and hostel charges."

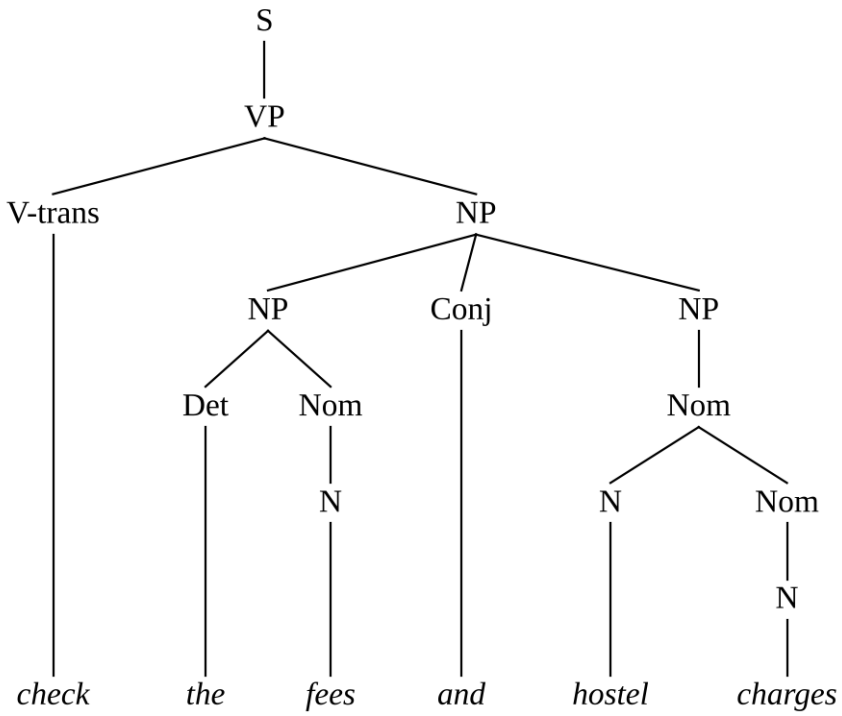


Figure 2.8 - First analysis of "Check the fees and hostel charges."

Bracketing: (S (VP (V-trans check) (NP (NP (Det the) (Nom (N fees))) (Conj and) (NP (Nom (N hostel) (Nom (N charges)))))))

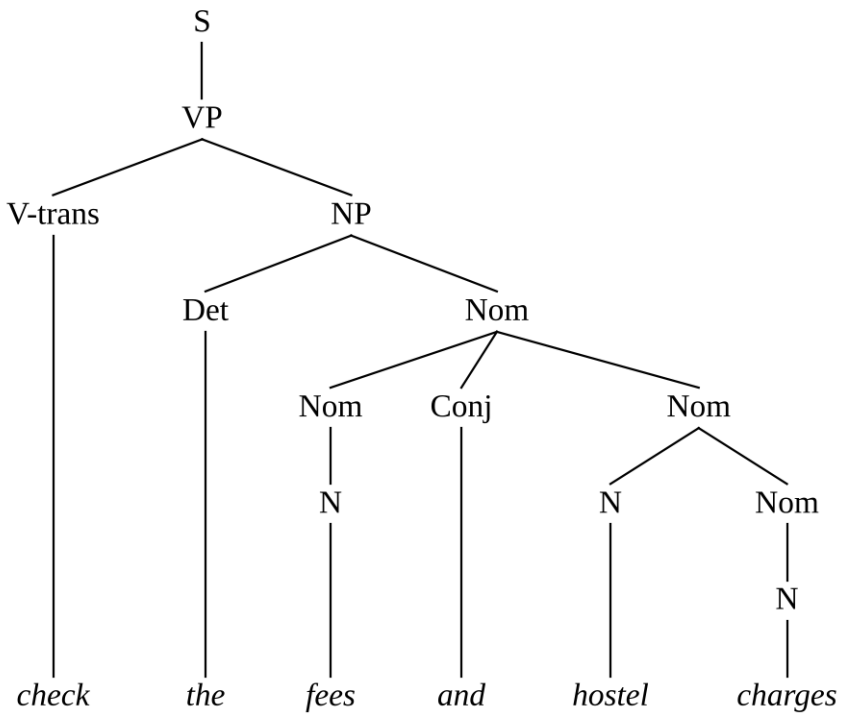


Figure 2.9 - Second analysis of "Check the fees and hostel charges."

Bracketing: (S (VP (V-trans check) (NP (Det the) (Nom (Nom (N fees)) (Conj and) (Nom (N hostel) (Nom (N charges)))))))

Difference in the first pair (PP-attachment). In "Show the students with the late fee notice" the prepositional phrase with the late fee notice attaches either to the verb phrase (VP -> VP PP: the showing is done with the notice, an instrument or accompaniment reading, tree height 8) or to the noun phrase the students (NP -> NP PP: the students are those who have the notice, tree height 9). The intended meaning for the helpdesk is the second, which is why Task 9 matters. Both trees use 18 rule applications.

Difference in the second pair (coordination scope). In "Check the fees and hostel charges" the coordinator joins either two noun phrases, [the fees] and [hostel charges], so that only the first noun phrase has a determiner, or two nominals inside one noun phrase, the [fees and hostel charges], so that the determiner covers both. The two trees use 14 and 13 rule applications respectively. The meaning differs: in the second analysis the applies to both the fees and the charges.

2.8 Derivations and trees

A **derivation** is a sequence of sentential forms in which each form is obtained from the previous one by one rule application; a **parse tree** is the order-free record of which rule was applied to

which node. The tree abstracts away the order in which independent non-terminals were expanded. Two derivations that differ only in the order of expanding sibling nodes give the same tree, which is why one tree corresponds to many derivations. The number of derivations of a tree equals the number of linear orderings of its rule applications in which every parent precedes its children, that is n! divided by the product of the subtree sizes (the tree analogue of the hook-length formula). For Q1 this gives 11!/(product of subtree sizes) = 9,450, and for the coordinated sentence Q8 it gives 5,675,670. Among all these derivations exactly one is leftmost and exactly one is rightmost, so leftmost derivations are in one-to-one correspondence with parse trees; this is why ambiguity is defined on trees (equivalently on leftmost derivations) and not on derivations in general. A sentence is ambiguous exactly when it has two different trees.

2.9 Testing whether a string belongs to L(G)

Procedure. Membership of a string w = w1 ... wn in L(G) is decidable. The procedure used in this work is:

1. Look up each word in the lexicon; if a word has no entry, reject (the string contains a symbol outside Sigma).
2. Run a chart recogniser (Earley, Task 7, or CKY after conversion to Chomsky normal form, Task 9) that records, for every span (i, j), the set of categories that can derive wi+1 ... wj.
3. Accept if and only if S is in the set of categories for the span (0, n).

Because G is epsilon-free, any derivation of w has bounded length, and the recogniser takes O(n^3 |G|) time. A naive generate-and-test search over derivations also terminates for an epsilon-free grammar without unit cycles but takes exponential time.

Limits. (i) Membership of the string in L(G) is not the same as grammaticality in English: G over-generates (Task 4 and Task 5 show accepted ungrammatical strings such as The students submits the form) and under-generates (Task 12 shows many natural messages outside L(G)). (ii) A positive answer does not tell which of several trees is intended; for that, probabilities are needed (Task 9). (iii) Out-of-vocabulary words make the string fall outside L(G) even if the structure is perfectly regular. (iv) The procedure answers for the grammar G, so any mismatch between G and the real language is invisible to it.

Task 3 - Sentence-Level Constructions

Guiding enquiry. Which sentence types must the helpdesk understand, and how does the grammar represent each of them?

3.1 The four main constructions

Students ask the helpdesk questions, give commands and make statements, so the grammar must cover declaratives, imperatives, yes/no questions and wh-questions. Table 3.1 lists the rules that realise each construction in G, with an example from the query set. The rules for the sentence types differ mainly in what precedes the verb phrase: a subject noun phrase in a declarative, nothing in an imperative, an inverted auxiliary in a yes/no question, and a fronted wh-phrase in a wh-question.

Table 3.1 - Rules for the sentence-level constructions

Construction	Rules	Example
Declarative	S → NP VP	The students paid the fee.
Imperative	S → VP (the subject you is understood and absent)	Send me the receipt.
Yes/no question	S → SQ SQ → Aux NP VP SQ → V-adj NP AdjP SQ → V-pass NP V-trans SQ → V-pp NP PP	Did the student submit the form? Is the fee due? Were the exams postponed?
Wh-question, subject	S → WhNP VP WhNP → WhPron WhNP → WhDet Nom	Who submitted the form? Which students paid the fee?
Wh-question, non-subject	S → WhNP SQ/NP SQ/NP → Aux NP VP/NP VP/NP → V-trans | V-ditrans NP | V-trpp PP | Aux VP/NP	What did you submit? Which form should I submit?
Wh-question, adjunct	S → WhAdv SQ	When does the library close?

3.2 Auxiliary inversion, wh-movement and embedded clauses

Auxiliary inversion. A yes/no question is the declarative with the finite auxiliary moved in front of the subject: The student will submit the form becomes Will the student submit the form? In a context-free grammar there is no movement operation, so the inverted order is written directly as a separate rule, SQ -> Aux NP VP, in which the verb phrase after the subject is a base-form verb phrase (submit the form). The copula be inverts in the same way, but

because it has no auxiliary partner it needs its own rules (SQ -> V-adj NP AdjP for Is the fee due?, SQ -> V-pass NP V-trans for Were the exams postponed? and SQ -> V-pp NP PP for Is the fee in the portal?).

Wh-movement through a gap. A non-subject wh-question such as What did you submit? contains the verb submit without its object; the object has been fronted. The grammar represents this with slash categories in the style of generalised phrase structure grammar: a category X/NP denotes an X that is missing one noun phrase. The rule S -> WhNP SQ/NP says that a fronted wh-phrase is followed by an inverted clause that lacks a noun phrase; SQ/NP -> Aux NP VP/NP passes the gap into the verb phrase, and VP/NP -> V-trans realises the gap by letting the transitive verb stand without its object. Gap-passing rules are needed wherever the gap may occur: after a transitive verb (VP/NP -> V-trans), in the second object of a ditransitive verb (VP/NP -> V-ditrans NP, as in the receipt that the office sent me), before a PP complement (VP/NP -> V-trpp PP) and below an auxiliary (VP/NP -> Aux VP/NP). Subject questions need no gap because the wh-word simply replaces the subject noun phrase (S -> WhNP VP), and the verb agrees with it.

Subordinate and relative clauses. Three clause types are included. A complement clause is introduced by the complementizer that (CP -> Comp S) and is selected by a verb frame (The notice says that the fee is due, Tell me that the fee is paid). An adverbial clause is introduced by a subordinator and attaches to the verb phrase as an adjunct (SubCl -> Sconj S, VP -> VP SubCl: Submit the form if the fee is paid). A relative clause modifies a noun phrase (NP -> NP RelCl): a subject relative uses RelCl -> Comp VP (the exams that were postponed), and an object relative uses RelCl -> Comp S/NP, so that the clause contains a gap (the form that I submitted).

3.3 Parse trees for each construction

Figure 2.1 (Task 2) shows the declarative "The student submitted the form". Its structure S -> NP VP is the reference against which the other constructions are compared.

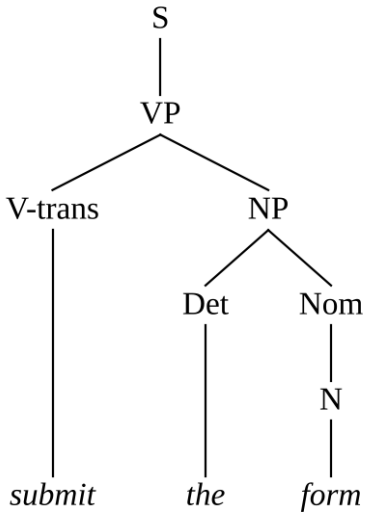


Figure 3.1 - Imperative: "Submit the form."

Difference from the declarative. The imperative tree has no subject noun phrase: S dominates a single VP directly, so the tree is shallower on the left and the verb is in its base form.

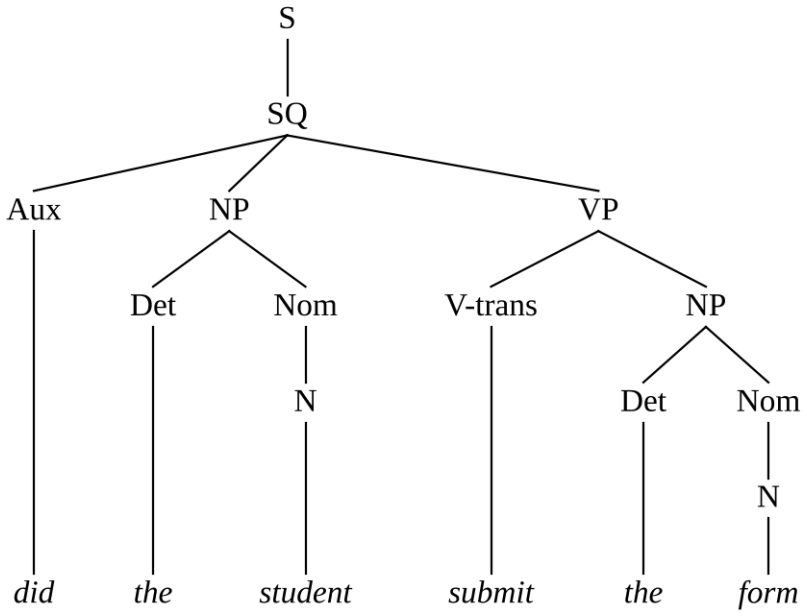


Figure 3.2 - Yes/no question: "Did the student submit the form?"

Difference. The auxiliary did is a sister of the subject NP and the VP, giving a flat SQ node with three daughters. The finite information sits on the auxiliary and the main verb is in the base form.

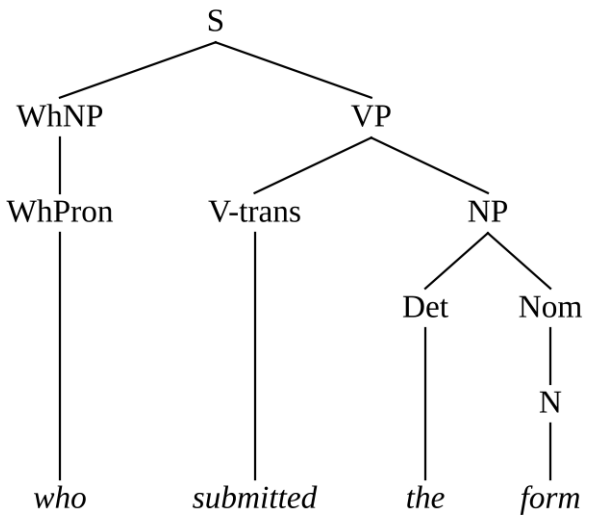


Figure 3.3 - Subject wh-question: "Who submitted the form?"

Difference. The wh-pronoun occupies the subject position as a WhNP and no inversion occurs; the structure is the declarative with the subject replaced.

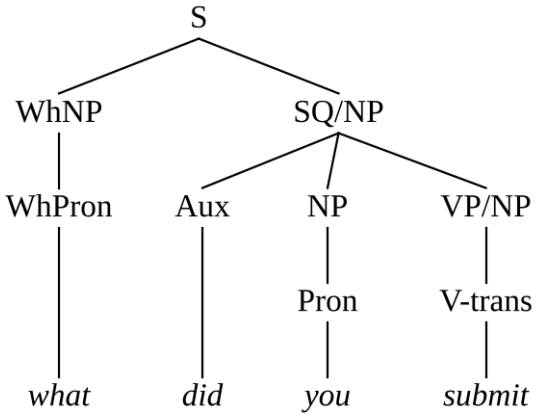


Figure 3.4 - Non-subject wh-question with a gap: "What did you submit?"

Difference. The tree contains the slash categories SQ/NP and VP/NP. The verb submit appears as the only daughter of VP/NP, so the object is missing, and the fronted WhNP is its filler; the long-distance link between the filler and the gap is encoded by the chain of slash categories from S down to VP/NP.

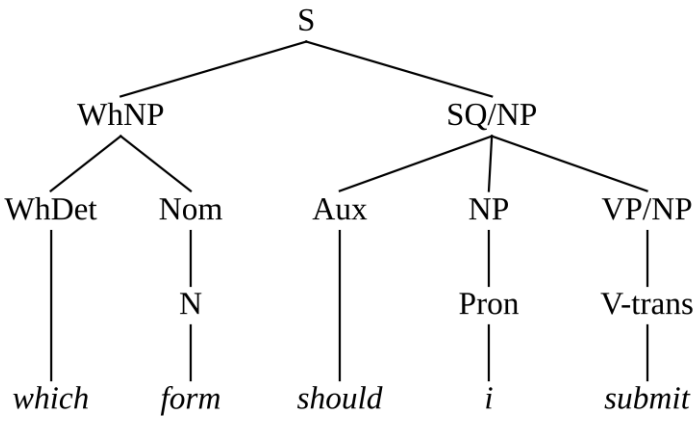


Figure 3.5 - Wh-determiner question: "Which form should I submit?"

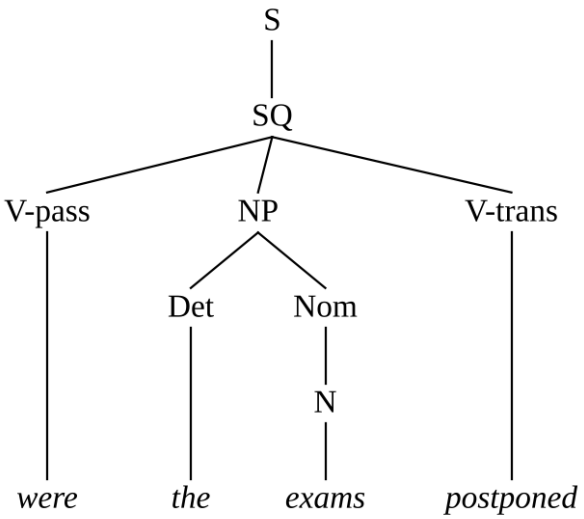


Figure 3.6 - Passive yes/no question: "Were the exams postponed?"

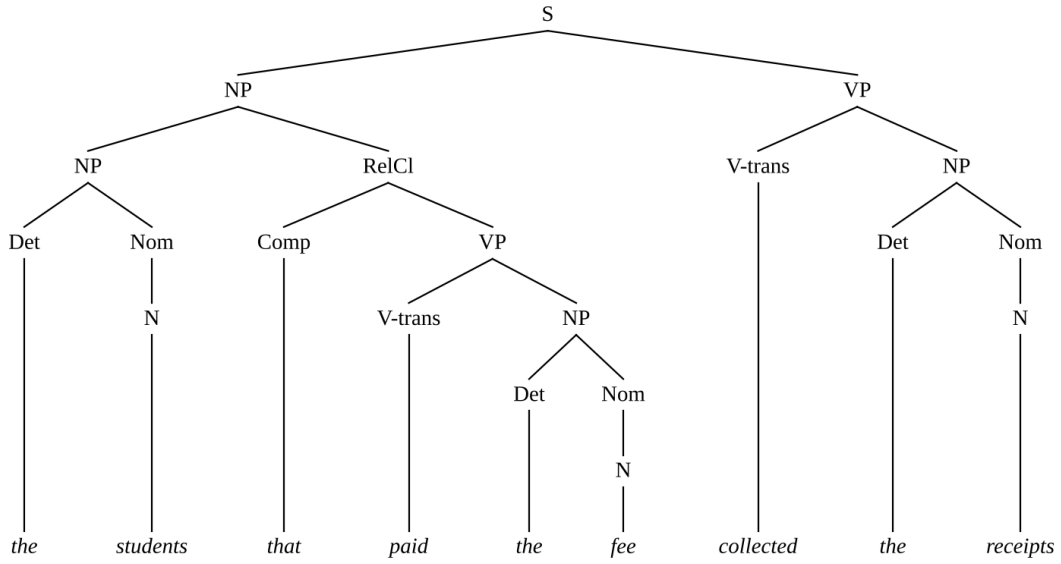


Figure 3.7 - Subject relative clause: "The students that paid the fee collected the receipts."

Difference. The subject noun phrase contains the relative clause RelCl -> Comp VP, which is a daughter of the NP and modifies the students; the relative clause has no subject of its own, because the relativized noun is the subject of paid.

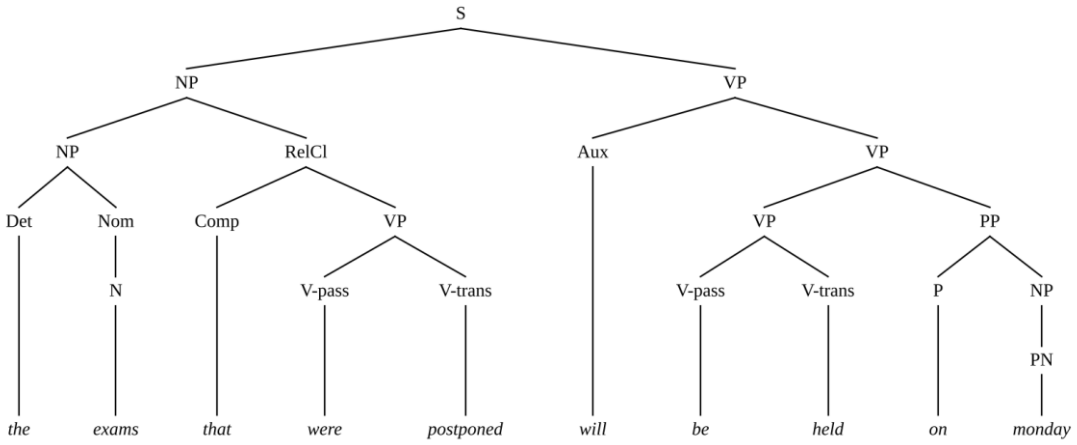


Figure 3.8 - Relative clause, passive and auxiliary: "The exams that were postponed will be held on Monday." (the analysis in which "on Monday" modifies "be held")

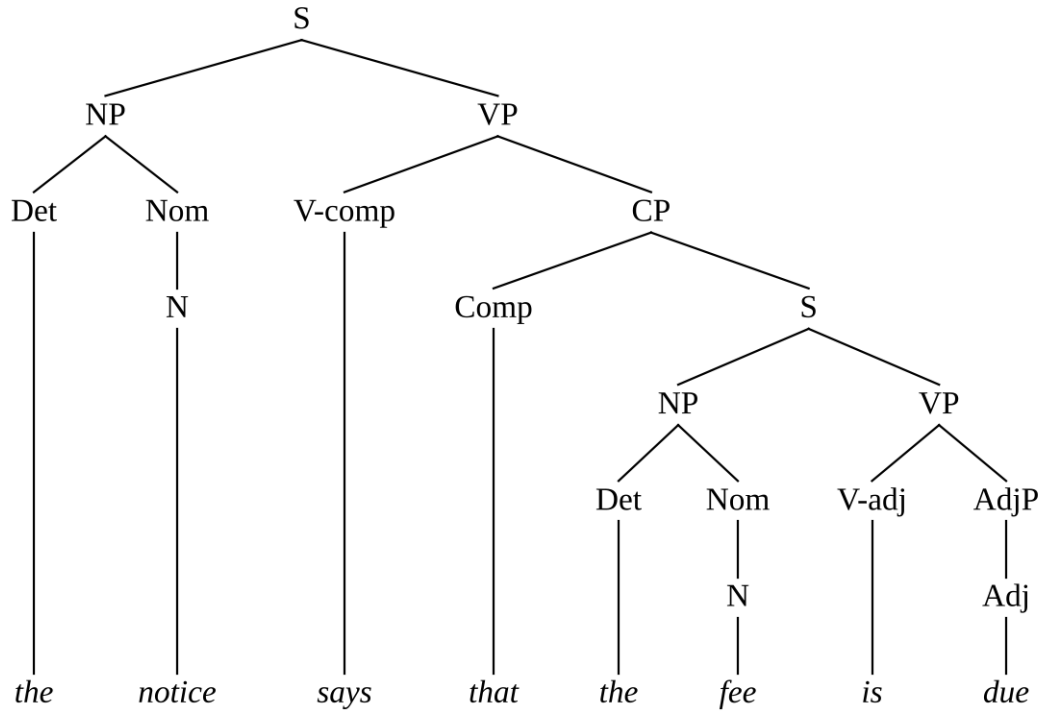


Figure 3.9 - Complement clause: "The notice says that the fee is due."

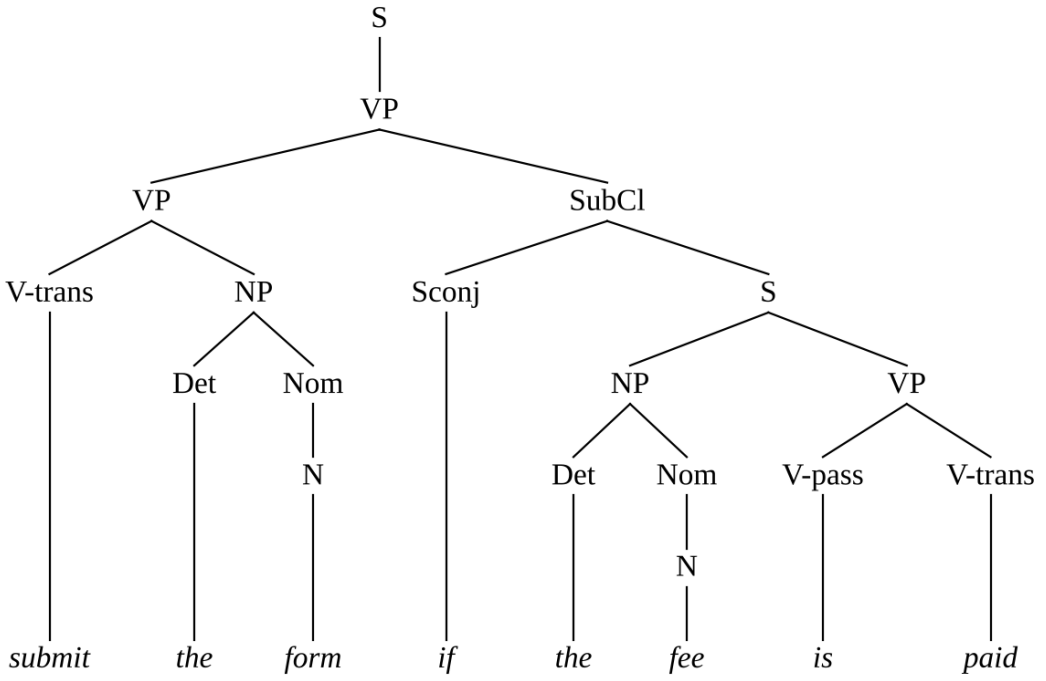


Figure 3.10 - Adverbial clause: "Submit the form if the fee is paid."

3.4 Coordination and long-distance dependencies

Coordination. Coordination is written with recursive rules of the form X -> X Conj X for X in {S, NP, Nom, VP, AdjP}. This handles fees and hostel charges, Check the timetable and renew the books and the clause coordination of Task 12 uniformly, and it can nest to any depth. Its price is **ambiguity of scope**: the rules do not say what is coordinated, so the fees and hostel charges gets two trees (Task 2) and Check the timetable and renew the books gets two (VP coordination below an imperative S, or coordination of two imperative clauses S -> S Conj S). In the query set, Q14 and Q19 contain this ambiguity, while Q8 (the forms and the fees) is unambiguous because each conjunct carries its own determiner. The coordinated conjuncts must also be like categories, a constraint that the plain CFG satisfies automatically because each rule coordinates the same category, but it cannot express that the conjuncts must share features: in The student paid the fee and submitted the form both verb phrases need the same finite form and the same subject agreement, which the feature grammar of Task 8 states by the equations in VP -> VP Conj VP.

Long-distance dependencies. The dependency between a filler and its gap can span any number of words and clauses (the form that the office said that the student submitted). In a CFG each position where a gap may occur needs its own slash rules, and each clause boundary a gap crosses needs the rules to pass the gap through (S/NP, VP/NP, CP/NP, SQ/NP, ...). The six gap rules included here handle one gap in a main-clause or relative-clause verb phrase. A grammar with full coverage would need a slash twin for every rule on every path between a filler and its possible gap sites, which roughly doubles the rule count, and with several gap types (NP gaps, PP gaps, adjective gaps) it grows multiplicatively. This is the main place where the plain CFG becomes unwieldy; feature-based approaches carry the gap in a single feature (SLASH) and avoid the duplication.

3.5 Effect of each construction on grammar size and ambiguity

To measure the cost of each construction, the grammar was built up in stages (the rule groups of Table 3.1 and Section 3.2) and each stage was run on the 30-sentence query set with the Earley parser. Table 3.2 reports the number of rules, the number of queries that receive at least one parse, the average number of parses of the covered queries and the average number of chart states per covered query. The lexicon is kept complete at every stage, so all differences come from the phrase-structure rules.

Table 3.2 - Grammar growth by stage (30 queries)

Stage (cumulative)	Rules	Queries covered	Avg. parses	Avg. states
Core declaratives (S -> NP VP, NP, Nom, basic VP frames, PP)	16	5 / 30	1.00	70
+ copula, adjective phrases, passive, auxiliaries	22	6 / 30	1.17	69
+ imperatives	23	13 / 30	2.23	98
+ yes/no questions (auxiliary inversion)	28	18 / 30	1.89	103
+ wh-questions and gap rules	39	23 / 30	1.70	104
+ subordinate and relative clauses	47	27 / 30	1.63	184
+ coordination	52	30 / 30	1.63	218

Reading the table. Each stage adds only a handful of rules (6 for the copula, passive and auxiliary group, one for imperatives, five for yes/no questions, eleven for wh-questions, eight for subordinate and relative clauses and five for coordination), yet coverage rises from 5 of 30 queries to all 30. The single imperative rule S -> VP raises coverage from 6 to 13 queries, because every imperative in the query set contains a bare verb phrase; at the same time it raises the average number of parses from 1.17 to 2.23, since the imperatives in the set are the sentences with the most PP-attachment ambiguity. The relative-clause and subordination stage nearly doubles the average chart size (from 104 to 184 states) because the new rules create left-recursive categories on NP and VP that must be predicted at many positions, and coordination adds a further 33 states on average. The conclusion is that grammar size grows roughly linearly with the number of constructions, but the chart size and the ambiguity depend on how many recursive rules are active: recursive modifier and coordination rules, not the question and clause types, are the main source of parsing cost.

Evidence summary for Task 3. Rules and trees were given for declaratives, imperatives, yes/no questions, subject and non-subject wh-questions, auxiliary inversion, relative clauses, complement and adverbial clauses; coordination and long-distance dependencies were analysed, and the effect of every group of rules on grammar size and ambiguity was measured.

Task 4 - Agreement

Guiding enquiry. How can a grammar prevent sentences such as "*The students submits the form" without producing an enormous set of rules?

4.1 Agreement phenomena in the fragment

Three agreement (and government) phenomena occur in the helpdesk fragment.

- **Subject-verb agreement (number and person).** A finite present-tense verb agrees with its subject: The student submits the form, The students submit the form, I submit the form. The two relevant classes are third-person singular (3sg: the student, he, she, it, this fee, who) and everything else (non3sg: plural nouns and the pronouns I, you, we, they). The form submitted is the same for both classes, and the auxiliaries will, can, should, did do not inflect. The copula (is/are, was/were) and the auxiliary do (do/does) also agree.
- **Determiner-noun agreement (number).** this and a and every require a singular noun and these and those a plural one, while the, my, our, some are neutral: this fee but *this fees, these fees but *these fee, the fee and the fees. The same agreement holds between the coordinators and the noun phrase: the fee and the charge is plural in agreement.
- **Pronoun case.** The subject of a finite clause must be nominative (I, he, she, we, they) and objects of verbs and prepositions must be accusative (me, him, us, them): He sent me the receipt but *Him sent I the receipt. The pronouns you and it have the same form in both cases.

4.2 First solution: multiplying the non-terminals

The first way to enforce agreement in a pure CFG is to split every category that carries a feature into one category per feature value. For number the rule S -> NP VP becomes two rules, S -> NP-sg VP-sg and S -> NP-pl VP-pl (we write -sg for 3sg and -pl for non3sg), and the noun phrase rule becomes NP-sg -> Det-sg Nom-sg and NP-pl -> Det-pl Nom-pl; the neutral determiner the has to be listed in both Det-sg and Det-pl. Every rule that mentions a category carrying the feature must be replicated for each consistent combination of values; rules whose categories are unrelated by agreement (such as PP -> P NP) are untouched. The splitting was carried out by a program (the function expand_grammar in Appendix A) that enumerates, for each rule, all assignments of feature values to the categories that carry the feature, subject to the equalities that the rule imposes. The counts in Table 4.1 are therefore exact.

Table 4.1 - Size of the plain CFG when agreement is enforced by multiplying categories

Grammar (cumulative)	Phrase-structure rules	Lexical rules	Rules relative to G
G (frames split, no agreement)	52	211	1.0 x
+ number (AGR: sg / non-3sg)	136	282	2.6 x
+ pronoun case	171	284	3.3 x
+ verb form (VFORM)	299	336	5.8 x
+ preposition form (PFORM)	3469	2216	66.7 x

Explanation of the growth. Splitting by number alone (AGR with two values) raises the phrase-structure rules from 52 to 136. The increase is more than a factor of two because rules with several agreeing categories (S -> NP VP; VP -> V NP with the verb and VP; VP -> VP Conj VP) multiply, and because categories that carry the feature but are not equated in a rule (the object NP in VP -> V-trans NP, which is free to be sg or pl) force every combination. Adding pronoun case splits NP into nominative and accusative versions and gives 171 rules. Adding the verb form (finite, base, participle) that is needed to reject The student submit the form and will submits brings the count to 299. Finally, listing the preposition form on the PP (needed to reject Apply in a hostel room in Task 5) multiplies the PP and the PP-selecting frames by the eleven prepositions and gives 3469 rules. The lexicon grows too, because every underspecified word (the, my, our, some, staff, you, it, submitted) must be listed once for each value it is compatible with.

4.3 The worst case: number, person, case and gender combined

The count above uses the two-valued agreement class of the helpdesk fragment. A more complete treatment keeps number (2 values: sg, pl), person (3 values: 1, 2, 3), case (2 values: nom, acc) and, for reflexive and pronoun coreference, gender (3 values: masculine, feminine, neuter) as independent features. In the worst case every rule that contains an agreement-carrying category is replicated once per combination of values. The carrying categories in G are Aux, Conj, Det, N, NP, Nom, PN, Pron, RelCl, V, VP, VP/NP, WhNP, WhPron. Of the 52 phrase-structure rules, 45 contain at least one carrying category and 7 do not. The worst-case numbers are therefore:

Table 4.2 - Worst-case rule counts when every feature combination needs its own rules

Features multiplied	Combinations per rule	Phrase-structure rules	Lexical rules (upper bound)	Total (upper bound)
number	2	97	276	373
number x person	6	277	828	1105
number x person x case	12	547	1656	2203
number x person x case x gender	36	1627	4968	6595

The formula is: worst-case phrase rules = r_a x (|NUM| x |PER| x |CASE| x |GEND|) + (r - r_a), where r = 52 and r_a = 45; with 2 x 3 x 2 x 3 = 36 combinations this gives 1627 rules. The lexical bound is 138 x 36 = 4968 entries, since each of the 138 (word, category) pairs of agreement-carrying categories may need up to one entry per combination. So the multiplication approach grows by a factor of about 31 in the phrase rules and the growth is multiplicative in the number of features, which makes it unmaintainable; the true counts are smaller because many words fix some of the features, but the trend is the point.

4.4 The economical alternative: feature-based rules

The alternative is to keep one rule per construction and to attach constraints on features. The same sentence rule becomes S -> NP VP with the equation <NP AGR> = <VP AGR>, and the noun phrase rule NP -> Det Nom with <Det AGR> = <Nom AGR>. Each lexical entry carries its own feature values: student has AGR = 3sg, students has AGR = non3sg, submits has AGR = 3sg, submit has AGR = non3sg (as a finite verb), submitted leaves AGR unspecified, the leaves it unspecified, and this has AGR = 3sg. A constraint equation succeeds when the two values are equal or when one of them is unspecified, and it fails when the values conflict. Informally:

- The students submits the form: <NP AGR> = non3sg (from students) and <VP AGR> = 3sg (from submits) conflict, so the rule S -> NP VP cannot be applied and no tree is built.
- These student submitted the form: <Det AGR> = non3sg (from these) and <Nom AGR> = 3sg (from student) conflict in NP -> Det Nom.
- Him submitted the form: the rule S -> NP VP requires <NP CASE> = nom, but him has CASE = acc.

- The students submitted the form: <NP AGR> = non3sg and <VP AGR> is unspecified (past tense), so the equation holds and the tree is built.

The number of rules is unchanged (52); only the lexicon is annotated. The formal definition of the feature structures and of unification follows in Task 8, where the feature grammar is also compared in size with the multiplied grammars.

4.5 Testing the baseline and the agreement-aware grammars

Twelve sentences were tested: six grammatical and six ungrammatical. Four grammars were run with the Earley parser: (1) the baseline G, with categories split by frame only and no agreement; (2) the grammar with number and case multiplied (171 rules); (3) the same plus the verb form (299 rules); and (4) the feature grammar G_F (52 rules with constraints).

Table 4.3 - Acceptance of grammatical (G) and ungrammatical (*) test sentences

Sentence	G baseline	G + number, case	+ verb form	Feature grammar G_F
The student submitted the form.	accepted	accepted	accepted	accepted
The students submit the forms.	accepted	accepted	accepted	accepted
These students paid the fees.	accepted	accepted	accepted	accepted
He sent me the receipt.	accepted	accepted	accepted	accepted
They collect the receipts.	accepted	accepted	accepted	accepted
This fee is due.	accepted	accepted	accepted	accepted
* The students submits the form.	accepted	rejected	rejected	rejected
* The student submit the form.	accepted	accepted	rejected	rejected
* These student submitted the form.	accepted	rejected	rejected	rejected
* A students paid the fees.	accepted	rejected	rejected	rejected
* Him submitted the form.	accepted	rejected	rejected	rejected
* Send I the receipt.	accepted	rejected	rejected	rejected

Results. All four grammars accept the six grammatical sentences. The baseline G accepts all six ungrammatical sentences (6 of 6 false accepts), as expected, because it has no agreement. The grammar with number and case multiplied rejects 5 of 6; the one sentence it still accepts, The student submit the form, is accepted because the base form submit is also a legal verb form for imperatives and infinitives and is listed with both number classes; to exclude it the grammar needs the verb-form split. With the verb form added (299 rules), 6 of 6 are rejected, and the

feature grammar G_F rejects 6 of 6 with only 52 rules. This matches the analysis: the feature grammar achieves the discrimination of the largest multiplied grammar with about one sixth of its rules, and G_F has the capacity to add the preposition-form constraint at no cost in rules, whereas the multiplied version would need 3469.

Known limitation. Because the helpdesk grammar uses the two-valued class AGR (3sg versus non3sg), it cannot separate I from you or we and therefore accepts *I are and *you is wrongly in the copula case (the class of are is non3sg, as is the class of I, while is is 3sg, so *I is is rejected but *I are is accepted). Splitting AGR into NUM and PER with disjunctive lexical entries repairs this at the cost of the multiplication shown in Table 4.2; none of the test sentences exercises this case.

Evidence summary for Task 4. The agreement phenomena were identified; the multiplication approach was implemented and its rule count measured exactly (52 to 171 for number and case, 299 with verb form, 3469 with preposition form); the worst case was computed (1627 phrase rules for 36 combinations); the feature-based alternative was introduced and tested on twelve sentences.

Task 5 - Subcategorization

Guiding enquiry. How does the grammar know which complements a verb may take, and how can this knowledge be encoded?

5.1 Subcategorization frames

Subcategorization (valence) is the property of a word, usually a verb, that determines which complements it requires. The verb submit requires a noun phrase (Submit the form) and cannot appear alone; crash takes no complement (The portal crashed) and cannot take a noun phrase object; apply takes a PP headed by for. A **frame** is the list of complements: NP for the transitive frame, NP NP for the ditransitive frame and so on. The helpdesk fragment uses nine frames.

Table 5.1 - Subcategorization frames of the helpdesk verbs

Frame	Category	Rule	Example	Verbs (base forms)
INTR	V-intr	VP → V-intr	The portal crashed.	crash, start, close, pay
NP	V-trans	VP → V-trans NP	Submit the form.	close, submit, check, pay, send, show, tell, book, postpone, collect, renew, hold, publish, be
NP + NP	V-ditrans	VP → V-ditrans NP NP	Send me the receipt.	send, show, book
CP	V-comp	VP → V-comp CP	The notice says that the fee is due.	check, say
NP + CP	V-trcomp	VP → V-trcomp NP CP	Tell me that the fee is paid.	tell
PP	V-pp	VP → V-pp PP	Apply for a hostel room.	apply, register, wait, be
NP + PP	V-trpp	VP → V-trpp NP PP	Send the form to the office.	submit, send
ADJ	V-adj	VP → V-adj AdjP	The fee is due.	be
PASS	V-pass	VP → V-pass V-trans	The exams were postponed.	be

Several verbs have more than one frame, for example send (transitive, ditransitive and NP + PP: Send the receipt, Send me the receipt, Send the form to the office), close (intransitive and transitive), tell (transitive and NP + clause) and check (transitive and clausal). The copula be is listed with four frames (adjective, noun phrase, PP and passive).

5.2 First encoding: distinct verb categories, and its drawbacks

In the plain grammar G (Task 1) each frame has its own preterminal (V-intr, V-trans, V-ditrans, V-comp, V-trcomp, V-pp, V-trpp, V-adj, V-pass) and each VP rule mentions exactly one of them. A verb is listed in every category whose frame it can take. This works and is easy to understand, but it has four drawbacks.

4. **Duplicated lexical entries.** A verb with k frames appears in k categories, with k copies of every inflected form. Of the 19 main-verb lemmas, eight have two or three frames; the copula has four.
5. **Multiplication with other features.** The frame category is crossed with number, verb form and preposition form. The nine frame categories become 18 with number alone (V-trans-sg, V-trans-pl, ...) and up to 54 with number and the three verb forms, and each of them needs its own VP rules and lexical entries (Table 4.1).
6. **Lost generalisations.** The passive, the gap rules and the question rules must each repeat the category names. The rule VP -> V-pass V-trans mentions V-trans because only transitive verbs passivise; if a new verb frame is added, every rule that quantifies over frames has to be updated.
7. **No way to state selection of specific prepositions.** apply for versus send to is a property of the pair verb-preposition. With plain categories the only option is separate categories (V-pp-for, V-trpp-to, ...) or accepting the over-generation of *Apply in a hostel room.

The feature-based alternative of Task 8 uses a single V category with a SUBCAT feature and one rule per frame in which an equation selects the frame (for example VP -> V NP with <V SUBCAT> = NP), plus a PFORM feature for the selected preposition.

5.3 Complements versus adjuncts

The grammar treats complements as part of the frame (selected by the verb) and adjuncts as optional modifiers (VP -> VP PP). Four tests decide which a given phrase is.

- **Obligatoriness.** A complement cannot normally be omitted: Apply for a hostel room but *Apply; Submit the form but *Submit. An adjunct can: The student submitted the form (on Friday).
- **Iterability.** Adjuncts can be repeated and stacked, complements cannot: The student submitted the form on Friday at the office after the seminar (three adjuncts) but *Submit the form the receipt the notice has no corresponding stacking.

- **Selection.** The verb restricts the form of a complement (apply for, not *apply in), whereas adjuncts are freely chosen: Submit the form on / near / after / before ....
- **Do so substitution.** do so replaces the verb and its complements but can leave an adjunct behind: The student submitted the form on Friday and the staff did so on Monday (did so = submitted the form). *The student submitted the form and the staff did so the receipt is bad because the complement NP cannot be left behind.

Applied to Submit the form to the office the tests give a genuine ambiguity of analysis. Submit has the frame NP + PP[to] (as in Send the form to the office, where the PP is a complement because send selects to), and it also has the plain transitive frame, so to the office can be an adjunct. The Earley parser returns 3 trees: (1) (S (VP (V-trpp submit) (NP (Det the) (Nom (N form))) (PP (P to) (NP (Det the) (Nom (N office)))))); (2) (S (VP (VP (V-trans submit) (NP (Det the) (Nom (N form)))) (PP (P to) (NP (Det the) (Nom (N office)))))); (3) (S (VP (V-trans submit) (NP (NP (Det the) (Nom (N form))) (PP (P to) (NP (Det the) (Nom (N office))))))). The first reading takes the frame, the second modifies the verb phrase and the third modifies the noun phrase. A grammar cannot decide among these, which is a pointer to Task 9.

5.4 Rejecting over- and under-complemented sentences

Table 5.2 shows six acceptable and six unacceptable sentences, tested with (1) the skeleton grammar in which all constraints are removed and V is a single category, (2) the plain grammar G with frame categories, and (3) the feature grammar G_F. The unacceptable examples include under-complemented sentences (Submit, The student sent), over-complemented sentences (The portal crashed the form), a wrong frame (Apply the form), a missing complementizer (Tell me the fee is paid) and a wrongly selected preposition (Apply in a hostel room).

Table 5.2 - Accept/reject results for subcategorization (parses found)

Sentence	Skeleton (single V)	Frame categories G	Feature grammar G_F
The portal crashed.	accepted	accepted	accepted
Submit the form.	accepted	accepted	accepted
Send me the receipt.	accepted	accepted	accepted
Tell me that the fee is paid.	accepted	accepted	accepted
Apply for a hostel room.	accepted	accepted	accepted
Send the form to the office.	accepted	accepted	accepted
* Submit.	accepted	rejected	rejected
* The portal crashed the form.	accepted	rejected	rejected
* The student sent.	accepted	rejected	rejected

Sentence	Skeleton (single V)	Frame categories G	Feature grammar G_F
* Apply the form.	accepted	rejected	rejected
* Tell me the fee is paid.	rejected	rejected	rejected
* Apply in a hostel room.	accepted	accepted	rejected

The skeleton grammar accepts every sentence except the one with the missing complementizer, which is excluded by the CP rule itself, so subcategorization checking is entirely the work of the frame categories or of the SUBCAT constraints. The frame categories of G reject five of the six bad sentences. They accept Apply in a hostel room, because V-pp does not record the preposition; the feature grammar G_F rejects it, because the equation <V PFORM> = <PP PFORM> compares for with in. G_F rejects all six, accepts all six good sentences and does so with the parse output of zero trees for the rejected ones (shown in the experiments of Task 10).

5.5 Verbs with multiple frames: ambiguity and parsing cost

Multiple frames create **lexical ambiguity** (the word send has three entries) and, through the interaction with adjunct rules, **structural ambiguity**. Send the form to the office has 3 parses (frame NP + PP, or frame NP with PP adjunct to the VP, or to the NP), whereas Apply for a hostel room has 1 parse because apply has a single frame. A verb with k frames contributes up to k entries to every position where it occurs, and each entry starts its own predictions in a chart parser; in a top-down parser each frame is a separate alternative to try and to backtrack over. In the measurements of Task 10, the query with the most ambiguity from this source (Q18, "Send the form to the office in the library") has 7 parses and 323 Earley states, while the single-frame query "Apply for a hostel room" has one parse and 198 states; the top-down parser needs 246,791 steps for Q18 and 1,433 steps for Q12 (the single-frame query of the same kind). The practical consequence is that the number of frames per verb should be kept as small as the data allow, and that frame choice must be left to a statistical component for ranking.

Evidence summary for Task 5. Subcategorization was defined and nine frames listed with verbs and rules; the category-based encoding and its four drawbacks were given; complements and adjuncts were distinguished by four tests; and twelve accept/reject cases and the effect of multiple frames on ambiguity and cost were reported with measured parse results.

Task 6 - Top-Down Parsing

Guiding enquiry. How does a top-down parser search for a tree rooted in S, and what are the strengths and weaknesses of this strategy?

6.1 The algorithm

A top-down (goal-driven, recursive-descent) parser starts from the goal S and tries to rewrite it into the input. It keeps an agenda (a stack) of symbols still to be recognised. If the first symbol on the agenda is a non-terminal A, the parser chooses a rule A -> X1 ... Xm, replaces A by X1 ... Xm and continues; if it is a part of speech, the parser checks whether the next input word can have that category and, if so, consumes the word. If no rule applies or a word does not match, the parser backtracks to the most recent choice point and tries the next alternative. A parse is found when the agenda and the input are both empty. The generator-based pseudocode below enumerates all parses; the two filters in the second step are optional and are explained in Section 6.4.

```
function TOPDOWN-PARSE(words):
   for each (tree, j) in PARSE-CATEGORY(S, 0, pending = 0):
      if j = length(words): report tree          // the whole input is consumed

function PARSE-CATEGORY(A, i, pending):
   if A is a part of speech:
      if i < n and A is a category of words[i]:  // match a word
         yield (leaf(A, words[i]), i + 1)
      return
   for each rule A -> X1 ... Xm in grammar order:          // expand
      if FILTER:                                  // optional filters
         if words[i] cannot begin X1: continue          //   left-corner filter
         if n - i < m + pending: continue               //   length filter
      for each (children, j) in PARSE-SEQUENCE([X1..Xm], i, pending):
         yield (node(A, children), j)

function PARSE-SEQUENCE([X1..Xm], i, pending):
   if m = 0: yield ([], i); return
   for each (t1, j1) in PARSE-CATEGORY(X1, i, pending + m - 1):
      for each (rest, j) in PARSE-SEQUENCE([X2..Xm], j1, pending):
         yield ([t1] + rest, j)
// Backtracking is implicit: when the caller rejects a result, the generator
// resumes and tries the next rule or the next way of parsing a daughter.
```

For the hand traces below a seven-rule sub-grammar G_mini of the helpdesk grammar is used, because the full grammar would produce traces of thousands of steps. G_mini consists of the rules S -> NP VP, NP -> Det N, NP -> NP PP, NP -> PN, VP -> V NP, VP -> VP PP and PP -

> P NP (in this order) and the lexicon Det = {the}, N = {student, form, notice}, V = {submitted}, P = {on, with}, PN = {friday}. Each row of a trace shows the agenda before the action, the unconsumed input and the action taken.

6.2 Trace on an unambiguous sentence

The sentence is "the student submitted the form". The first parse is found at step 10. The remaining steps explore the other alternatives to prove that there is no second parse; the length filter prunes the left-recursive alternatives, which is explained in Section 6.4.

Table 6.1 - Top-down trace for "the student submitted the form" (all parses searched)

Step	Agenda (stack)	Remaining input	Action
1	S	the student submitted the form	expand S -> NP VP
2	NP VP	the student submitted the form	expand NP -> Det N
3	Det N VP	the student submitted the form	match Det = 'the'
4	N VP	student submitted the form	match N = 'student'
5	VP	submitted the form	expand VP -> V NP
6	V NP	submitted the form	match V = 'submitted'
7	NP	the form	expand NP -> Det N
8	Det N	the form	match Det = 'the'
9	N	form	match N = 'form'
10	(empty)	(empty)	SUCCESS: parse found
11	NP	the form	BACKTRACK to NP: try NP -> NP PP
12	NP PP	the form	expand NP -> Det N, PRUNE: 3 symbols pending > 2 words left
13	NP PP	the form	BACKTRACK to NP: try NP -> NP PP, PRUNE: 3 symbols pending > 2 words left
14	NP PP	the form	BACKTRACK to NP: try NP -> PN
15	PN PP	the form	FAIL: PN cannot match 'the'
16	NP	the form	BACKTRACK to NP: try NP -> PN
17	PN	the form	FAIL: PN cannot match 'the'
18	VP	submitted the form	BACKTRACK to VP: try VP -> VP PP

Step	Agenda (stack)	Remaining input	Action
19	VP PP	submitted the form	expand VP -> V NP
20	V NP PP	submitted the form	match V = 'submitted'
21	NP PP	the form	expand NP -> Det N, PRUNE: 3 symbols pending > 2 words left
22	NP PP	the form	BACKTRACK to NP: try NP -> NP PP, PRUNE: 3 symbols pending > 2 words left
23	NP PP	the form	BACKTRACK to NP: try NP -> PN
24	PN PP	the form	FAIL: PN cannot match 'the'
25	VP PP	submitted the form	BACKTRACK to VP: try VP -> VP PP
26	VP PP PP	submitted the form	expand VP -> V NP, PRUNE: 4 symbols pending > 3 words left
27	VP PP PP	submitted the form	BACKTRACK to VP: try VP -> VP PP, PRUNE: 4 symbols pending > 3 words left
28	NP VP	the student submitted the form	BACKTRACK to NP: try NP -> NP PP
29	NP PP VP	the student submitted the form	expand NP -> Det N
30	Det N PP VP	the student submitted the form	match Det = 'the'
31	N PP VP	student submitted the form	match N = 'student'
32	PP VP	submitted the form	expand PP -> P NP
33	P NP VP	submitted the form	FAIL: P cannot match 'submitted'
34	NP PP VP	the student submitted the form	BACKTRACK to NP: try NP -> NP PP
35	NP PP PP VP	the student submitted the form	expand NP -> Det N
36	Det N PP PP VP	the student submitted the form	match Det = 'the'
37	N PP PP VP	student submitted the form	match N = 'student'
38	PP PP VP	submitted the form	expand PP -> P NP, PRUNE: 4 symbols pending > 3 words left
39	NP PP PP VP	the student submitted the form	BACKTRACK to NP: try NP -> NP PP
40	NP PP PP PP VP	the student submitted the form	expand NP -> Det N, PRUNE: 6 symbols pending > 5 words left

Step	Agenda (stack)	Remaining input	Action
41	NP PP PP PP VP	the student submitted the form	BACKTRACK to NP: try NP -> NP PP, PRUNE: 6 symbols pending > 5 words left
42	NP PP PP PP VP	the student submitted the form	BACKTRACK to NP: try NP -> PN
43	PN PP PP PP VP	the student submitted the form	FAIL: PN cannot match 'the'
44	NP PP PP VP	the student submitted the form	BACKTRACK to NP: try NP -> PN
45	PN PP PP VP	the student submitted the form	FAIL: PN cannot match 'the'
46	NP PP VP	the student submitted the form	BACKTRACK to NP: try NP -> PN
47	PN PP VP	the student submitted the form	FAIL: PN cannot match 'the'
48	NP VP	the student submitted the form	BACKTRACK to NP: try NP -> PN
49	PN VP	the student submitted the form	FAIL: PN cannot match 'the'
50	PN VP	the student submitted the form	no alternatives left: search ends

The trace has 50 steps, of which 17 are backtracking points. The choice points are the alternative rules for NP (NP -> NP PP and NP -> PN after NP -> Det N) and for VP (VP -> VP PP after VP -> V NP); each of them is rejected without finding a second parse.

6.3 Trace on an ambiguous sentence

The sentence "the student submitted the form on friday" has two parses (the PP on friday modifies either the verb phrase submitted the form or the noun phrase the form). The trace is the same search and shows the first parse at the first success, the backtracking to the alternative that attaches the PP at a different place and the second parse.

Table 6.2 - Top-down trace for "the student submitted the form on friday"

Step	Agenda (stack)	Remaining input	Action
1	S	the student submitted the form on friday	expand S -> NP VP
2	NP VP	the student submitted the form on friday	expand NP -> Det N
3	Det N VP	the student submitted the form on friday	match Det = 'the'

Step	Agenda (stack)	Remaining input	Action
4	N VP	student submitted the form on friday	match N = 'student'
5	VP	submitted the form on friday	expand VP -> V NP
6	V NP	submitted the form on friday	match V = 'submitted'
7	NP	the form on friday	expand NP -> Det N
8	Det N	the form on friday	match Det = 'the'
9	N	form on friday	match N = 'form'
10	(empty)	on friday	FAIL: input left over
11	NP	the form on friday	BACKTRACK to NP: try NP -> NP PP
12	NP PP	the form on friday	expand NP -> Det N
13	Det N PP	the form on friday	match Det = 'the'
14	N PP	form on friday	match N = 'form'
15	PP	on friday	expand PP -> P NP
16	P NP	on friday	match P = 'on'
17	NP	friday	expand NP -> Det N, PRUNE: 2 symbols pending > 1 words left
18	NP	friday	BACKTRACK to NP: try NP -> NP PP, PRUNE: 2 symbols pending > 1 words left
19	NP	friday	BACKTRACK to NP: try NP -> PN
20	PN	friday	match PN = 'friday'
21	(empty)	(empty)	SUCCESS: parse found
22	NP PP	the form on friday	BACKTRACK to NP: try NP -> NP PP
23	NP PP PP	the form on friday	expand NP -> Det N
24	Det N PP PP	the form on friday	match Det = 'the'
25	N PP PP	form on friday	match N = 'form'
26	PP PP	on friday	expand PP -> P NP, PRUNE: 3 symbols pending > 2 words left
27	NP PP PP	the form on friday	BACKTRACK to NP: try NP -> NP PP
28	NP PP PP PP	the form on friday	expand NP -> Det N, PRUNE: 5 symbols pending > 4 words left
29	NP PP PP PP	the form on friday	BACKTRACK to NP: try NP -> NP PP, PRUNE: 5 symbols pending > 4 words left

Step	Agenda (stack)	Remaining input	Action
30	NP PP PP PP	the form on friday	BACKTRACK to NP: try NP -> PN
31	PN PP PP PP	the form on friday	FAIL: PN cannot match 'the'
32	NP PP PP	the form on friday	BACKTRACK to NP: try NP -> PN
33	PN PP PP	the form on friday	FAIL: PN cannot match 'the'
34	NP PP	the form on friday	BACKTRACK to NP: try NP -> PN
35	PN PP	the form on friday	FAIL: PN cannot match 'the'
36	NP	the form on friday	BACKTRACK to NP: try NP -> PN
37	PN	the form on friday	FAIL: PN cannot match 'the'
38	VP	submitted the form on friday	BACKTRACK to VP: try VP -> VP PP
39	VP PP	submitted the form on friday	expand VP -> V NP
40	V NP PP	submitted the form on friday	match V = 'submitted'
41	NP PP	the form on friday	expand NP -> Det N
42	Det N PP	the form on friday	match Det = 'the'
43	N PP	form on friday	match N = 'form'
44	PP	on friday	expand PP -> P NP
45	P NP	on friday	match P = 'on'
46	NP	friday	expand NP -> Det N, PRUNE: 2 symbols pending > 1 words left
47	NP	friday	BACKTRACK to NP: try NP -> NP PP, PRUNE: 2 symbols pending > 1 words left
48	NP	friday	BACKTRACK to NP: try NP -> PN
49	PN	friday	match PN = 'friday'
50	(empty)	(empty)	SUCCESS: parse found
51	NP PP	the form on friday	BACKTRACK to NP: try NP -> NP PP
52	NP PP PP	the form on friday	expand NP -> Det N
53	Det N PP PP	the form on friday	match Det = 'the'
54	N PP PP	form on friday	match N = 'form'
55	PP PP	on friday	expand PP -> P NP, PRUNE: 3 symbols pending > 2 words left

Step	Agenda (stack)	Remaining input	Action
56	NP PP PP	the form on friday	BACKTRACK to NP: try NP -> NP PP
57	NP PP PP PP	the form on friday	expand NP -> Det N, PRUNE: 5 symbols pending > 4 words left
58	NP PP PP PP	the form on friday	BACKTRACK to NP: try NP -> NP PP, PRUNE: 5 symbols pending > 4 words left
59	NP PP PP PP	the form on friday	BACKTRACK to NP: try NP -> PN
60	PN PP PP PP	the form on friday	FAIL: PN cannot match 'the'
61	NP PP PP	the form on friday	BACKTRACK to NP: try NP -> PN
62	PN PP PP	the form on friday	FAIL: PN cannot match 'the'
63	NP PP	the form on friday	BACKTRACK to NP: try NP -> PN
64	PN PP	the form on friday	FAIL: PN cannot match 'the'
65	VP PP	submitted the form on friday	BACKTRACK to VP: try VP -> VP PP
66	VP PP PP	submitted the form on friday	expand VP -> V NP
67	V NP PP PP	submitted the form on friday	match V = 'submitted'
68	NP PP PP	the form on friday	expand NP -> Det N
69	Det N PP PP	the form on friday	match Det = 'the'
70	N PP PP	form on friday	match N = 'form'
71	PP PP	on friday	expand PP -> P NP, PRUNE: 3 symbols pending > 2 words left
72	NP PP PP	the form on friday	BACKTRACK to NP: try NP -> NP PP
73	NP PP PP PP	the form on friday	expand NP -> Det N, PRUNE: 5 symbols pending > 4 words left
74	NP PP PP PP	the form on friday	BACKTRACK to NP: try NP -> NP PP, PRUNE: 5 symbols pending > 4 words left
75	NP PP PP PP	the form on friday	BACKTRACK to NP: try NP -> PN
76	PN PP PP PP	the form on friday	FAIL: PN cannot match 'the'
77	NP PP PP	the form on friday	BACKTRACK to NP: try NP -> PN
78	PN PP PP	the form on friday	FAIL: PN cannot match 'the'

Step	Agenda (stack)	Remaining input	Action
79	VP PP PP	submitted the form on friday	BACKTRACK to VP: try VP -> VP PP
80	VP PP PP PP	submitted the form on friday	expand VP -> V NP
81	V NP PP PP PP	submitted the form on friday	match V = 'submitted'
82	NP PP PP PP	the form on friday	expand NP -> Det N, PRUNE: 5 symbols pending > 4 words left
83	NP PP PP PP	the form on friday	BACKTRACK to NP: try NP -> NP PP, PRUNE: 5 symbols pending > 4 words left
84	NP PP PP PP	the form on friday	BACKTRACK to NP: try NP -> PN
85	PN PP PP PP	the form on friday	FAIL: PN cannot match 'the'
86	VP PP PP PP	submitted the form on friday	BACKTRACK to VP: try VP -> VP PP
87	VP PP PP PP PP	submitted the form on friday	expand VP -> V NP, PRUNE: 6 symbols pending > 5 words left
88	VP PP PP PP PP	submitted the form on friday	BACKTRACK to VP: try VP -> VP PP, PRUNE: 6 symbols pending > 5 words left
89	NP VP	the student submitted the form on friday	BACKTRACK to NP: try NP -> NP PP
90	NP PP VP	the student submitted the form on friday	expand NP -> Det N
91	Det N PP VP	the student submitted the form on friday	match Det = 'the'
92	N PP VP	student submitted the form on friday	match N = 'student'
93	PP VP	submitted the form on friday	expand PP -> P NP
94	P NP VP	submitted the form on friday	FAIL: P cannot match 'submitted'
95	NP PP VP	the student submitted the form on friday	BACKTRACK to NP: try NP -> NP PP
96	NP PP PP VP	the student submitted the form on friday	expand NP -> Det N
97	Det N PP PP VP	the student submitted the form on friday	match Det = 'the'
98	N PP PP VP	student submitted the form on friday	match N = 'student'

Step	Agenda (stack)	Remaining input	Action
99	PP PP VP	submitted the form on friday	expand PP -> P NP
100	P NP PP VP	submitted the form on friday	FAIL: P cannot match 'submitted'
101	NP PP PP VP	the student submitted the form on friday	BACKTRACK to NP: try NP -> NP PP
102	NP PP PP PP VP	the student submitted the form on friday	expand NP -> Det N
103	Det N PP PP PP VP	the student submitted the form on friday	match Det = 'the'
104	N PP PP PP VP	student submitted the form on friday	match N = 'student'
105	PP PP PP VP	submitted the form on friday	expand PP -> P NP
106	P NP PP PP VP	submitted the form on friday	FAIL: P cannot match 'submitted'
107	NP PP PP PP VP	the student submitted the form on friday	BACKTRACK to NP: try NP -> NP PP
108	NP PP PP PP PP VP	the student submitted the form on friday	expand NP -> Det N
109	Det N PP PP PP PP VP	the student submitted the form on friday	match Det = 'the'
110	N PP PP PP PP VP	student submitted the form on friday	match N = 'student'
111	PP PP PP PP VP	submitted the form on friday	expand PP -> P NP, PRUNE: 6 symbols pending > 5 words left
112	NP PP PP PP PP VP	the student submitted the form on friday	BACKTRACK to NP: try NP -> NP PP
113	NP PP PP PP PP PP VP	the student submitted the form on friday	expand NP -> Det N, PRUNE: 8 symbols pending > 7 words left
114	NP PP PP PP PP PP VP	the student submitted the form on friday	BACKTRACK to NP: try NP -> NP PP, PRUNE: 8 symbols pending > 7 words left
115	NP PP PP PP PP PP VP	the student submitted the form on friday	BACKTRACK to NP: try NP -> PN
116	PN PP PP PP PP PP VP	the student submitted the form on friday	FAIL: PN cannot match 'the'
117	NP PP PP PP PP VP	the student submitted the form on friday	BACKTRACK to NP: try NP -> PN
118	PN PP PP PP PP VP	the student submitted the form on friday	FAIL: PN cannot match 'the'

Step	Agenda (stack)	Remaining input	Action
119	NP PP PP PP VP	the student submitted the form on friday	BACKTRACK to NP: try NP -> PN
120	PN PP PP PP VP	the student submitted the form on friday	FAIL: PN cannot match 'the'
121	NP PP PP VP	the student submitted the form on friday	BACKTRACK to NP: try NP -> PN
122	PN PP PP VP	the student submitted the form on friday	FAIL: PN cannot match 'the'
123	NP PP VP	the student submitted the form on friday	BACKTRACK to NP: try NP -> PN
124	PN PP VP	the student submitted the form on friday	FAIL: PN cannot match 'the'
125	NP VP	the student submitted the form on friday	BACKTRACK to NP: try NP -> PN
126	PN VP	the student submitted the form on friday	FAIL: PN cannot match 'the'
127	PN VP	the student submitted the form on friday	no alternatives left: search ends

The two parses are found at steps 21 and 50 of 127. The noun phrase the form is built from scratch 6 times (the word form is matched 6 times) and the determiner the is matched 11 times in total, although the sentence contains it only twice. This is the repeated re-parsing problem: the parser has no memory of the sub-parses that it has already built, so every alternative that needs the form at the same position builds it again.

6.4 Left recursion and repeated sub-parses

Left recursion

A rule is left-recursive if its first right-hand-side symbol is the left-hand side, as in NP -> NP PP. The top-down parser expands the goal NP into NP PP, then the first NP into NP PP again and so on, without ever consuming a word. The trace below puts the left-recursive rule first in G_mini and shows the agenda growing at each step.

Table 6.3 - Left recursion: the agenda grows without consuming input

Step	Agenda (stack)	Remaining input	Action
1	S	the student submitted the form	expand S -> NP VP
2	NP VP	the student submitted the form	expand NP -> NP PP

Step	Agenda (stack)	Remaining input	Action
3	NP PP VP	the student submitted the form	expand NP -> NP PP
4	NP PP PP VP	the student submitted the form	expand NP -> NP PP
5	NP PP PP PP VP	the student submitted the form	expand NP -> NP PP
6	NP PP PP PP PP VP	the student submitted the form	expand NP -> NP PP
7	NP PP PP PP PP PP VP	the student submitted the form	expand NP -> NP PP
8	NP PP PP PP PP PP PP VP	the student submitted the form	expand NP -> NP PP
9	NP PP PP PP PP PP PP PP VP	the student submitted the form	expand NP -> NP PP

The input never advances, so the agenda has one more PP after every step, and the parser never stops. Even when the non-recursive rule is listed first, the parser loops as soon as it has to backtrack into the recursive alternative; this happens for every sentence that is not parsed by the first alternative chosen at each point. In the full grammar, with nine recursive rules, the unfiltered top-down parser diverged (a stack overflow after about 2,300 expansions) on 30 of the 30 queries when asked for all parses, and found a first parse (before looping) for only 3 of the 30. On G_mini the unfiltered parser also diverges, after 1975 steps (status: recursion) on the unambiguous sentence of Table 6.1.

Remedy 1: left-recursion elimination

The standard grammar transformation replaces the left-recursive rules NP -> NP PP | Det N | PN by NP -> Det N NP' | PN NP' and NP' -> PP NP' | epsilon, and the VP rules VP -> VP PP | V NP by VP -> V NP VP' and VP' -> PP VP' | epsilon. The transformed grammar generates the same strings, and a top-down parser terminates on it. It has two costs. First, the trees change shape: the left-branching chain [[NP PP] PP] becomes a right-branching chain NP -> Det N NP', NP' -> PP NP', and the NP' and VP' nodes have no linguistic meaning, so the trees must be converted back before they can be interpreted; the two attachment readings of a PP are no longer distinguishable by the node that dominates the PP. Second, the epsilon rules break the assumption that every symbol consumes at least one word, so the length filter below is no longer valid.

Remedy 2: left-corner filter and length filter (used in this work)

The left-corner filter exploits that the next word must be able to start the first daughter of the rule: a rule A -> X1 ... is tried only if some part of speech of the next word is in the left-corner set of X1 (for example, Det and Pron and PN and N and Adj can begin an NP). The length filter exploits that the grammar is epsilon-free, so the symbols on the agenda need at least as many words as there are symbols: a rule is rejected when the number of pending symbols (its own daughters and those of its ancestors) exceeds the number of remaining words. The second filter terminates left recursion because NP -> NP PP adds one pending symbol without consuming a word, so after at most n expansions the length test fails. Both filters leave the set of trees unchanged: the trees of the filtered parser are exactly those of the original grammar, and they are identical to the trees of the Earley parser in all 30 queries. The effect on the trace of Table 6.1 can be seen in the PRUNE actions.

6.5 Worst-case behaviour with k prepositional phrases

Consider the sentence "the student submitted the form" followed by k prepositional phrases of the form P the notice. Every PP can attach to the verb phrase or to any noun phrase to its left, and the number of parses is the Catalan number C(k+1): 1, 2, 5, 14, 42, 132 for k = 0 to 5. The filtered top-down parser enumerates all parses and re-derives every constituent each time. Table 6.4 reports, for G_mini, the number of words, parses, expansion steps, the number of times a constituent (category, start, end) was completed, the number of distinct constituents of this kind, and the redundant completions (completions minus distinct constituents).

Table 6.4 - Top-down cost with k prepositional phrases (G_mini, filtered parser)

k	Words	Parses	Steps	Completions	Distinct	Redundant	Earley states
0	5	1	40	6	4	2	32
1	8	2	160	30	9	21	54
2	11	5	613	135	16	119	79
3	14	14	2,421	580	25	555	107
4	17	42	9,801	2,445	36	2,409	138
5	20	132	40,259	10,246	49	10,197	172

Analysis. The number of steps grows by a factor of about 3.8, 3.9, 4.0, 4.1 for each extra PP (successive ratios), i.e. by a factor tending to 4, in step with the Catalan numbers (C(k+1)/C(k) tends to 4). The redundant completions grow in the same way (from 21 for k = 1 to 10,197 for k = 5), whereas the number of distinct constituents grows only polynomially (9 for k = 1 and

49 for k = 5). In the worst case the time of a top-down parser that enumerates all parses without memoisation is therefore exponential in k, Theta(4^k / k^1.5) up to polynomial factors; this is unavoidable if all parses must be printed, but the redundant sub-parses are avoidable: the same 49 constituents could have been built once. In the full grammar the effect is stronger: the 12-word member of the family needs 4,442,875 steps and the 15-word and 18-word members exhaust a budget of 6,000,000 steps (Task 10).

Strengths and weaknesses. Strengths: the parser never builds a constituent that cannot be part of an S (it is goal-directed), it needs no chart, it is simple to implement and to extend with feature checks, and with the filters it terminates. Weaknesses: it builds constituents that are inconsistent with the input (it expands NP at a verb position until a word fails to match), it loops on left recursion unless filtered, it recomputes sub-trees after backtracking (exponential in the worst case), and it reports all parses only at exponential cost. The Earley parser of Task 7 keeps the goal-directed prediction but stores every sub-result once.

Evidence summary for Task 6. Pseudocode, two complete traces (50 and 127 steps) and a left-recursion trace are given; the left-recursion and re-parsing problems are shown on concrete examples; the two remedies and their effect on trees are discussed; and the cost with k prepositional phrases is measured and analysed.

Task 7 - Earley Parsing

Guiding enquiry. How does the Earley algorithm use dynamic programming to avoid the failures of simple top-down parsing, and what does its chart contain?

7.1 Earley states and the three operations

An Earley **state** is a dotted rule together with the position at which the rule was started: A -> alpha . beta, [i, j]. The dot marks how much of the right-hand side has been recognised; i (the origin) is the input position where the recognition of A began and j is the current position, the index of the chart column S[j] that holds the state. A state with the dot at the end is complete: A has been recognised from position i to j. The parser builds columns S[0], ..., S[n] for the n words, starting from the dummy state gamma -> . S, [0, 0] in S[0]. For each state in column S[j], in order of insertion, exactly one of the following operations applies; no state is added twice.

- **PREDICTOR.** If the state is A -> alpha . B beta, [i, j] where B is a non-terminal, add B -> . gamma, [j, j] for every rule B -> gamma. This is the top-down component: it predicts what may start at position j.
- **SCANNER.** If the state is A -> alpha . B beta, [i, j] where B is a part of speech and word j+1 can be a B, add B -> word ., [j, j+1] to the column S[j+1]. This is the bottom-up check against the input.
- **COMPLETER.** If the state is complete, B -> gamma ., [k, j], then for every state A -> alpha . B beta, [i, k] in S[k] add A -> alpha B . beta, [i, j] to S[j]. This advances every waiting parent over the newly found constituent.

The input is accepted if S[n] contains a complete state S -> gamma ., [0, n]. The chart is a dynamic-programming table: the work of recognising B over [k, j] is done once and used by all parents.

7.2 Hand simulation on a helpdesk sentence

The sentence is "the student submitted the form on friday" (seven words) with G_mini of Task 6. Because of the rules NP -> NP PP and VP -> VP PP, it is left-recursive and ambiguous. Tables 7.1 to 7.8 list every state of the columns S[0] to S[7] with its origin, the operation that created it, and the state(s) it was created from (the notation S[c].m refers to state number m of column S[c]). The chart contains 52 states in total (column sizes 5, 2, 7, 5, 2, 11, 5, 15), and the program output of the Earley implementation is identical to the table.

Table 7.1 - Chart column S[0] - before any word is read

#	State	Origin	Operation (source)
0	γ → • S	0	INIT
1	S → • NP VP	0	PREDICTOR (S[0].0)
2	NP → • Det N	0	PREDICTOR (S[0].1)
3	NP → • NP PP	0	PREDICTOR (S[0].1)
4	NP → • PN	0	PREDICTOR (S[0].1)

Table 7.2 - Chart column S[1] - after reading "the"

#	State	Origin	Operation (source)
0	Det → the •	0	SCANNER (S[0].2)
1	NP → Det • N	0	COMPLETER (S[0].2 + S[1].0)

Table 7.3 - Chart column S[2] - after reading "student"

#	State	Origin	Operation (source)
0	N → student •	1	SCANNER (S[1].1)
1	NP → Det N •	0	COMPLETER (S[1].1 + S[2].0)
2	S → NP • VP	0	COMPLETER (S[0].1 + S[2].1)
3	NP → NP • PP	0	COMPLETER (S[0].3 + S[2].1)
4	VP → • V NP	2	PREDICTOR (S[2].2)
5	VP → • VP PP	2	PREDICTOR (S[2].2)
6	PP → • P NP	2	PREDICTOR (S[2].3)

Table 7.4 - Chart column S[3] - after reading "submitted"

#	State	Origin	Operation (source)
0	V → submitted •	2	SCANNER (S[2].4)
1	VP → V • NP	2	COMPLETER (S[2].4 + S[3].0)
2	NP → • Det N	3	PREDICTOR (S[3].1)
3	NP → • NP PP	3	PREDICTOR (S[3].1)
4	NP → • PN	3	PREDICTOR (S[3].1)

Table 7.5 - Chart column S[4] - after reading "the"

#	State	Origin	Operation (source)
0	Det → the •	3	SCANNER (S[3].2)
1	NP → Det • N	3	COMPLETER (S[3].2 + S[4].0)

Table 7.6 - Chart column S[5] - after reading "form"

#	State	Origin	Operation (source)
0	N → form •	4	SCANNER (S[4].1)
1	NP → Det N •	3	COMPLETER (S[4].1 + S[5].0)
2	VP → V NP •	2	COMPLETER (S[3].1 + S[5].1)
3	NP → NP • PP	3	COMPLETER (S[3].3 + S[5].1)
4	S → NP VP •	0	COMPLETER (S[2].2 + S[5].2)
5	VP → VP • PP	2	COMPLETER (S[2].5 + S[5].2)
6	PP → • P NP	5	PREDICTOR (S[5].3)
7	PP → P • NP	0	COMPLETER (S[0].0 + S[5].4)
8	NP → • Det N	5	PREDICTOR (S[5].7)
9	NP → • NP PP	5	PREDICTOR (S[5].7)
10	NP → • PN	5	PREDICTOR (S[5].7)

Table 7.7 - Chart column S[6] - after reading "on"

#	State	Origin	Operation (source)
0	P → on •	5	SCANNER (S[5].6)
1	PP → P • NP	5	COMPLETER (S[5].6 + S[6].0)
2	NP → • Det N	6	PREDICTOR (S[6].1)
3	NP → • NP PP	6	PREDICTOR (S[6].1)
4	NP → • PN	6	PREDICTOR (S[6].1)

Table 7.8 - Chart column S[7] - after reading "friday"

#	State	Origin	Operation (source)
0	PN → friday •	6	SCANNER (S[6].4)
1	NP → PN •	6	COMPLETER (S[6].4 + S[7].0)
2	PP → P NP •	5	COMPLETER (S[6].1 + S[7].1)
3	NP → NP • PP	6	COMPLETER (S[6].3 + S[7].1)
4	NP → NP PP •	3	COMPLETER (S[5].3 + S[7].2)
5	VP → VP PP •	2	COMPLETER (S[5].5 + S[7].2)
6	PP → • P NP	7	PREDICTOR (S[7].3)
7	VP → V NP •	2	COMPLETER (S[3].1 + S[7].4)
8	NP → NP • PP	3	COMPLETER (S[3].3 + S[7].4)
9	S → NP VP •	0	COMPLETER (S[2].2 + S[7].5) [2 back-pointers]
10	VP → VP • PP	2	COMPLETER (S[2].5 + S[7].5) [2 back-pointers]

#	State	Origin	Operation (source)
11	PP → P • NP	0	COMPLETER (S[0].0 + S[7].9)
12	NP → • Det N	7	PREDICTOR (S[7].11)
13	NP → • NP PP	7	PREDICTOR (S[7].11)
14	NP → • PN	7	PREDICTOR (S[7].11)

The final column S[7] contains the complete state S -> NP VP • with origin 0 (state number 9), so the sentence is accepted. The first predictions of S[0] show how left recursion is handled: the state NP -> • NP PP, [0,0] is predicted once, and it never needs to be predicted again.

7.3 Recovering the trees with back-pointers

A recogniser answers yes or no; to obtain trees every state must record how it was created. Whenever the COMPLETER advances a parent state p over a completed state c, it stores the pair (p, c) with the new state. Following the pairs from a complete S state down gives the tree: c is the last daughter and p, recursively, supplies the daughters before it. If the same state is created in more than one way, the pairs are kept together and the state represents several sub-analyses at once; this is the packed forest.

In the example the complete state S -> NP VP • in S[7] has the two back-pointer pairs below (its single parent is S -> NP • VP [0,2], state S[2].2 which is the same in both):

- (S[2].2 = S → NP • VP, S[7].5 = VP → VP PP •)
- (S[2].2 = S → NP • VP, S[7].7 = VP → V NP •)

The first pair completes S with the verb phrase VP -> VP PP •, which means the PP on friday modifies the VP submitted the form (VP attachment). The second pair uses VP -> V NP •, whose NP is the already combined NP -> NP PP: this is NP attachment. The two trees share all the states below the point of divergence, which is the packing: instead of two complete trees the chart stores one shared structure for the common parts (the subject, the verb, the noun phrase the form and the PP on friday) and two alternatives at one node. The states that carry more than one back-pointer are:

- S[7].9: S → NP VP • [origin 0] with pairs (S[2].2, S[7].5) and (S[2].2, S[7].7)
- S[7].10: VP → VP • PP [origin 2] with pairs (-, S[7].5) and (-, S[7].7)

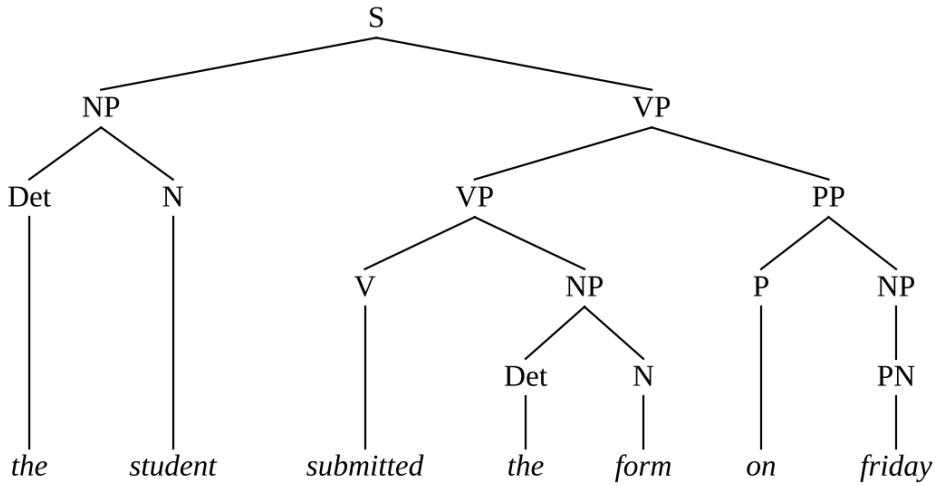


Figure 7.1 - First tree recovered from the chart

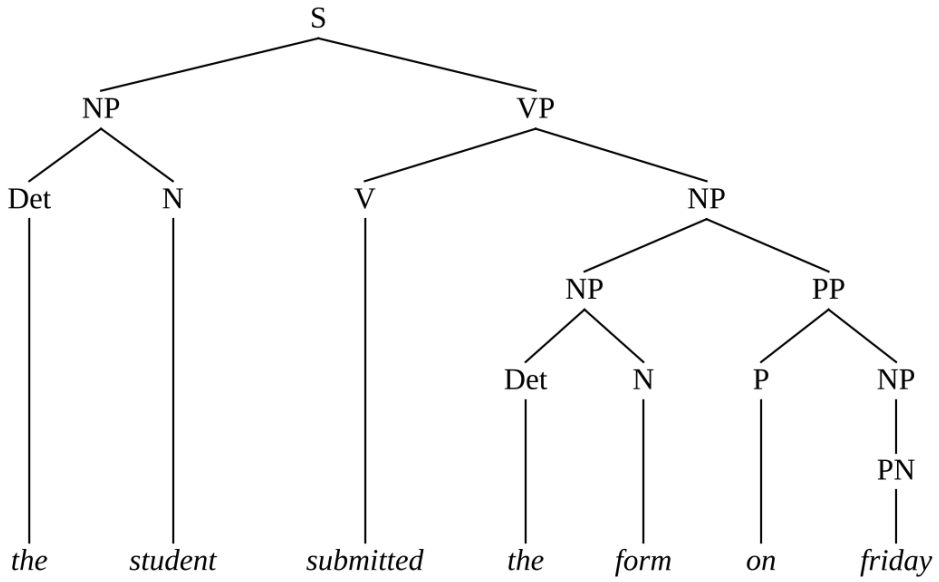


Figure 7.2 - Second tree recovered from the chart

Bracketed forms: (1) (S (NP (Det the) (N student)) (VP (VP (V submitted) (NP (Det the) (N form))) (PP (P on) (NP (PN friday))))); (2) (S (NP (Det the) (N student)) (VP (V submitted) (NP (NP (Det the) (N form)) (PP (P on) (NP (PN friday)))))). The number of trees is computed without enumerating them by the recursion count(state) = sum over its back-pointer pairs of count(p) x count(c) (with count = 1 for lexical states), which gives 2 for the final state. In the full grammar this counting gives 132 trees for the 18-word sentence of Task 10 from a chart of 659 states.

7.4 Left recursion and ambiguous attachment without looping

The top-down parser of Task 6 loops on NP -> NP PP because it expands the same goal NP repeatedly at the same position. In the Earley chart a state is identified by (rule, dot, origin) within one column, and an attempt to add a state that is already present is ignored. The prediction NP -> • NP PP, [j, j] is therefore made once; when it is processed, the predictor would again predict NP rules at position j, but all of them are already in the column, so nothing new is added and the process stops. Left recursion costs nothing extra: the number of states in a column is bounded by (number of dotted rules) x (number of origins), whatever the grammar. The grammar is epsilon-free, so no state can be completed at the position where it was predicted, and the COMPLETER never needs to revisit a column that is still being built. Attachment ambiguity does not make the chart explode either: the two attachments of on friday give two different complete VP states for the span [2, 7] (VP -> VP PP • and VP -> V NP •), but all their sub-constituents are shared and the final S state is reached once with two back-pointer pairs, as shown in Section 7.3.

7.5 Complexity and comparison with the top-down parser

Time. Let n be the number of words and |G| the number of dotted rules (the total length of all right-hand sides plus the number of rules). A column S[j] contains at most |G| x (j + 1) states, because a state is determined by its dotted rule and its origin, and so the whole chart has O(|G| n^2) states, that is, O(n^2) states for a fixed grammar. The PREDICTOR and SCANNER take constant time per state (a fixed number of rules for the predicted category). The COMPLETER for a complete state [k, j] looks at all states of S[k] waiting for the completed category; there are at most |G| k of them, so it costs O(n) per complete state. There are O(n^2) states, so the total time is O(n^2) x O(n) = O(n^3). For epsilon-free grammars each of the O(n^2) complete states is advanced O(n) times, which is the n^3 term.

Space. The chart stores O(n^2) states, so O(n^2) space is needed for the recogniser. If every state stores all its back-pointer pairs, a state may carry up to O(n) pairs (one per split point), so the packed forest takes O(n^3) space in the worst case; the forest is still polynomial although it represents exponentially many trees. This is the advantage over the top-down parser, whose output is the explicit list of trees, with Catalan growth.

Comparison on the same sentence. For "the student submitted the form on friday" on G_mini, the filtered top-down parser performs 104 expansion steps (including the redundant rebuilding of the form), while the Earley parser creates 52 states; the unfiltered top-down parser does not

terminate (Task 6). For the full grammar, the 9-word sentence of the PP-chain family of Task 10 ("Book the room near the library in the hostel", 5 parses) needs 214,174 top-down steps and 313 Earley states, and the 12-word member needs 4,442,875 steps against 418 states. The Earley parser creates a number of states that grows polynomially with the sentence length and independently of the number of parses, while the number of top-down steps grows with the number of parses. These measurements are collected in Task 10.

Evidence summary for Task 7. The Earley state and its three operations were defined; all 52 states of the seven-word sentence were listed with state, origin and operation; trees were recovered through back-pointers and the packing of the two analyses was shown; handling of left recursion and attachment ambiguity was explained; and the O(n^3) time and O(n^2) space bounds were derived and compared with the measured top-down cost.

Task 8 - Feature Structures and Unification

Guiding enquiry. How can attribute-value structures and unification replace multiplied categories to handle agreement and subcategorization elegantly?

8.1 Feature structures, paths and reentrancy

A **feature structure** is a set of attribute-value pairs written as an attribute-value matrix (AVM). Values are atomic (3sg, nom, NP) or are again feature structures. A **feature path** is a sequence of attributes that leads into a nested structure, written with angle brackets: <NP AGREEMENT NUM> is the value of NUM inside the AGREEMENT of the noun phrase. **Reentrancy** (structure sharing) means that two paths lead to the same value, not to equal copies; it is written with a shared tag such as [1]. If the shared value is later specified by one path, it is specified for the other as well. In the grammar, reentrancy is what makes agreement work: S -> NP VP with <NP AGREEMENT> = <VP AGREEMENT> is the statement that the two paths share one structure. For clarity of exposition the notation below uses the complex AGREEMENT [NUM, PER]; the implementation uses its atomic summary AGR (3sg or non3sg), which is sufficient for the fragment (Task 4).

Table 8.1 gives the lexical entries (eight representative entries are shown). CAT is the category, NUM and PER the number and person of nominals, AGR the agreement class, CASE the case, VFORM the verb form, SUBCAT the subcategorization frame and PFORM the preposition form that the verb selects.

Table 8.1 - Lexical entries as attribute-value matrices

Word	Feature structure
students	[CAT N, AGR non3sg, NUM pl, PER 3]
the	[CAT Det]
these	[CAT Det, AGR non3sg]
me	[CAT Pron, AGR non3sg, PER 1, NUM sg, CASE acc]
he	[CAT Pron, AGR 3sg, PER 3, NUM sg, CASE nom]
submits	[CAT V, VFORM fin, AGR 3sg, SUBCAT NP]
send	[CAT V, VFORM base, SUBCAT NP_NP]
apply	[CAT V, VFORM base, SUBCAT PP, PFORM for]

Underspecification is visible: the has no AGR value, so it unifies with any noun; submitted (not shown) has no AGR value and fits any subject; the pronoun you has no CASE value and can be a subject or an object.

8.2 Unification and subsumption

Subsumption. A feature structure F1 subsumes F2, written F1 ⊑ F2, if F1 contains no information that F2 lacks: every attribute of F1 is in F2 with a value that F1's value subsumes. The empty structure [] subsumes everything, and [AGR 3sg] subsumes [AGR 3sg, CASE nom]. **Unification** F1 ⊔ F2 is the least upper bound of the two structures in the subsumption order: the most general structure that both F1 and F2 subsume, that is, the structure that contains exactly the information of both. If the structures contain conflicting atomic values, unification fails. Examples 1 to 3 below were computed with the unification function of the implementation; example 4 illustrates reentrancy:

Table 8.2 - Worked unifications

No.	First structure	Second structure	Result	Explanation
1	[AGR 3sg, CASE nom]	[AGR 3sg]	[AGR 3sg, CASE nom]	The common attribute has equal values; the second structure adds nothing.
2	[AGR 3sg]	[AGR non3sg]	FAIL	Conflicting atomic values: the subject the students and the verb submits cannot combine.
3	[NUM pl]	[PER 3, CASE nom]	[NUM pl, PER 3, CASE nom]	Disjoint attributes: the result contains the information of both.
4	[AGR 1] shared by NP and VP, NP = [AGR 3sg]	VP = [AGR 3sg, VFORM fin]	[AGR 3sg, VFORM fin] for both	Reentrancy: specifying AGR through the NP specifies it for the VP.

8.3 The feature-augmented grammar

Table 8.3 shows the complete grammar G_F: the phrase-structure skeleton of G with single V and the equations. A position number i refers to the i-th daughter, 0 to the mother; the notation i.F = j.G means <daughter i F> = <daughter j G>, and i.F = value means the path has this value. Six of the rules are discussed first.

- S -> NP VP with <NP AGR> = <VP AGR>, <NP CASE> = nom and <VP VFORM> = fin: the subject agrees with the verb phrase, is in the nominative, and the verb phrase is finite.

- NP -> Det Nom with <Det AGR> = <Nom AGR> and <NP AGR> = <Nom AGR>: determiner-noun agreement, and the noun phrase inherits the agreement of its head nominal.
- VP -> V NP with <V SUBCAT> = NP and <NP CASE> = acc: the verb must belong to the transitive frame, the object is accusative, and the verb phrase inherits AGR and VFORM from the verb (<VP AGR> = <V AGR>, <VP VFORM> = <V VFORM>).
- VP -> V NP PP with <V SUBCAT> = NP_PP and <V PFORM> = <PP PFORM>: the frame NP + PP and the selection of the preposition.
- SQ -> Aux NP VP with <Aux AGR> = <NP AGR>, <NP CASE> = nom and <VP VFORM> = base: auxiliary agreement in an inverted question, and the verb after the subject is a base form.
- PP -> P NP with <PP PFORM> = <P PFORM> and <NP CASE> = acc: the PP passes the identity of its preposition upward and requires an accusative object.

Table 8.3 - The feature grammar G_F: all rules with their constraint equations

No.	Rule	Constraints
1	S → NP VP	1.CASE=nom; 1.AGR=2.AGR; 2.VFORM=fin
2	S → VP	1.VFORM=base
3	S → SQ	(none)
4	S → WhNP VP	1.AGR=2.AGR; 2.VFORM=fin
5	S → WhNP SQ/NP	(none)
6	S → WhAdv SQ	(none)
7	S → S Conj S	(none)
8	SQ → Aux NP VP	1.AGR=2.AGR; 2.CASE=nom; 3.VFORM=base
9	SQ → V NP AdjP	1.SUBCAT=ADJ; 1.VFORM=fin; 1.AGR=2.AGR; 2.CASE=nom
10	SQ → V NP V	1.SUBCAT=PASS; 1.VFORM=fin; 1.AGR=2.AGR; 2.CASE=nom; 3.VFORM=pp; 3.SUBCAT=NP
11	SQ → V NP PP	1.SUBCAT=PP; 1.VFORM=fin; 1.AGR=2.AGR; 2.CASE=nom; 1.PFORM=3.PFORM
12	SQ/NP → Aux NP VP/NP	1.AGR=2.AGR; 2.CASE=nom; 3.VFORM=base
13	S/NP → NP VP/NP	1.CASE=nom; 1.AGR=2.AGR; 2.VFORM=fin
14	VP/NP → V	1.SUBCAT=NP; 0.AGR=1.AGR; 0.VFORM=1.VFORM
15	VP/NP → V NP	1.SUBCAT=NP_NP; 2.CASE=acc; 0.AGR=1.AGR; 0.VFORM=1.VFORM
16	VP/NP → V PP	1.SUBCAT=NP_PP; 1.PFORM=2.PFORM; 0.AGR=1.AGR; 0.VFORM=1.VFORM

No.	Rule	Constraints
17	VP/NP → Aux VP/NP	2.VFORM=base; 0.AGR=1.AGR; 0.VFORM=1.VFORM
18	NP → Det Nom	1.AGR=2.AGR; 0.AGR=2.AGR
19	NP → Nom	1.AGR=non3sg; 0.AGR=1.AGR
20	NP → Pron	0.AGR=1.AGR; 0.CASE=1.CASE
21	NP → PN	0.AGR=1.AGR
22	NP → NP PP	0.AGR=1.AGR; 0.CASE=1.CASE
23	NP → NP Conj NP	0.AGR=2.AGR; 0.CASE=1.CASE
24	NP → NP RelCl	0.AGR=1.AGR; 0.CASE=1.CASE; 1.AGR=2.AGR
25	Nom → N	0.AGR=1.AGR
26	Nom → N Nom	1.AGR=3sg; 0.AGR=2.AGR
27	Nom → Adj Nom	0.AGR=2.AGR
28	Nom → Nom Conj Nom	0.AGR=2.AGR
29	WhNP → WhPron	0.AGR=1.AGR
30	WhNP → WhDet Nom	0.AGR=2.AGR
31	VP → V	1.SUBCAT=INTR; 0.AGR=1.AGR; 0.VFORM=1.VFORM
32	VP → V NP	1.SUBCAT=NP; 2.CASE=acc; 0.AGR=1.AGR; 0.VFORM=1.VFORM
33	VP → V NP NP	1.SUBCAT=NP_NP; 2.CASE=acc; 3.CASE=acc; 0.AGR=1.AGR; 0.VFORM=1.VFORM
34	VP → V CP	1.SUBCAT=CP; 0.AGR=1.AGR; 0.VFORM=1.VFORM
35	VP → V NP CP	1.SUBCAT=NP_CP; 2.CASE=acc; 0.AGR=1.AGR; 0.VFORM=1.VFORM
36	VP → V PP	1.SUBCAT=PP; 1.PFORM=2.PFORM; 0.AGR=1.AGR; 0.VFORM=1.VFORM
37	VP → V NP PP	1.SUBCAT=NP_PP; 2.CASE=acc; 1.PFORM=3.PFORM; 0.AGR=1.AGR; 0.VFORM=1.VFORM
38	VP → V AdjP	1.SUBCAT=ADJ; 0.AGR=1.AGR; 0.VFORM=1.VFORM
39	VP → V V	1.SUBCAT=PASS; 2.VFORM=pp; 2.SUBCAT=NP; 0.AGR=1.AGR; 0.VFORM=1.VFORM
40	VP → Aux VP	2.VFORM=base; 0.AGR=1.AGR; 0.VFORM=1.VFORM
41	VP → VP PP	0.AGR=1.AGR; 0.VFORM=1.VFORM
42	VP → VP SubCl	0.AGR=1.AGR; 0.VFORM=1.VFORM
43	VP → VP Conj VP	0.AGR=1.AGR; 1.AGR=3.AGR; 0.VFORM=1.VFORM; 1.VFORM=3.VFORM

No.	Rule	Constraints
44	PP → P NP	0.PFORM=1.PFORM; 2.CASE=acc
45	AdjP → Adj	(none)
46	AdjP → Adv Adj	(none)
47	AdjP → Adj PP	(none)
48	AdjP → AdjP Conj AdjP	(none)
49	RelCl → Comp VP	2.VFORM=fin; 0.AGR=2.AGR
50	RelCl → Comp S/NP	(none)
51	CP → Comp S	(none)
52	SubCl → Sconj S	(none)

Unified feature structure of one complete parse

The sentence "The students submitted the form" has the single tree below. At every node the feature structure of the mother, computed by unifying the daughters under the rule's equations, is shown (only features mentioned in the constraints are kept). The nodes read from the top.

Table 8.4 - Feature structures at the nodes of the parse of "The students submitted the form"

Node (indentation = depth)	Feature structure
S	[]
NP	[AGR non3sg]
Det the	[]
Nom	[AGR non3sg]
N students	[AGR non3sg, NUM pl, PER 3]
VP	[VFORM fin]
V submitted	[VFORM fin, SUBCAT NP]
NP	[AGR 3sg]
Det the	[]
Nom	[AGR 3sg]
N form	[AGR 3sg, NUM sg, PER 3]

At the root, S -> NP VP unifies <NP AGR> = non3sg (inherited from students) with <VP AGR>, which is unspecified for the past tense, so the equation holds; the VP inherits VFORM = fin from submitted, its SUBCAT = NP selects the rule VP -> V NP, and the object NP the form has AGR = 3sg, which is irrelevant for the verb phrase. With submits instead of submitted, <VP AGR> would be 3sg and unification at S would fail.

8.4 Redoing agreement and subcategorization with features

Tables 4.3 and 5.2 already contain the results of the feature grammar: all twelve ungrammatical sentences of the two tests are rejected and all twelve grammatical ones are accepted. The comparison of the sizes is in Table 8.5; G_F states agreement (AGR), case, verb form, subcategorization and preposition selection, i.e. the full feature set, in 52 rules and 251 annotated lexical entries.

Table 8.5 - Grammar size with and without features

Grammar	Phenomena enforced	Phrase rules	Lexical entries
G (plain CFG)	frames only	52	211
G + number, case (multiplied)	agreement, case	171	284
G + number, case, verb form (multiplied)	+ verb form	299	336
G + ... + preposition form (multiplied)	+ preposition selection	3469	2216
G_F (feature grammar)	all of the above	52	251

The feature grammar enforces all five constraints with 52 rules, the same as the baseline G, whereas the multiplied grammar that enforces the same constraints needs 3469 rules, a factor of 67. Even the subset (number and case) costs 171 rules, a factor of 3.3. The lexicon grows from 251 annotated entries to 2,216 category-word pairs. Subcategorization is also tidier: the nine frame categories disappear and the verb entry carries a single SUBCAT value; the nine VP rules differ only in the equation <V SUBCAT> = frame.

8.5 Unification inside the Earley parser, and its cost

Integration. Unification is added to the COMPLETER. Every state stores, besides the dotted rule and the origin, the feature structures of the daughters recognised so far. When the COMPLETER advances A -> alpha . B beta over the completed B, it unifies the feature structure of the completed B with the equations of the rule for the new daughter position; if unification fails the new state is not created, and so the ungrammatical analysis is pruned at the earliest possible point, before the remaining daughters are parsed. When the dot reaches the end, the mother's feature structure is computed and stored in the complete state, from which its own parents will read it. A state is now identified by its dotted rule, origin and the relevant feature values, so two states with the same rule and span but different feature values are

separate states (the parent state may still be shared when the values do not matter, because only the features mentioned in constraints are kept in the state).

Measured effect. On the 30 queries, the Earley parser with the skeleton grammar (all constraints ignored) creates 8,145 states and 70 complete trees, while with unification it creates 6,527 states (20% fewer) and 49 trees; the 21 removed trees are the over-generated analyses (wrong frame, wrong attachment of a complement, wrong verb form). The total time for the 30 queries was 46 ms without and 31 ms with unification, so the pruning more than pays for the cost of unifying on this grammar.

What is lost in complexity. (1) Each COMPLETER step now includes a unification, costing time proportional to the number of constraint equations of the rule. (2) Packing is weaker: states with different feature values cannot be merged, so the number of states per (rule, origin) can be multiplied by the number of distinct value combinations; for the finite domains here (AGR with two values, CASE with two, VFORM with three, SUBCAT with nine and PFORM with eleven) this is a constant factor, and the O(n^3) bound becomes O(n^3 x F) with F the number of combinations that occur. (3) If feature values may be unbounded structures, such as recursive lists of subcategorized complements or an unrestricted SLASH feature with arbitrary nesting, F is not bounded and the worst case can be exponential; for unrestricted unification grammars recognition is even undecidable. The helpdesk grammar uses only atomic values from finite sets, which keeps parsing polynomial. (4) The ranking by a PCFG over such a grammar is not a true probability model, because the probability mass assigned to trees that the features reject is lost (this is discussed in Task 9).

Evidence summary for Task 8. Feature structures, paths and reentrancy were defined; eight lexical entries were shown as AVMs; four unifications were worked, one of them failing; the complete feature-augmented grammar with 52 rules and six discussed in detail was given; a full parse was annotated with feature structures; Tasks 4 and 5 were redone with features and the grammar sizes compared; and the integration with the Earley COMPLETER and its cost were discussed and measured.

Task 9 - Probabilistic Context-Free Grammars

Guiding enquiry. When a query has several valid parses, how can probabilities select the most plausible one?

9.1 Definition and the normalisation condition

A probabilistic context-free grammar (PCFG) is a CFG in which every rule A -> beta has a probability P(A -> beta | A) and the probabilities of all rules with the same left-hand side sum to one: the sum over beta of P(A -> beta | A) = 1 for every non-terminal A. The probability of a parse tree is the product of the probabilities of the rules used in it, and the probability of a sentence is the sum of the probabilities of all its trees. The PCFG of the helpdesk is defined on the skeleton of the feature grammar G_F: its rules are the 52 phrase-structure rules and the lexical rules Category -> word, and features act as a filter on top of the probabilistic model (Section 9.5).

Table 9.1 - Normalisation check: the probabilities of each left-hand side sum to 1

Left-hand side	Kind	Rules (K)	Treebank count C(A)	Sum of P(A -> beta)
S	phrasal	7	21	1.000000
SQ	phrasal	4	3	1.000000
SQ/NP	phrasal	1	1	1.000000
S/NP	phrasal	1	0	1.000000
VP/NP	phrasal	4	1	1.000000
NP	phrasal	7	44	1.000000
Nom	phrasal	4	38	1.000000
WhNP	phrasal	2	2	1.000000
VP	phrasal	13	23	1.000000
PP	phrasal	1	8	1.000000
AdjP	phrasal	4	2	1.000000
RelCl	phrasal	2	1	1.000000
CP	phrasal	1	1	1.000000
SubCl	phrasal	1	0	1.000000
Det	lexical	9	35	1.000000
PN	lexical	2	2	1.000000
Adv	lexical	2	0	1.000000
Aux	lexical	6	3	1.000000
Conj	lexical	2	1	1.000000

Left-hand side	Kind	Rules (K)	Treebank count C(A)	Sum of P(A -> beta)
Pron	lexical	11	2	1.000000
Adj	lexical	7	3	1.000000
Sconj	lexical	4	0	1.000000
WhPron	lexical	2	2	1.000000
N	lexical	43	37	1.000000
WhDet	lexical	2	0	1.000000
P	lexical	11	8	1.000000
WhAdv	lexical	4	1	1.000000
V	lexical	63	24	1.000000
Comp	lexical	1	2	1.000000

All 29 left-hand sides (14 phrasal and 15 lexical categories) sum to 1, with a largest deviation of 2.2e-16 (floating-point rounding). The condition holds by construction of the estimator in Section 9.2, and it was checked by program for every non-terminal.

9.2 Maximum-likelihood estimation from the mini-treebank

The treebank. The mini-treebank consists of 20 helpdesk sentences (120 words, 385 tree nodes, 265 rule applications) that are listed with their bracketed trees in Appendix B. They are different from the 30 test queries, so the evaluation in Section 9.5 is on unseen sentences. The trees were annotated by selection: the Earley parser produced all analyses of each sentence, and the annotator chose the correct one (for the seven sentences with attachment choices, the choice follows the meaning: temporal and locative PPs modify the verb phrase, with the late fee and for the fee modify the noun phrase, and to the office after submit is a complement).

Formula. The maximum-likelihood estimate is P(A -> beta) = C(A -> beta) / C(A), where C(A -> beta) is the number of times the rule is used in the treebank and C(A) = sum over beta' of C(A -> beta'). A raw estimate gives probability zero to every rule that the 20 trees do not use (Table 9.2 shows that 19 of the 52 phrase rules are unseen) and to every unseen word, which would make most test sentences impossible. The estimates are therefore **smoothed by add-alpha (Lidstone) smoothing** with alpha = 0.1:

$$P(A \to \beta) = (C(A \to \beta) + \alpha) / (C(A) + \alpha \times K_A),$$

where K_A is the number of rules (or words, for a lexical category) with left-hand side A. This keeps the sums at exactly 1, gives an unseen rule the probability alpha / (C(A) + alpha K_A), and shrinks the seen probabilities slightly. The value alpha = 0.1 was chosen to keep the seen

counts dominant while avoiding zero probabilities; the sensitivity to alpha is examined in Section 9.5. **Unseen rules** are thus handled by smoothing: they remain possible but improbable, and every unseen rule of the same left-hand side has the same probability. For lexical rules, which are the largest group, the same formula is used with K_A = number of words of the category.

Table 9.2 - Counts and smoothed probabilities of the phrase-structure rules

No.	Rule	Count C(A → β)	P(A → β)
1	S → NP VP	10	0.4654
2	S → VP	6	0.2811
3	S → SQ	2	0.0968
4	S → WhNP VP	1	0.0507
5	S → WhNP SQ/NP	1	0.0507
6	S → WhAdv SQ	1	0.0507
7	S → S Conj S	0	0.0046
8	SQ → Aux NP VP	2	0.6176
9	SQ → V NP AdjP	1	0.3235
10	SQ → V NP V	0	0.0294
11	SQ → V NP PP	0	0.0294
12	SQ/NP → Aux NP VP/NP	1	1.0000
13	S/NP → NP VP/NP	0	1.0000
14	VP/NP → V	1	0.7857
15	VP/NP → V NP	0	0.0714
16	VP/NP → V PP	0	0.0714
17	VP/NP → Aux VP/NP	0	0.0714
18	NP → Det Nom	35	0.7852
19	NP → Nom	0	0.0022
20	NP → Pron	2	0.0470
21	NP → PN	2	0.0470
22	NP → NP PP	3	0.0694
23	NP → NP Conj NP	1	0.0246
24	NP → NP RelCl	1	0.0246
25	Nom → N	35	0.9141
26	Nom → N Nom	2	0.0547
27	Nom → Adj Nom	1	0.0286
28	Nom → Nom Conj Nom	0	0.0026
29	WhNP → WhPron	2	0.9545

No.	Rule	Count C(A → β)	P(A → β)
30	WhNP → WhDet Nom	0	0.0455
31	VP → V	2	0.0864
32	VP → V NP	10	0.4156
33	VP → V NP NP	2	0.0864
34	VP → V CP	0	0.0041
35	VP → V NP CP	1	0.0453
36	VP → V PP	1	0.0453
37	VP → V NP PP	1	0.0453
38	VP → V AdjP	1	0.0453
39	VP → V V	2	0.0864
40	VP → Aux VP	0	0.0041
41	VP → VP PP	3	0.1276
42	VP → VP SubCl	0	0.0041
43	VP → VP Conj VP	0	0.0041
44	PP → P NP	8	1.0000
45	AdjP → Adj	2	0.8750
46	AdjP → Adv Adj	0	0.0417
47	AdjP → Adj PP	0	0.0417
48	AdjP → AdjP Conj AdjP	0	0.0417
49	RelCl → Comp VP	1	0.9167
50	RelCl → Comp S/NP	0	0.0833
51	CP → Comp S	1	1.0000
52	SubCl → Sconj S	0	1.0000

Table 9.3 - Lexical rules: counts per category and the probability of a word seen once or never

Category	Tokens C(A)	Words K	P(word seen once)	P(unseen word)
Det	35	9	0.0306	0.0028
N	37	43	0.0266	0.0024
PN	2	2	0.5000	0.0455
Pron	2	11	0.3548	0.0323
WhPron	2	2	0.5000	0.0455
WhDet	0	2	5.5000	0.5000
WhAdv	1	4	0.7857	0.0714
Adj	3	7	0.2973	0.0270
Adv	0	2	5.5000	0.5000

Category	Tokens C(A)	Words K	P(word seen once)	P(unseen word)
P	8	11	0.1209	0.0110
Conj	1	2	0.9167	0.0833
Comp	2	1	0.5238	0.0476
Sconj	0	4	2.7500	0.2500
Aux	3	6	0.3056	0.0278
V	24	63	0.0363	0.0033

The treebank is very small. The Table 9.2 probabilities are dominated by smoothing for the rarely used rules, and the figures are used in this work to demonstrate the method and to rank analyses, not as accurate statistics of English. In particular, the probability of the PP-attachment rules is determined by only three NP-attached and three VP-attached cases (Section 9.5).

9.3 Probability of two parse trees and of the sentence

The sentence is "Show the students with the notice" (an ambiguous PP-attachment). Table 9.4 lists the rule applications of the VP-attachment tree and Table 9.5 those of the NP-attachment tree with the probabilities of Table 9.2 and the lexical probabilities derived from Table 9.3. The tree probability is the product of the 14 factors.

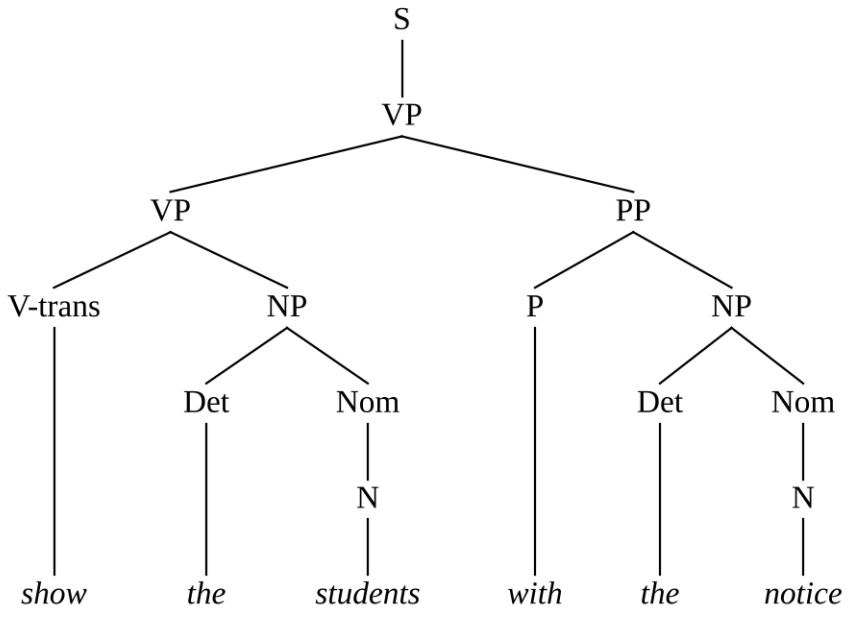


Figure 9.1 - Tree 1 (VP attachment) for "Show the students with the notice"

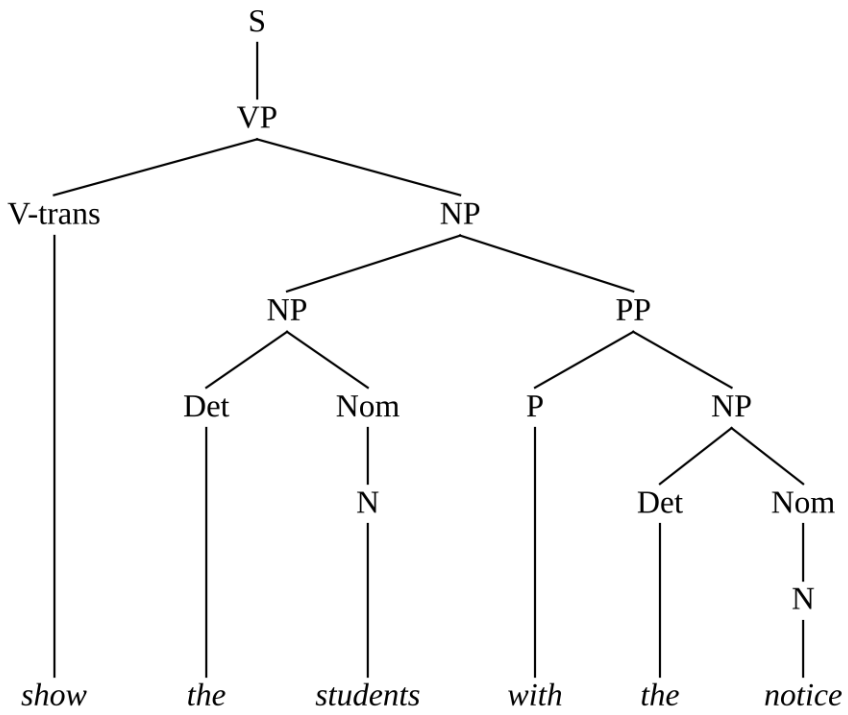


Figure 9.2 - Tree 2 (NP attachment)

Table 9.4 - Rule probabilities of tree 1 (VP attachment)

Rule applied	Probability
S → VP	0.2811
VP → VP PP	0.1276
VP → V NP	0.4156
V → show	0.0363
NP → Det Nom	0.7852
Det → the	0.9777
Nom → N	0.9141
N → students	0.0508
PP → P NP	1.0000
P → with	0.1209
NP → Det Nom	0.7852
Det → the	0.9777
Nom → N	0.9141
N → notice	0.0266
Product P(tree 1)	**4.362 x 10^-8**

Table 9.5 - Rule probabilities of tree 2 (NP attachment)

Rule applied	Probability
S → VP	0.2811
VP → V NP	0.4156
V → show	0.0363
NP → NP PP	0.0694
NP → Det Nom	0.7852
Det → the	0.9777
Nom → N	0.9141
N → students	0.0508
PP → P NP	1.0000
P → with	0.1209
NP → Det Nom	0.7852
Det → the	0.9777
Nom → N	0.9141
N → notice	0.0266
Product P(tree 2)	**2.372 x 10^-8**

Best parse. P(tree 1) = 4.362 x 10^-8 and P(tree 2) = 2.372 x 10^-8, both obtained by multiplying the factors of the tables (the program value and the hand product agree to 12 significant digits). The ratio is 1.84, so the PCFG selects the VP-attachment tree (tree 1). **Sentence probability.** The sentence has exactly two trees, so P(sentence) = P(tree 1) + P(tree 2) = 6.734 x 10^-8. The Viterbi tree carries 64.8% of this probability mass, which is also the posterior probability that the PCFG assigns to the VP-attachment reading.

Why tree 1 wins. The two trees use the same lexical rules and the same rules for the two noun phrases. They differ only in the attachment rules: tree 1 uses VP -> VP PP (P = 0.1276) and the one NP rule NP -> Det Nom for the students, tree 2 uses NP -> NP PP (P = 0.0694) together with the extra NP node. The attachment decision is thus determined by two numbers only, the relative frequency of the two rules in the treebank.

Table 9.6 - Probabilities for all ambiguous test queries (best tree and sentence)

Query	Parses	P(best tree)	P(sentence) = sum	Best tree = gold?
Q7	2	3.36 x 10^-8	4.61 x 10^-8	no
Q9	2	1.83 x 10^-20	3.66 x 10^-20	yes
Q14	2	6.97 x 10^-13	9.17 x 10^-13	yes

Query	Parses	P(best tree)	P(sentence) = sum	Best tree = gold?
Q16	2	2.51 x 10^-12	3.87 x 10^-12	no
Q17	5	1.98 x 10^-13	5.31 x 10^-13	no
Q18	7	1.71 x 10^-10	6.83 x 10^-10	no
Q19	2	1.28 x 10^-11	1.31 x 10^-11	no
Q20	5	2.67 x 10^-10	7.15 x 10^-10	no

Gold trees for the eight ambiguous queries were fixed in advance (Appendix B). The Viterbi parse equals the gold tree for 2 of 8 queries. One of the two successes (Q9) is a tie: its two trees use the same number of rules with identical probabilities (the PP attaches to be held or to will be held), so the parser's choice there is arbitrary and the success should not be counted as evidence for the model.

9.4 Probabilistic CKY (Viterbi) after conversion to Chomsky normal form

Conversion. The CKY algorithm needs rules with at most two symbols on the right-hand side. The conversion has three steps, applied to the 52 skeleton rules.

8. **Binarisation.** Each of the 13 ternary rules A -> X Y Z is replaced by A -> X A@2 (with the probability of the original rule) and A@2 -> Y Z (probability 1), where A@2 is a new symbol made unique for the rule. Example: VP -> V NP NP becomes VP -> V VP@r.2 and VP@r.2 -> NP NP. The 29 binary rules are kept, so there are 55 binary rules after the step (29 original + 13 top rules + 13 intermediate rules). Probabilities are preserved, because each tree of the original grammar maps to exactly one tree of the binarised grammar with the same product.
9. **Unit rules.** The grammar contains 10 unit rules: S -> VP; S -> SQ; VP/NP -> V; NP -> Nom; NP -> Pron; NP -> PN; Nom -> N; WhNP -> WhPron; VP -> V; AdjP -> Adj. In strict CNF they are removed by collapsing chains; for example NP -> Nom, Nom -> N and N -> student give NP -> student with probability P(NP -> Nom) x P(Nom -> N) x P(N -> student), and the chain is remembered so that the tree can be restored. The implementation does the equivalent without enlarging the grammar: after each cell is filled, it applies the unit rules in topological order (children first) as a unary closure, multiplying the probability by that of the unit rule. The result is a grammar in CNF-equivalent form (binary rules plus unit closure), which the task permits.
10. **Lexical rules.** The 169 lexical rules have the form Category -> word and are already in CNF.

Pseudocode. The chart cell (i, j) maps each item (symbol, feature key) to its best probability, the sum of all probabilities (inside probability) and a back-pointer. The feature key is empty for a pure PCFG; in this implementation it is the set of interface features of the constituent so that agreement and subcategorization are checked as items are combined.

```
function VITERBI-CKY(words):
   for i = 0 .. n-1:                          // lexical row
      for each entry (category, features) of words[i]:
         cell[i,i+1][category, features] = P(category -> words[i])
      UNARY-CLOSURE(cell[i,i+1])
   for span = 2 .. n:
      for i = 0 .. n-span:  j = i + span
         for k = i+1 .. j-1:                  // split point
            for each item (Y,fy) in cell[i,k] and (Z,fz) in cell[k,j]:
               for each binary rule  A -> Y Z  (or A@m -> Y Z):
                  features' = UNIFY(rule constraints, fy, fz)
                  if unification fails: continue
                  p = best[Y,fy] * best[Z,fz] * P(rule)
                  if p > best[A,features']: best[A,features'] = p, back-pointer = (k,Y,Z)
                  inside[A,features'] += inside[Y,fy] * inside[Z,fz] * P(rule)
         UNARY-CLOSURE(cell[i,j])
   return the item (S, .) in cell[0,n] with the largest best value (Viterbi tree);
         the sum of inside over all (S, .) items is P(sentence)
```

Worked table. Table 9.7 shows the chart for "show the students with the notice" (six words). To keep it readable, each cell lists only the items that occur in at least one complete parse; the numbers are the Viterbi (best) probability of the item, with the inside probability in brackets when the item has more than one analysis. The empty cells (spans that are not constituents in any parse) are omitted. The top cell [0,6] contains S with the best probability 4.362 x 10^-8, which is the probability of tree 1, and the inside probability 6.734 x 10^-8, which equals P(sentence); the table contains 33 items in total in the full chart (the table shows the useful ones), and the combination step was executed 35 times.

Table 9.7 - Viterbi CKY table for "show the students with the notice"

Cell	Words	Items: best probability (inside, number of trees)
[0,1]	show	V: 3.63 x 10^-2 (1.45 x 10^-1, 4 trees)
[1,2]	the	Det: 9.78 x 10^-1
[2,3]	students	N: 5.08 x 10^-2; Nom: 4.65 x 10^-2
[3,4]	with	P: 1.21 x 10^-1
[4,5]	the	Det: 9.78 x 10^-1
[5,6]	notice	N: 2.66 x 10^-2; Nom: 2.43 x 10^-2

Cell	Words	Items: best probability (inside, number of trees)
[1,3]	the students	NP: 3.57 x 10^-2
[4,6]	the notice	NP: 1.87 x 10^-2
[0,3]	show the students	VP: 5.38 x 10^-4 (1.08 x 10^-3, 2 trees)
[3,6]	with the notice	PP: 2.26 x 10^-3
[1,6]	the students with the notice	NP: 5.59 x 10^-6
[0,6]	show the students with the notice	VP: 1.55 x 10^-7 (4.79 x 10^-7, 4 trees); S: 4.36 x 10^-8 (6.73 x 10^-8, 2 trees)

The two analyses meet in cell [0,6]: the S item there can be built from VP -> VP PP over [0,6] or from VP -> V NP over [0,6] with NP -> NP PP in [1,6]. The Viterbi value takes the maximum of the two, and the inside value takes the sum. The cell [1,6] contains both NP (the students with the notice, via NP -> NP PP) and the cell [0,3] contains the VP show the students.

9.5 Independence assumptions and their failures

Context-freeness and position invariance

A PCFG assumes that the probability of a rule depends only on its left-hand side: it is independent of the position of the node in the tree (position invariance), of the ancestors (context-freeness) and of the words. As a consequence the same probability P(VP -> VP PP) applies whether the verb phrase is at the top of an imperative or deep inside a relative clause, and the same P(NP -> Pron) applies to a subject and to an object, although pronouns are far more frequent as subjects. In the helpdesk, the NP-attachment rule is used for the three NP-attached treebank PPs, so the model has learned a single global ratio of the two attachments, shown below.

Lack of lexical sensitivity

The model chooses between attachments by comparing the two rules, never the words involved. A human decides by the verb and the nouns (show the students with the notice: students can have a notice; pay the fee in the office: the office is where the paying happens). The experiment of Table 9.8 uses six sentences with a PP that can attach to the verb phrase or the noun phrase, with the gold attachment fixed by meaning. The PCFG probability ratio P(VP attachment) / P(NP attachment) is identical for all six sentences, because it involves only the two attachment rules and the PP and noun phrase below them are identical in both trees.

Table 9.8 - PP-attachment decisions: gold versus PCFG

Sentence	Gold attachment	PCFG choice	P(VP) / P(NP)	Correct?
Show the students with the late fee notice.	NP	VP	1.840	no
Show the notice on Friday.	VP	VP	1.840	yes
Collect the receipt for the fee.	NP	VP	1.840	no
Pay the fee in the office.	VP	VP	1.840	yes
Book the room near the library.	NP	VP	1.840	no
Submit the form on Friday.	VP	VP	1.840	yes

The PCFG always prefers VP attachment (the ratio is about 1.84 for all sentences) and is therefore correct in 3 of 6 cases, and wrong in all the cases where the meaning demands NP attachment (show the students with the late fee notice, collect the receipt for the fee, book the room near the library). This is the failure case of lexical insensitivity: near the library describes the room, but a PCFG cannot know that rooms have locations while forms are not submitted near anything. The usual repairs are lexicalised PCFGs, which condition the rules on the head word, or the use of the SUBCAT and PFORM features as probabilistic cues.

Preference for shorter trees

Because the probability of a tree is a product of numbers smaller than one, a tree with more rule applications is penalised, all else being equal. For coordination ambiguity the effect decides the result: Check the fees and hostel charges has two trees, the tree with the nominal coordination (13 rule applications) and the tree with the noun-phrase coordination (14 applications). The shorter tree has the probability 1.28 x 10^-11, the longer one 2.71 x 10^-13, i.e. it is 47 times more probable, simply because of the smaller number of factors (and because the nominal rules are common in the treebank), whereas the gold analysis is the longer one. The bias affects absolute probabilities too: they fall exponentially with sentence length, so the probabilities of different sentences cannot be compared directly.

Table 9.9 - Viterbi probability against sentence length (PP-chain family)

Words	Parses	Viterbi probability	Sentence probability	Probability per word (geometric mean)
3	1	7.21 x 10^-6	7.21 x 10^-6	1.93 x 10^-2
6	2	3.61 x 10^-10	5.57 x 10^-10	2.67 x 10^-2
9	5	3.79 x 10^-13	1.01 x 10^-12	4.17 x 10^-2
12	14	3.70 x 10^-17	1.82 x 10^-16	4.27 x 10^-2

Words	Parses	Viterbi probability	Sentence probability	Probability per word (geometric mean)
15	42	2.04 x 10^-20	1.89 x 10^-19	4.87 x 10^-2
18	132	1.12 x 10^-23	2.02 x 10^-22	5.31 x 10^-2

Other consequences

- **Features and normalisation.** The PCFG is defined on the skeleton but the features reject some trees. The probability mass assigned to rejected trees is lost, so the probabilities of the remaining trees do not sum to 1 and the model is not a proper distribution; for ranking this does not matter, since the rankings are not changed, but P(sentence) is an underestimate.
- **Sensitivity to the treebank size and smoothing.** The number of correct gold choices for the eight ambiguous test queries depends on alpha: 2 of 8 for alpha = 0.01, 2 of 8 for alpha = 0.1, 3 of 8 for alpha = 0.5, 3 of 8 for alpha = 1. With only 20 trees the numbers are noise-dominated; Task 11 gives the learning curve.

Evidence summary for Task 9. The PCFG was defined and the normalisation condition verified for all non-terminals; probabilities were estimated by maximum likelihood with add-alpha smoothing from the 20-sentence treebank, with all counts shown; two trees of an ambiguous sentence were multiplied out, the best chosen and the sentence probability computed as a sum; the CNF conversion, pseudocode and a Viterbi table were given; and three independence-assumption failures were illustrated with measured examples.

Task 10 - Implement and Test the Parsers

Guiding enquiry. Can a working implementation reproduce the behaviour you derived by hand?

10.1 Implementation

All parsers are implemented in plain Python 3 (no external libraries; the figures use matplotlib) in the module helpdesk_parsers.py reproduced in Appendix A. The components are:

- **Grammar and lexicon.** BASE_RULES (the 52 rules with their constraint equations), build_lexicon() (251 feature-annotated entries for 165 word forms), and the class Grammar, which also computes the interface features and the left-corner sets. make_GF() builds the feature grammar and make_G_nofeat() the skeleton.
- **Features.** evaluate() unifies the daughters' features under a rule's equations (a union-find over path terms with atomic values); unify() unifies two structures. The same function is called by all three parsers, which guarantees that they apply identical agreement and subcategorization checks (Task 8).
- **Top-down parser.** The class TopDown implements the pseudocode of Task 6 with generators; the flag filtered switches the left-corner and length filters on and off, and a step budget stops runaway searches.
- **Earley parser.** The class Earley implements PREDICTOR, SCANNER and COMPLETER with back-pointers, packs ambiguity, counts and enumerates trees, and (through the use_features flag) integrates unification in the COMPLETER.
- **Probabilistic parser.** The class PCFG estimates smoothed probabilities from trees and the class PCKY implements the binarised Viterbi/inside CKY algorithm with unary closure and feature-carrying items.
- **Other tools.** expand_grammar() splits categories (Task 4), tree utilities compute brackets, statistics and derivations (Task 2), and trace_tools.py reproduces the hand traces of Tasks 6 and 7 on G_mini.

10.2 Test procedure

The procedure is reproducible by running the scripts supplied with the report: exp_a.py (Tasks 1 to 5, 8), exp_b.py (Tasks 6, 7, 9, 11), exp_c.py (Task 10) and exp_d.py (Task 12). Each sentence is lower-cased, stripped of punctuation and split on blanks. For each of the 30 queries, the three parsers are run and the following are recorded: whether a parse was found, the number of parses, the work done (top-down expansion steps, Earley chart states, CKY chart items) and

the run-time measured with the high-resolution timer of the interpreter. The top-down parser has a budget of 6,000,000 steps in the long-sentence experiment and 3,000,000 steps elsewhere; the unfiltered version has a budget of 200,000 steps. Consistency is checked automatically: for each query the number of trees found by the three parsers must agree, the sum of the probabilities of the Earley trees must equal the CKY inside probability, and the probability of the best Earley tree must equal the Viterbi value.

10.3 Results on the 30-sentence query set

Table 10.1 - Parses and work for each query (the filtered top-down parser, Earley and CKY)

Id	Words	Naive top-down	TD parses	TD steps	Earley parses	Earley states	CKY parses	CKY items
Q1	5	found (13)	1	500	1	157	1	18
Q2	3	found (7)	1	55	1	120	1	7
Q3	5	diverges	1	432	1	191	1	25
Q4	5	found (13)	1	489	1	186	1	25
Q5	6	diverges	1	1,536	1	214	1	26
Q6	8	diverges	1	6,883	1	232	1	24
Q7	6	diverges	2	1,389	2	218	2	21
Q8	7	diverges	1	8,680	1	257	1	29
Q9	10	diverges	2	37,695	2	312	2	46
Q10	3	diverges	1	265	1	125	1	13
Q11	4	diverges	1	1,028	1	189	1	23
Q12	5	diverges	1	1,433	1	198	1	19
Q13	7	diverges	1	26,225	1	339	1	40
Q14	7	diverges	2	20,667	2	258	2	35
Q15	8	diverges	1	66,229	1	283	1	36
Q16	8	diverges	2	68,065	2	293	2	53
Q17	9	diverges	5	214,174	5	313	5	51
Q18	9	diverges	7	246,791	7	323	7	51
Q19	6	diverges	2	6,938	2	302	2	39
Q20	8	diverges	5	56,423	5	290	5	39
Q21	6	diverges	1	3,179	1	188	1	23
Q22	6	diverges	1	3,910	1	240	1	40
Q23	4	diverges	1	723	1	145	1	15
Q24	4	diverges	1	718	1	136	1	21
Q25	7	diverges	1	7,645	1	236	1	34

Id	Words	Naive top-down	TD parses	TD steps	Earley parses	Earley states	CKY parses	CKY items
Q26	4	diverges	1	261	1	140	1	14
Q27	5	diverges	1	303	1	182	1	23
Q28	4	diverges	1	211	1	152	1	16
Q29	5	diverges	1	73	1	149	1	23
Q30	5	diverges	1	250	1	159	1	17

Table 10.2 - Run-times in milliseconds (single run per sentence)

Id	Top-down (filtered)	Earley	CKY	Id	Top-down (filtered)	Earley	CKY
Q1	1.5	0.9	0.3	Q16	115.2	1.4	0.7
Q2	0.4	0.4	0.1	Q17	364.1	1.8	0.9
Q3	0.9	1.0	0.4	Q18	489.8	1.8	0.8
Q4	1.2	0.7	0.5	Q19	14.5	1.4	0.4
Q5	4.8	1.4	0.7	Q20	84.3	1.2	0.5
Q6	11.6	1.1	0.4	Q21	1.7	0.7	0.3
Q7	3.3	1.1	0.3	Q22	4.4	1.0	0.5
Q8	19.9	1.1	0.5	Q23	1.8	0.5	0.2
Q9	93.5	1.6	0.9	Q24	1.8	0.5	0.3
Q10	0.9	0.7	0.2	Q25	5.3	1.0	0.5
Q11	4.1	1.3	0.3	Q26	0.6	0.6	0.2
Q12	3.3	0.9	0.3	Q27	0.7	0.7	0.3
Q13	68.5	2.0	0.7	Q28	0.2	0.4	0.3
Q14	49.0	1.5	0.7	Q29	0.1	0.6	0.3
Q15	123.7	1.2	0.5	Q30	0.3	0.6	0.3

Findings. (1) All 30 queries are parsed by all three parsers, and the number of parses agrees in 30 of 30 queries; the ambiguity counts (1 for most, 2 for Q7, Q9, Q14, Q16 and Q19, 5 for Q17 and Q20, 7 for Q18) are the same ones that the hand analysis of Task 2 predicts. (2) The unfiltered top-down parser finds a first parse for only 3 queries (the short ones where the first alternative at every choice is right) and diverges on the rest, as predicted in Task 6. (3) The total times for the 30 queries are 1471 ms for the filtered top-down parser, 31 ms for Earley (with unification) and 13 ms for the probabilistic CKY parser; the top-down parser is slowest on the ambiguous queries (Q17 and Q18 need 214,174 and 246,791 steps), while the chart parsers need a few hundred states or items. (4) The Earley chart contains more entries than the

CKY chart because it stores dotted rules, whereas the CKY chart stores finished constituents of the binarised grammar.

10.4 Special test cases

Left recursion and ambiguous attachment

Table 10.3 - Left-recursive input: unfiltered versus filtered top-down parser, Earley

Sentence	Unfiltered TD (all parses)	Filtered TD parses / steps	Earley parses / states
The form in the office near the library crashed.	diverges (recursion)	2 / 12,113	2 / 232
Submit the form in the office near the library.	diverges (recursion)	5 / 203,764	5 / 309

The unfiltered parser diverges on both left-recursive sentences, while the filtered parser and the Earley parser find all analyses (2 and 5 respectively, the Catalan numbers for the PP sequences) and agree.

Ungrammatical sentences

Table 10.4 - Ungrammatical sentences (agreement, case and subcategorization violations)

Sentence	TD parses / steps	Earley parses / states	CKY parses / items
* The students submits the form.	0 / 447	0 / 110	0 / 13
* The student submit the form.	0 / 507	0 / 124	0 / 19
* These student submitted the form.	0 / 257	0 / 52	0 / 14
* A students paid the fees.	0 / 245	0 / 47	0 / 21
* Him submitted the form.	0 / 96	0 / 46	0 / 13
* Send I the receipt.	0 / 925	0 / 69	0 / 14
* Submit.	0 / 25	0 / 57	0 / 6
* The portal crashed the form.	0 / 419	0 / 155	0 / 11
* Apply in a hostel room.	0 / 1,429	0 / 108	0 / 13

All nine ungrammatical sentences are rejected by all three parsers (no complete S is found), because they all apply the same feature checks. The chart parsers reject them with fewer states than the grammatical sentences of the same length, since the failing equation prunes the analysis early.

Long sentences (up to 18 words)

A family of sentences "Book the room" followed by k prepositional phrases (near the library, in the hostel, at the office, on the portal, for the seminar) was used with k from 0 to 5, i.e. from 3 to 18 words; the number of parses is the Catalan number C(k+1).

Table 10.5 - Long sentences: parses and work

k	Words	Parses (Earley = CKY)	Top-down steps	Earley states	CKY items	Time TD / Earley / CKY (ms)
0	3	1	306	129	17	1 / 0.7 / 0.2
1	6	2	9,163	217	31	17 / 1.0 / 0.4
2	9	5	214,174	313	51	364 / 1.6 / 0.9
3	12	14	4,442,875	418	79	8073 / 2.1 / 1.2
4	15	42	> 6,000,000 (budget)	533	117	9145 / 3.5 / 2.2
5	18	132	> 6,000,000 (budget)	659	167	8775 / 4.4 / 3.3

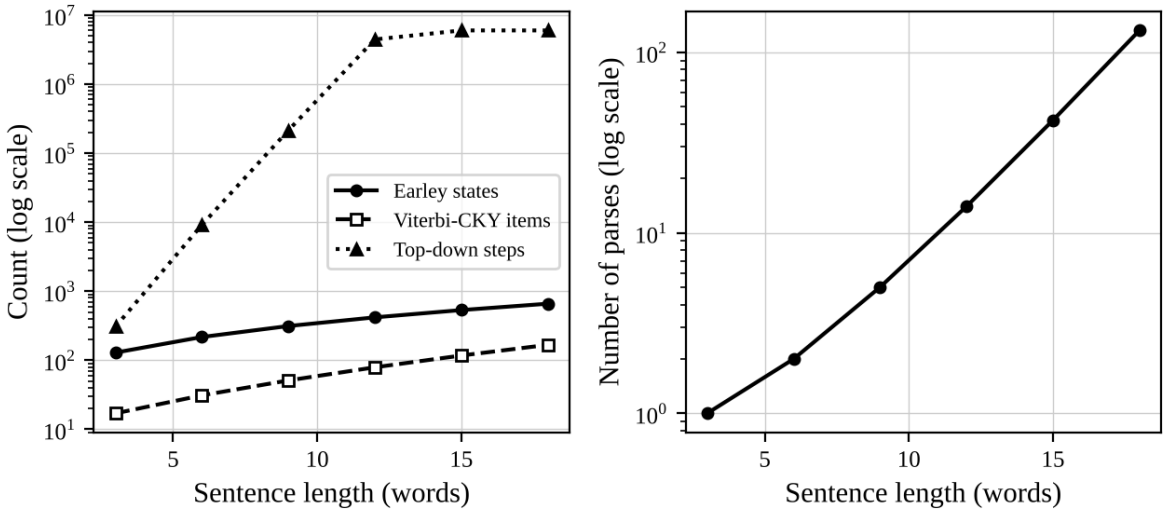


Figure 10.1 - Work against sentence length (left) and number of parses (right), logarithmic vertical axes

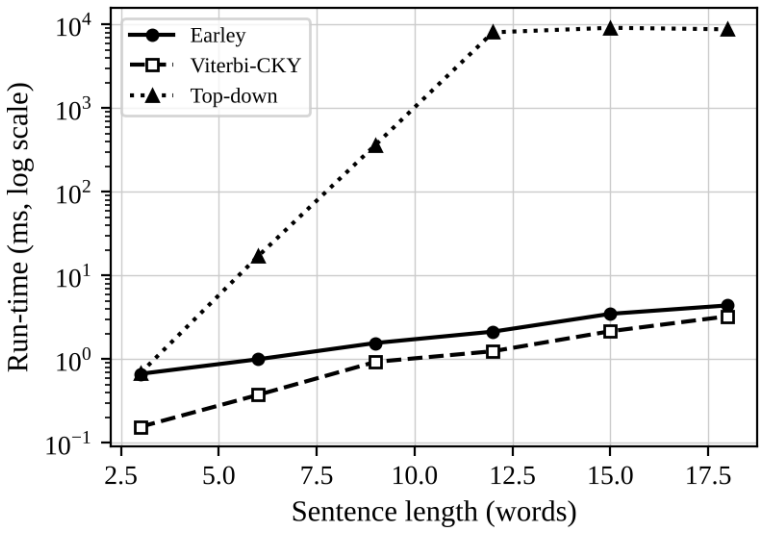


Figure 10.2 - Run-time against sentence length

The plot shows the central result. The number of parses grows by a factor of about three at each step of three words (1, 2, 5, 14, 42, 132); the Earley chart grows only from 129 to 659 states and the CKY chart from 17 to 167 items, i.e. polynomially, while the top-down steps grow from 306 to more than six million. The chart size for this family is almost linear in the length, because the grammar is small and the PP chain has a fixed pattern; the O(n^3) bound is an upper bound for worst-case grammars.

10.5 Reference check against the hand results

Table 10.6 - Hand-computed values against program output

Quantity	Hand-derived value	Program output	Check
Task 7: states in the Earley chart of "the student submitted the form on friday" (G_mini)	52 (Tables 7.1 to 7.8)	52 (program)	match
Task 7: number of trees recovered from the chart	2 (VP and NP attachment)	2	match
Task 9: P(tree 1) by multiplying 14 factors	4.362447 x 10^-8	4.362447 x 10^-8	match
Task 9: P(tree 2) by multiplying 14 factors	2.371532 x 10^-8	2.371532 x 10^-8	match
Task 9: sentence probability, sum of trees against CKY inside value	6.733978 x 10^-8	6.733978 x 10^-8	match
Task 9: Viterbi probability against the best enumerated tree	4.362447 x 10^-8	4.362447 x 10^-8	match
Task 6: parses for k = 0..5 PPs against the Catalan numbers C(k+1)	1, 2, 5, 14, 42, 132	1, 2, 5, 14, 42, 132	match

Quantity	Hand-derived value	Program output	Check
Parse counts of the three parsers on the 30 queries	equal	30 of 30 equal	match

All checks agree. Two further automatic checks run inside the test script: for every one of the 30 queries the sum of the probabilities of all Earley trees equals the CKY inside probability (relative difference below 1e-9), and the Viterbi probability equals the probability of the most probable enumerated tree.

Evidence summary for Task 10. The grammar, the top-down parser (with and without filters), the Earley parser and the Viterbi CKY parser, all with feature checks, are implemented and listed in Appendix A; they were run on 30 queries, 9 ungrammatical sentences, 2 left-recursive cases and 6 sentences of up to 18 words; parse found, parse counts, work and run-time are reported in comparable tables and a plot; and the program output was checked against the hand results of Tasks 6, 7 and 9.

Task 11 - Compare, Recommend and Defend the Parsing Design

Guiding enquiry. Which grammar-and-parser combination would you deploy in the helpdesk, and what evidence supports your choice?

11.1 Comparison of the three parsers

Table 11.1 - Top-down, Earley and probabilistic parsing compared

Criterion	Top-down (filtered)	Earley (with unification)	Probabilistic CKY (Viterbi)
Coverage of the 30 controlled queries	30 of 30 (the unfiltered version: first parse found for 3)	30 of 30	30 of 30 (same grammar)
Handling of left recursion	only with the two filters; otherwise diverges	natural, no change needed	natural (binarised grammar)
Ambiguity handling	enumerates every tree explicitly; no ranking	packs all trees in a forest; counts them without enumeration; no ranking	chooses the most probable tree, can give the sentence probability; the ranking quality depends on the treebank
Worst work on the query set	246,791 steps (Q18)	339 states (Q13)	53 items
Growth with a PP chain (3 to 18 words)	306 to more than 6,000,000 steps	129 to 659 states	17 to 167 items
Time (30 queries, total)	1471 ms	31 ms	13 ms
Feature checks	at each completed daughter (simple)	in the COMPLETER, prunes early	in the items; the grammar must be binarised
Ease of grammar maintenance	high (rules stay as written); order of rules matters	high (any CFG, any order)	lower: needs binarisation and probabilities; probabilities need re-estimation after every rule change
Partial results for failed input	none	yes: completed constituents in the chart (used by the fall-back of Task 12)	yes: completed cells

The measured costs show why speed alone is the wrong criterion: the filtered top-down parser is as fast as the chart parsers on short, unambiguous queries (a few milliseconds), but it needs hundreds of thousands of steps on the ambiguous ones and cannot finish long ambiguous sentences, while the chart parsers stay within a few hundred states or items. The decisive properties are, however, qualitative: only Earley (or CKY) copes with left-recursive rules without grammar surgery, only the probabilistic parser ranks the readings, and only the chart parsers keep partial constituents for input that fails.

11.2 Recommendation

Recommended design: a feature-based CFG (G_F) parsed with an Earley parser that returns the packed forest, the forest being ranked by a PCFG, with a fragment fall-back for the input that has no complete parse. The reasons are the following.

- The feature grammar enforces agreement, case, verb form, subcategorization and preposition selection with 52 rules where the multiplied grammar needs 3469 (Task 8), and all 12 ungrammatical test sentences of Tasks 4 and 5 are rejected, as are the 9 of Task 10.
- Earley copes with the nine recursive rules and with a PP chain of 18 words (132 parses) in 659 states, while the top-down parser needs more than six million steps for the same sentences (Task 10).
- A ranking is needed for the helpdesk, because 8 of 30 queries are ambiguous and the answer depends on the intended structure; the PCFG supplies it at a small extra cost (the CKY chart has 167 items for 18 words).
- Because the forest is shared, the PCFG can be replaced by a better ranker (lexicalised or neural) without changing the grammar or the parser.

Expected accuracy. The grammar parses all 30 queries of the controlled fragment (100% coverage) with correct trees among the candidates; on the Day-4 messages the original grammar gives a complete parse for only 7 of 20 messages and the extended grammar for 17 of 20 (Task 12). The ranking is the weak part: the PCFG trained on 20 trees selects the gold tree for only 2 of the 8 ambiguous test queries, which is poor (the correct tree is not the most probable one in six of the eight cases), and the limitations of Task 9 (no lexical sensitivity, identical attachment preference for every verb) show that this is a structural weakness and not only a data problem. **Limitations of the recommendation:** it needs a larger treebank before the ranking can be trusted; the controlled vocabulary of 165 words means unknown words

block a parse; the agreement class AGR is coarse; and the grammar is hand-written and incomplete for conversational language.

11.3 Learning curve of the ranking and the economics of manual effort

To judge when statistics pay off, the PCFG was re-estimated from the first n treebank sentences (n = 0, 5, 10, 15, 20) and the Viterbi tree was compared with the gold tree for the eight ambiguous test queries. n = 0 means the uniform distribution (all rules of a left-hand side equally likely).

Table 11.2 - Gold-tree accuracy of the PCFG against the number of annotated sentences

Annotated sentences	Correct gold trees	Accuracy	Annotation time at 8 min per sentence
0	5 of 8	63%	0.0 h
5	3 of 8	38%	0.7 h
10	2 of 8	25%	1.3 h
15	1 of 8	13%	2.0 h
20	2 of 8	25%	2.7 h

The curve is **not** a learning curve yet: it falls from 5 of 8 (the uniform model favours trees with few rule applications and rules from categories with few alternatives, which by chance coincides with the gold tree in five queries and has no linguistic meaning) to between 1 and 3 correct when the treebank, whose three VP-attached and three NP-attached PPs favour VP attachment by the rule-frequency effect of Task 9, is added. With eight test items, one sentence is 12.5 percentage points, so no trend can be claimed from these numbers. The honest conclusion is that 20 sentences are far too few; the experiment shows the mechanism and the size of the effect of data, not the accuracy of the final system.

Manual effort

Table 11.3 - Effort log for Tasks 1 to 11 (estimates with explicit unit times)

Activity	Hours	Cost at INR 500 per hour
Grammar writing: 52 rules with constraints, 12 min per rule	10.4	INR 5,200
Lexicon: 165 word forms with features, 2 min per word	5.5	INR 2,750
Query set: 30 sentences written and checked, 4 min each	2.0	INR 1,000

Activity	Hours	Cost at INR 500 per hour
Treebank annotation by selection: 20 sentences, 8 min each	2.7	INR 1,333
Grammar testing and revision (the 12 + 12 test sentences, rule debugging)	6.0	INR 3,000
Implementation of parsers and tools	16.0	INR 8,000
Experiments, tables and report	8.0	INR 4,000
Total	**50.6**	**INR 25,283**

The unit times are assumptions of a trained student language engineer working with the parser available; the hourly rate is an assumed cost of labour. Hand-written grammar and lexicon cost about 31% of the effort; annotation costs only 2.7 hours because the parser proposes the candidates.

When does the statistical approach become more economical?

Hand-written rules and treebank annotation buy different things. A rule buys coverage of one construction; an annotated sentence buys, on average, 13 rule applications of evidence for the ranking of all constructions it contains. Task 12 gives a measured value for rule writing: eleven rules and fifteen words (about 4.2 hours at the unit times above) raised complete parses of the messages from 7 to 17 of 20, about 12 percentage points per hour. Two effects make this rate fall: (i) the tail of constructions is long, so every further rule is used by a smaller fraction of the messages (diminishing returns), and (ii) every added rule can interact with the existing ones and create new ambiguity (the imperative rule of Task 3 raised the average number of parses from 1.17 to 2.23), so the testing effort per rule grows with the size of the grammar. Annotation by selection costs 8 minutes per sentence regardless of the size of the grammar, but needs the grammar to propose the right tree, and its benefit per sentence is highest while the model is far from converged. A reasonable switch-over point is when (a) the hand-written grammar has several hundred rules, so that each new rule costs more than an annotated sentence (for instance 0.2 hours of writing plus 0.2 hours of regression testing per rule against 0.13 hours per sentence) and (b) the main error is no longer missing analyses but the choice among existing ones, which is exactly the situation of the helpdesk today: coverage of the controlled fragment is complete and the ranking accuracy is low. Under these assumptions about 500 annotated sentences (about 67 hours, INR 33,000) are a sensible next investment; it is the point at which a vanilla PCFG begins to give attachment preferences that are better than a constant guess, although published experience with vanilla treebank PCFGs suggests that lexicalisation is still

necessary for good attachment accuracy. These last figures are projections from the unit times above, not measurements.

Evidence summary for Task 11. The three parsers were compared on coverage, ambiguity handling, cost and maintenance with measured values; a design was recommended with its expected accuracy and limitations; the manual effort was logged and the economic switch-over between rule-writing and annotation was discussed with a measured learning curve.

Task 12 - Day-4 Design Challenge: Re-evaluate and Adapt the Design

Changed requirement. The helpdesk must accept conversational, multi-clause and partly ungrammatical messages, for example "Hi, can you tell me when the exams that were postponed will be held, and also whether the hostel fee is due?", with coordinated clauses, embedded questions, relative clauses and missing punctuation. At least 20% of the messages are expected to contain a construction not covered by the grammar.

To make the evaluation concrete, a set of twenty messages (M1 to M20) was written in the style of student messages, including the example above; they are listed in Table 12.1. They were parsed with the original grammar G_F and with the extended grammar of Part C.

Part A - Impact analysis

Table 12.1 - The twenty messages and the original grammar

Id	Message	Words	Complete parse (original)	Problem for the original grammar
M1	Hi, can you tell me when the exams that were postponed will be held, and also whether the hostel fee is due?	22	no	greeting hi, also, whether unknown; embedded questions
M2	Can you tell me when the exams will be held?	10	yes	parsed, but when the exams will be held analysed as an adverbial clause (wrong structure)
M3	Please tell me where I can find the timetable.	9	no	please, find unknown
M4	Do you know whether the library is open on Friday?	10	no	know, whether unknown
M5	Tell me if the portal crashed.	6	yes	parsed, but if clause analysed as adverbial (wrong structure)
M6	Hi, did the office publish the results?	7	no	greeting hi unknown
M7	Send me the receipt and tell me that the fee is paid.	12	yes	covered (VP coordination)
M8	Who collected the certificates that the office sent?	8	yes	covered (object relative clause)
M9	The students that paid the fee collected the receipts.	9	yes	covered (subject relative clause)
M10	Is the fee due on Friday?	6	yes	covered

Id	Message	Words	Complete parse (original)	Problem for the original grammar
M11	Can you tell me when the library closes and whether the hostel is open?	14	no	whether unknown; coordinated embedded questions
M12	hi can you tell me when the exams that were postponed will be held	14	no	as M1, without punctuation
M13	please send me the receipt for the fee	8	no	please unknown
M14	Do you know who collected the certificates?	7	no	know unknown
M15	Tell me what you submitted.	5	no	embedded wh-question with a gap (what you submitted) not in the grammar
M16	I paid the fee but the portal crashed.	8	no	but unknown (clause coordination with a new conjunction)
M17	Need the timetable for the exam.	6	no	ellipsis (no subject), need unknown
M18	The exams that were postponed.	5	no	fragment: a noun phrase, not a sentence
M19	u can tell me when the results will be published	10	yes	parsed, but when ... analysed as adverbial (wrong); needs the repair of u to you
M20	Hello, when does the library close and is the portal open?	11	no	hello unknown; two coordinated questions

Constructions that the original grammar cannot parse. (1) Embedded questions with a wh-word or whether (M1, M3, M4, M11, M12, M14, M15, M20) as the complement of tell me and know: the grammar has only the complement clause introduced by that. (2) Coordination of clauses and of embedded questions, especially with and also and with a new conjunction but (M1, M11, M16, M20). (3) Greetings and politeness words (hi, hello, please) in front of or inside the message. (4) Fragments and ellipsis (M17, M18). (5) Unknown vocabulary and noise (u for you). Overall 13 of the 20 messages (65%) have no complete parse in the original grammar, far above the 20% mentioned in the brief. In addition, three messages (M2, M5, M19) are parsed but analysed wrongly: because when and if are listed as subordinators, tell me when the exams will be held is analysed as the command tell me plus an adverbial clause, not as the embedded question; this is worse than a failure because the error is silent.

Affected assumptions. (i) Controlled vocabulary: 11 of the 20 messages contain at least one word that is not in the lexicon. (ii) Fully grammatical input: the messages contain fragments, missing punctuation, lower-case sentence starts and abbreviations; the grammar, which treats punctuation as irrelevant, is unaffected by the missing punctuation itself, but the loss of commas removes the clues to clause boundaries and increases the ambiguity of coordination. (iii) Small ambiguity: the messages are longer (up to 22 words, against 10 for the longest query) and contain several coordination points, so the number of potential bracketings of coordination and PP attachment multiplies. (iv) Hand-written coverage: 65% of the messages fall outside the language of the grammar.

Effect of longer sentences. In the PP-chain family of Task 10 the number of parses grew by a factor of 132 from 3 to 18 words (Catalan growth), while the Earley chart grew by a factor of 5.1 and the top-down work from 306 steps to more than six million. For a message with p attachment sites and c coordination points, the upper bound on the number of trees grows as the Catalan number C(p+c+1) per clause, so the number of parses can grow exponentially with the length, whereas the chart size grows roughly like the third power of the length in the worst case and was close to linear in the measurements. The practical consequence is that the parser must work on a forest and must not enumerate trees, and that unification must prune early; in the 22-word message M1 the extended grammar creates 741 chart states and returns 1 tree.

Part B - Evaluation of the original design

Three messages (M1, M3, M11) were run through the original grammar with the top-down parser and the Earley parser, and the first failure was recorded. The longest viable prefix is the largest k for which the Earley chart column S[k] is non-empty.

Table 12.2 - What fails first in the original design

Message	Words	First failure in the Earley chart	Top-down steps until failure	Earley states	Earley states after redesign (trees)
M1	22	word 1: hi	7	38	741 (1)
M3	9	word 1: please	7	38	305 (1)
M11	14	word 10: whether	> 2,000,000 (budget)	347	484 (1)

M1. The very first word hi is unknown, so the Earley parser stops after 38 states and the top-down parser after 7 steps (it tries the seven rules for S, none of which can begin with hi). To

see what would fail next, the message was cut step by step (Table 12.3); the hand-trace shows that each repair reveals the next gap.

Table 12.3 - Successive failures of the original grammar on M1

Variant of M1	Words	Complete parse	First failing word	Earley states
M1 as sent	22	no	hi	38
M1 without 'hi'	21	no	also	466
M1 without 'hi', 'also'	20	no	whether	466
M1 without 'hi','also','whether'	19	yes (but see below)	-	622

After hi is removed, the chart is built for the first 14 words (up to held) and stops at also; without also the next failure is whether; and if all three words are removed the 19-word remainder is accepted, but with the **wrong structure**: when the exams that were postponed will be held is read as an adverbial clause and the final clause is attached by clause coordination. This shows that the original design fails in two ways, loudly (unknown words) and silently (wrong analyses). **M3** fails at please in the first position (and the verb find is also unknown). **M11** is the instructive case for the top-down parser: the original grammar accepts the first nine words (can you tell me when the library closes and) and fails at whether, which is word 10; the top-down parser needed more than two million steps before it could conclude that no parse exists, because it re-tries all attachments of the coordination and the adverbial clause after each failure, while the Earley chart reached the same conclusion with 347 states.

Growth of ambiguity and chart size. Using the Task 7 and Task 10 results: the number of parses of a sentence with k attachment sites is the Catalan number C(k+1) (1, 2, 5, 14, 42, 132 for k = 0 to 5), i.e. it grows by about a factor of four per additional site; the Earley chart for the same sentences grows by a constant of about 106 states per additional PP (a roughly linear growth in this family) and the CKY chart by about 30 items. The conversational messages will lie between these extremes: their coordination points add parses multiplicatively, and the chart cost per word will not exceed the worst-case cubic bound. A three-clause message of 25 words with four attachment sites and two coordination points can therefore have thousands of trees, which no parser should enumerate.

Is another strategy more economical? Yes, for the failures: (1) the top-down parser is clearly unsuitable (the M11 failure alone exhausts a two-million-step budget); (2) the Earley chart can be reused for partial parsing: the completed constituents in the chart (S, CP, NP, VP, PP) are

available even when no complete S exists, and a left-to-right dynamic program over them gives the minimum set of fragments that covers the message while skipping the unknown words. This fall-back costs one Earley run per start position (at most n runs), i.e. a small constant factor over the full parse, and covers messages that no hand-written grammar will fully parse. The performance of this fall-back is measured in Part C.

Part C - Adaptation or redesign

Decision: MODIFY the grammar and SUPPLEMENT the parser; do not redesign. The core of the original design (feature grammar, Earley parser, PCFG ranking) is retained because its failures are failures of coverage in a few identifiable constructions, not of the method: the chart parser has no difficulty with the longer sentences, the feature mechanism extends to the new rules, and the parse forest is the right input for a ranker. A redesign (for example a statistical parser trained from a large treebank) would cost far more annotation than is available (Task 11). The changes are of four kinds.

11. **Extend the grammar.** Eleven rules are added (Table 12.4): embedded questions (CP -> WhAdv S, CP -> Whether S, CP -> WhNP VP, CP -> WhNP S/NP), coordination of embedded questions (CP -> CP Conj CP), the coordinator with an adverb (ConjP -> Conj Adv, used by S, CP and VP coordination) and greetings and politeness words (S -> Intj S, S -> S Intj). The embedded questions reuse the existing frame V NP CP of tell, which is how the verb frames are already generalised: the complement CP now has four forms. The gap in what you submitted uses the existing S/NP rules and the agreement constraint of CP -> WhNP VP is <WhNP AGR> = <VP AGR>. The lexicon gets 15 new words (also, find, finds, found, hello, hey, hi, knew, know, knows, opened, opens, please, thanks, whether), including the new frames of know (clause or noun phrase), find and open.
12. **Add a normaliser and a fall-back.** A normaliser maps frequent abbreviations (u, pls, plz, thx) to words of the lexicon; lower-casing and the removal of punctuation were already part of the tokeniser. The fragment fall-back (Part B) covers the rest.
13. **Re-estimate the PCFG.** Ten new sentences with the new constructions were annotated by selection (Table B.3 in Appendix B) and added to the 20 old ones; the probabilities of Table 12.4 come from the 30-tree treebank. The additional annotation costs about 1.3 hours. The old rule CP -> Comp S now competes with four new CP rules, so its probability falls from 1.00 (the only CP rule) to 0.103, and the embedded-question rules CP -> WhAdv S and CP -> Whether S, each seen three times, receive 0.290.

14. **Keep a human in the loop.** Messages for which the fall-back yields only fragments are passed to a human agent with the recognised fragments highlighted; they also supply the next batch of constructions for the grammar.

Table 12.4 - New grammar rules and their estimated probabilities

Rule	Constraints	Count in 30 trees	Probability (new)	Probability before
CP → Comp S	(none)	1	0.1028	1.0000
S → Intj S	(none)	4	0.0976	-
S → S Intj	(none)	0	0.0024	-
CP → WhAdv S	(none)	3	0.2897	-
CP → Whether S	(none)	3	0.2897	-
CP → WhNP VP	1.AGR=2.AGR; 2.VFORM=fin	1	0.1028	-
CP → WhNP S/NP	(none)	1	0.1028	-
CP → CP Conj CP	(none)	1	0.1028	-
ConjP → Conj Adv	(none)	0	1.0000	-
S → S ConjP S	(none)	0	0.0024	-
CP → CP ConjP CP	(none)	0	0.0093	-
VP → VP ConjP VP	0.AGR=1.AGR; 1.AGR=3.AGR; 0.VFORM=1.VFORM; 1.VFORM=3.VFORM	0	0.0023	-

Table 12.5 lists the effect of the redesign on all twenty messages. The original grammar finds a complete parse for 7 of 20 messages (three of them with the wrong structure), the extended grammar for 17.

Table 12.5 - Results of the redesign on the twenty messages

Id	Words	Original: parse	Extended: parse (trees)	Earley states (original / extended)	Best tree found
M1	22	no	yes (1)	38 / 741	probability 1.2 x 10^-32
M2	10	yes	yes (2)	341 / 306	probability 5.6 x 10^-15
M3	9	no	yes (1)	38 / 305	probability 6.8 x 10^-15
M4	10	no	yes (3)	87 / 325	probability 3.5 x 10^-15

Id	Words	Original: parse	Extended: parse (trees)	Earley states (original / extended)	Best tree found
M5	6	yes	yes (2)	249 / 212	probability 1.6 x 10^-9
M6	7	no	yes (1)	38 / 202	probability 8.2 x 10^-9
M7	12	yes	yes (2)	566 / 472	probability 1.6 x 10^-17
M8	8	yes	yes (1)	256 / 209	probability 1.5 x 10^-11
M9	9	yes	yes (1)	268 / 249	probability 1.5 x 10^-11
M10	6	yes	yes (1)	210 / 152	probability 4.6 x 10^-9
M11	14	no	yes (1)	347 / 484	probability 1.8 x 10^-17
M12	14	no	yes (2)	38 / 442	probability 1.6 x 10^-22
M13	8	no	yes (2)	38 / 256	probability 1.6 x 10^-10
M14	7	no	yes (1)	87 / 213	probability 7.6 x 10^-11
M15	5	no	yes (1)	141 / 170	probability 6.3 x 10^-8
M16	8	no	no	178 / 132	fragments only
M17	6	no	no	38 / 42	fragments only
M18	5	no	no	149 / 158	fragments only
M19	10	yes	yes (3)	336 / 303	probability 2.8 x 10^-15
M20	11	no	yes (2)	38 / 319	probability 8.2 x 10^-19

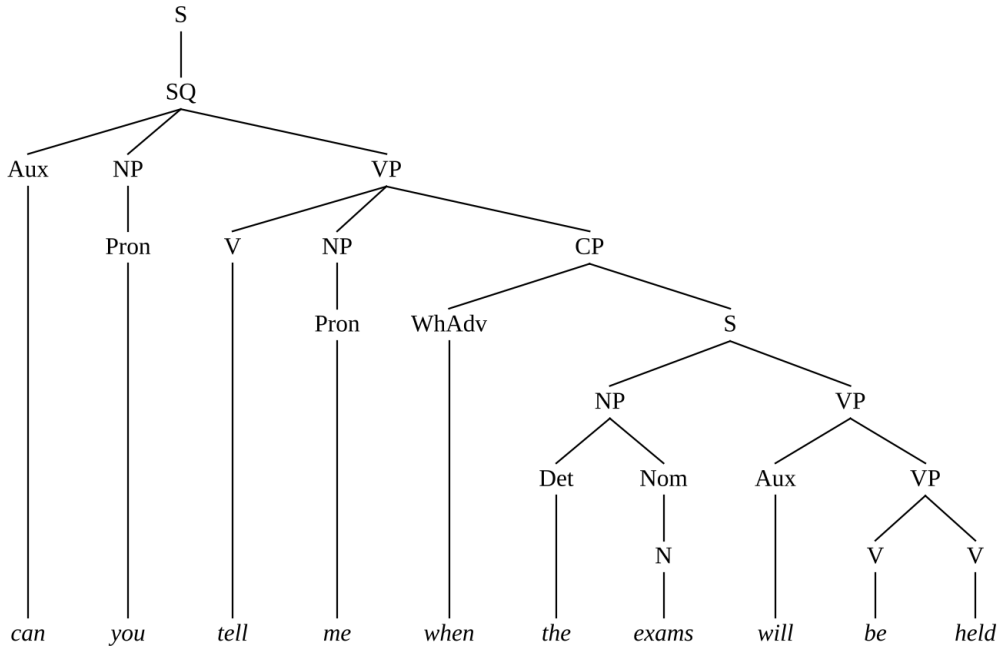


Figure 12.1 - Best tree of M2 "Can you tell me when the exams will be held?" with the extended grammar (embedded question)

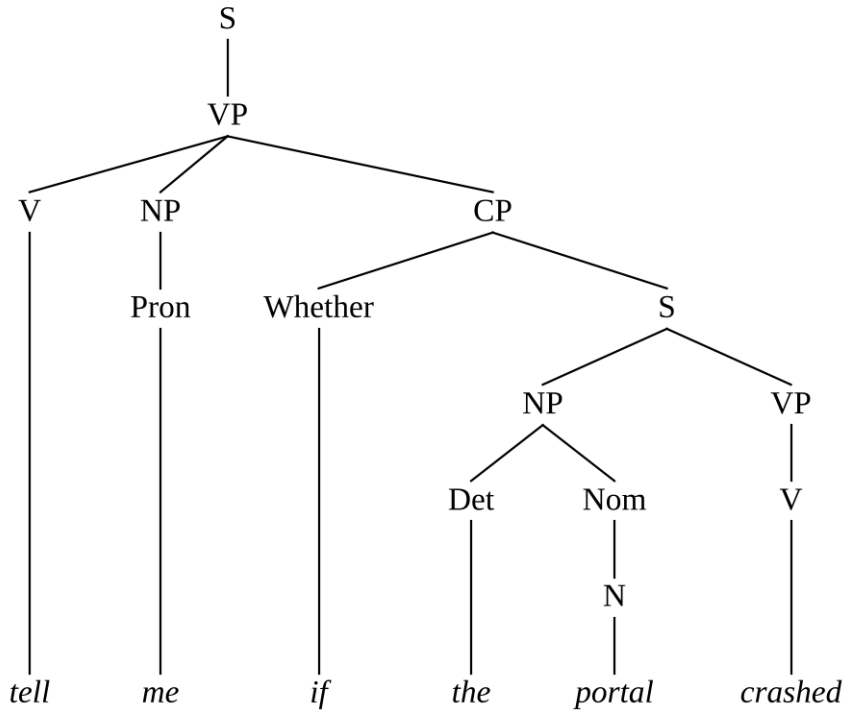


Figure 12.2 - Best tree of M5 "Tell me if the portal crashed." (embedded question, not an adverbial clause)

The extended grammar and the PCFG re-estimated from 30 trees choose the embedded-question analysis for M2, M5 and M19, which repairs the silent errors of the original design.

All three messages remain ambiguous (two or three trees) because the adverbial reading is still available, and the correct choice rests on a few counts of the treebank; it is the kind of decision that needs the lexical sensitivity discussed in Task 9 (the verb tell takes questions, a property of the verb).

Fall-back results

Table 12.6 - Fragment fall-back: skipped words / fragments for the original and the extended grammar

Id	Words	Original (skipped / fragments)	Extended (skipped / fragments)	Fragments with the extended grammar
M1	22	4 / 2	0 / 1	S: hi can you tell me when the exams that were postponed will be held and also whether the hostel fee is due
M2	10	0 / 1	0 / 1	S: can you tell me when the exams will be held
M3	9	4 / 3	0 / 1	S: please tell me where i can find the timetable
M4	10	3 / 2	0 / 1	S: do you know whether the library is open on friday
M5	6	0 / 1	0 / 1	S: tell me if the portal crashed
M6	7	1 / 1	0 / 1	S: hi did the office publish the results
M7	12	0 / 1	0 / 1	S: send me the receipt and tell me that the fee is paid
M8	8	0 / 1	0 / 1	S: who collected the certificates that the office sent
M9	9	0 / 1	0 / 1	S: the students that paid the fee collected the receipts
M10	6	0 / 1	0 / 1	S: is the fee due on friday
M11	14	2 / 2	0 / 1	S: can you tell me when the library closes and whether the hostel is open
M12	14	1 / 1	0 / 1	S: hi can you tell me when the exams that were postponed will be held
M13	8	1 / 1	0 / 1	S: please send me the receipt for the fee
M14	7	2 / 2	0 / 1	S: do you know who collected the certificates
M15	5	2 / 2	0 / 1	S: tell me what you submitted
M16	8	1 / 2	1 / 2	S: i paid the fee | [skip: but] | S: the portal crashed
M17	6	1 / 1	1 / 1	[skip: need] | NP: the timetable for the exam
M18	5	0 / 1	0 / 1	NP: the exams that were postponed

Id	Words	Original (skipped / fragments)	Extended (skipped / fragments)	Fragments with the extended grammar
M19	10	0 / 1	0 / 1	S: you can tell me when the results will be published
M20	11	1 / 1	0 / 1	S: hello when does the library close and is the portal open

Effect on coverage, ambiguity, cost and annotation effort. Coverage: complete parses rise from 7 of 20 (35%, three of them with the wrong structure) to 17 of 20 (85%); with the fall-back, the fraction of words that are covered by a constituent rises from 87.7% (original grammar) to 98.9% (extended grammar), where the only words left out are but in M16 and need in M17. The three messages without a complete parse (M16, M17, M18) are handled by fragments: M16 yields the two clauses i paid the fee and the portal crashed, M17 the noun phrase the timetable for the exam and M18 the noun phrase the exams that were postponed. Ambiguity: the parsed messages have on average 1.6 trees (range 1 to 3); the new rules add the second analysis to M2, M5 and M19 only. Parsing cost: the total number of Earley states over the twenty messages rises from 3,481 to 5,692 (because messages that used to fail early are now parsed in full); the 22-word message M1 needs 741 states. Effort: eleven new rules (about 3.7 hours at 20 minutes per rule including testing), 15 words (0.5 hours), ten annotated trees (1.3 hours), the normaliser and the fall-back (5 hours) and testing (3 hours): about 13.5 hours, INR 6,750 at the rate of Task 11, which is about 27% of the effort of the original design.

Part D - Evidence-based defence of the decision

The decision follows from the evidence.

15. The failures are local. Table 12.1 and Table 12.3 show that each failure is caused by a specific missing construction or word, which means that the grammar can be repaired incrementally. The extension made 17 of the 20 messages parsable with 11 rules and 15 words, which is the measured proof that modification is enough; the remaining three failures (an unknown conjunction, an unknown verb and a fragment) need vocabulary or the fall-back, not new theory.
16. The parser is not the bottleneck. The Earley chart grows polynomially (129 to 659 states from 3 to 18 words, 741 states for the 22-word M1), while the number of trees grows by the Catalan numbers and the top-down parser exhausts a two-million-step budget on M11. Therefore the parsing strategy must be kept (Earley, forest) and the top-down strategy rejected, whatever the grammar does.

17. Feature checks stay essential. With unification the 22-word message M1 receives exactly one tree, whereas the skeleton grammar produced 43% more trees on the query set (Task 8); without the features, the longer messages would produce many more spurious analyses.
18. Silent errors need statistics. Three messages were parsed wrongly by the original grammar without any warning. The remedy in the extended grammar is a statistical preference (the PCFG re-estimated from ten more trees), which is why the PCFG ranking stays in the design, and why the annotation effort is spent on constructions where the grammar is ambiguous, not on those where it is not.
19. A redesign is not justified by the evidence. A fully statistical parser would need hundreds to thousands of annotated sentences (Task 11: about 67 hours for 500), while the modification cost 13.5 hours and already reached 85% complete parses and about 99% word coverage with the fall-back.

Remaining limitations. (1) The evaluation set has only twenty messages, written by the author in the style of student messages; the coverage figures are indicative, not statistical estimates, and real messages will contain constructions that no sample of twenty reveals (spelling errors, code-mixing, emoji, questions without a verb). (2) The ranking of the PCFG is not reliable for attachment (Task 9): two of the three silent errors were repaired by the re-estimation, but the repair is based on three annotated examples per construction. (3) The agreement class AGR is coarse, and the lexicon is controlled; every new domain word needs a lexical entry with features. (4) The fall-back returns fragments, not meaning; the semantic component must decide what to do with the exams that were postponed as a fragment. (5) Spelling normalisation is limited to a small table; a proper spell-checker or a character-level model would be needed for real noise. The next steps are therefore: collect real messages, annotate by selection those with new constructions, move to a lexicalised ranker once about 500 trees exist, and measure complete-parse coverage and fragment quality on a held-out sample of real messages.

Evidence summary for Task 12. The impact of the changed requirement was analysed on twenty messages (65% outside the original grammar, three silently mis-analysed); the original design was traced on three messages with the first failures located; the decision to modify and supplement was justified and quantified for coverage (35% to 85% complete parses, 87.7% to 98.9% word coverage with the fall-back), ambiguity, cost and effort; and the decision was defended with the evidence generated in the preceding tasks.

Appendix A - Complete Source Code

The listings below are the complete source files used for every result in this report. They can be run with Python 3.8 or later without installing any package (matplotlib is needed only for the figures). helpdesk_parsers.py contains the grammar and all parsers; trace_tools.py reproduces the traces of Tasks 6 and 7; data_sets.py holds the query set, the treebank sentences and the gold-tree predicates; exp_c.py is the test driver of Task 10. The scripts exp_a.py, exp_b.py and exp_d.py (supplied with the report) generate the tables of Tasks 1 to 9, 11 and 12 in the same way.

A.1 helpdesk_parsers.py - grammar, features, top-down, Earley and probabilistic CKY parsers

```
"""
helpdesk_parsers.py
Grammar-based parsing toolkit for the University Helpdesk Assistant


Contents
--------
1.  Feature utilities      : flat attribute-value structures and unification
2.  Lexicon and grammar    : the helpdesk fragment (feature-augmented CFG G_F)
3.  Grammar transformations: category-splitting expansion (plain CFG variants)
4.  Tree utilities         : Node, bracketed notation, statistics, derivations
5.  Top-down parser        : backtracking recursive descent (naive and filtered)
6.  Earley parser          : chart with back-pointers and unification
7.  PCFG                   : maximum-likelihood estimation with smoothing
8.  Probabilistic CKY      : binarised grammar, unary closure, Viterbi + inside
"""
import itertools
import math
import time
from collections import defaultdict, OrderedDict

 1. FEATURE UTILITIES
 A feature structure is a flat dict {feature: atomic value}.  A missing feature
 is unspecified and unifies with anything.  Rule constraints are written as
strings such as "1.AGR=2.AGR" (path equation between two daughters; position 0
is the mother) or "2.CASE=acc" (a constant).


def fkey(d):
    """Hashable form of a feature dict."""
    return tuple(sorted(d.items()))


def _term(x):
```

```
    p = x.split('.')
    if len(p) == 2 and p[0].isdigit():
        return ('t', int(p[0]), p[1])
    return ('c', x)


def compile_constraints(strs):
    out = []
    for s in strs:
        a, b = s.split('=')
        out.append((_term(a), _term(b)))
    return out


def evaluate(cons, children):
    """Unify the daughters' features under the rule's equations.

    children : list with one entry per daughter (a dict, or None if that
               daughter has not been recognised yet).
    Returns the mother's feature dict, or None if unification fails.
    """
    if not cons:
        return {}
    parent, val = {}, {}

    def find(t):
        while t in parent:
            t = parent[t]
        return t

    def setv(t, v):
        r = find(t)
        cur = val.get(r)
        if cur is None:
            val[r] = v
            return True
        return cur == v

    def union(a, b):
        ra, rb = find(a), find(b)
        if ra == rb:
            return True
        va, vb = val.get(ra), val.get(rb)
        if va is not None and vb is not None and va != vb:
            return False
        parent[ra] = rb
        if vb is None and va is not None:
            val[rb] = va
        return True
```

```
    terms = set()
    for a, b in cons:
        for x in (a, b):
            if x[0] == 't':
                terms.add(x)
    for x in terms:
        i, f = x[1], x[2]
        if 1 <= i <= len(children) and children[i - 1] is not None:
            v = children[i - 1].get(f)
            if v is not None and not setv(x, v):
                return None
    for a, b in cons:
        if a[0] == 't' and b[0] == 't':
            if not union(a, b):
                return None
        elif a[0] == 't':
            if not setv(a, b[1]):
                return None
        elif b[0] == 't':
            if not setv(b, a[1]):
                return None
    out = {}
    for x in terms:
        if x[1] == 0:
            v = val.get(find(x))
            if v is not None:
                out[x[2]] = v
    return out


def unify(f1, f2):
    """Unification of two flat feature structures (None on failure)."""
    out = dict(f1)
    for k, v in f2.items():
        if k in out and out[k] != v:
            return None
        out[k] = v
    return out

2. LEXICON AND GRAMMAR (feature-augmented CFG, G_F)

SUBCAT_NAMES = OrderedDict([
    ('INTR', 'V-intr'), ('NP', 'V-trans'), ('NP_NP', 'V-ditrans'),
    ('CP', 'V-comp'), ('NP_CP', 'V-trcomp'), ('PP', 'V-pp'),
    ('NP_PP', 'V-trpp'), ('ADJ', 'V-adj'), ('PASS', 'V-pass')])

# (lemma, base, 3sg, past, past-participle, [(frame, PFORM or None), ...])
VERBS = [
    ('crash', 'crash', 'crashes', 'crashed', 'crashed', [('INTR', None)]),
    ('start', 'start', 'starts', 'started', 'started', [('INTR', None)]),
```

```
    ('close', 'close', 'closes', 'closed', 'closed', [('INTR', None), ('NP', None)]),
    ('submit', 'submit', 'submits', 'submitted', 'submitted', [('NP', None), ('NP_PP', 'to')]),
    ('check', 'check', 'checks', 'checked', 'checked', [('NP', None), ('CP', None)]),
    ('pay', 'pay', 'pays', 'paid', 'paid', [('INTR', None), ('NP', None)]),
    ('send', 'send', 'sends', 'sent', 'sent', [('NP', None), ('NP_NP', None), ('NP_PP', 'to')]),
    ('show', 'show', 'shows', 'showed', 'shown', [('NP', None), ('NP_NP', None)]),
    ('tell', 'tell', 'tells', 'told', 'told', [('NP', None), ('NP_CP', None)]),
    ('say', 'say', 'says', 'said', 'said', [('CP', None)]),
    ('apply', 'apply', 'applies', 'applied', 'applied', [('PP', 'for')]),
    ('register', 'register', 'registers', 'registered', 'registered', [('PP', 'for')]),
    ('wait', 'wait', 'waits', 'waited', 'waited', [('PP', 'for')]),
    ('book', 'book', 'books', 'booked', 'booked', [('NP', None), ('NP_NP', None)]),
    ('postpone', 'postpone', 'postpones', 'postponed', 'postponed', [('NP', None)]),
    ('collect', 'collect', 'collects', 'collected', 'collected', [('NP', None)]),
    ('renew', 'renew', 'renews', 'renewed', 'renewed', [('NP', None)]),
    ('hold', 'hold', 'holds', 'held', 'held', [('NP', None)]),
    ('publish', 'publish', 'publishes', 'published', 'published', [('NP', None)]),
]

NOUNS = [('student', 'students'), ('exam', 'exams'), ('form', 'forms'), ('fee', 'fees'),
        ('hostel', 'hostels'), ('room', 'rooms'), ('library', 'libraries'),
        ('timetable', 'timetables'), ('receipt', 'receipts'), ('notice', 'notices'),
        ('seminar', 'seminars'), ('placement', 'placements'), ('portal', 'portals'),
        ('deadline', 'deadlines'), ('book', 'books'), ('charge', 'charges'),
        ('result', 'results'), ('course', 'courses'), ('office', 'offices'),
        ('certificate', 'certificates'), ('hall', 'halls')]


def build_lexicon():
    L = []

    def add(w, c, **f):
        L.append((w, c, f))

    for w, a in [('the', None), ('a', '3sg'), ('this', '3sg'), ('these', 'non3sg'),
                 ('those', 'non3sg'), ('my', None), ('our', None), ('every', '3sg'),
                 ('some', None)]:
        add(w, 'Det', **({'AGR': a} if a else {}))
    for sg, pl in NOUNS:
        add(sg, 'N', AGR='3sg', NUM='sg', PER=3)
        add(pl, 'N', AGR='non3sg', NUM='pl', PER=3)
    add('staff', 'N', NUM='sg/pl', PER=3)
    for w in ('friday', 'monday'):
        add(w, 'PN', AGR='3sg', NUM='sg', PER=3)
    for w, a, c, per, num in [('i', 'non3sg', 'nom', 1, 'sg'), ('you', 'non3sg', None, 2, 'sg/pl'),
                              ('he', '3sg', 'nom', 3, 'sg'), ('she', '3sg', 'nom', 3, 'sg'),
                              ('it', '3sg', None, 3, 'sg'), ('we', 'non3sg', 'nom', 1, 'pl'),
                              ('they', 'non3sg', 'nom', 3, 'pl'), ('me', 'non3sg', 'acc', 1, 'sg'),
                              ('him', '3sg', 'acc', 3, 'sg'), ('us', 'non3sg', 'acc', 1, 'pl'),
                              ('them', 'non3sg', 'acc', 3, 'pl')]:
```

```
        f = {'AGR': a, 'PER': per, 'NUM': num}
        if c:
            f['CASE'] = c
        add(w, 'Pron', **f)
    add('who', 'WhPron', AGR='3sg')
    add('what', 'WhPron', AGR='3sg')
    add('which', 'WhDet')
    add('what', 'WhDet')
    for w in ('when', 'where', 'how', 'why'):
        add(w, 'WhAdv')
    for w in ('due', 'late', 'available', 'pending', 'new', 'final', 'open'):
        add(w, 'Adj')
    for w in ('very', 'already'):
        add(w, 'Adv')
    for w in ('in', 'on', 'near', 'for', 'to', 'with', 'at', 'by', 'from', 'after', 'before'):
        add(w, 'P', PFORM=w)
    add('and', 'Conj', AGR='non3sg')
    add('or', 'Conj')
    add('that', 'Comp')
    for w in ('if', 'because', 'when', 'unless'):
        add(w, 'Sconj')
    for w, a in [('will', None), ('can', None), ('should', None), ('did', None),
                 ('do', 'non3sg'), ('does', '3sg')]:
        f = {'VFORM': 'fin'}
        if a:
            f['AGR'] = a
        add(w, 'Aux', **f)
    # main verbs and the copula
    for lemma, base, s3, past, pp, frames in VERBS:
        for frame, pf in frames:
            extra = {'SUBCAT': frame}
            if pf:
                extra['PFORM'] = pf
            add(base, 'V', VFORM='base', **extra)
            add(base, 'V', VFORM='fin', AGR='non3sg', **extra)
            add(s3, 'V', VFORM='fin', AGR='3sg', **extra)
            add(past, 'V', VFORM='fin', **extra)
            if frame == 'NP':
                add(pp, 'V', VFORM='pp', **extra)
    for frame in ('ADJ', 'NP', 'PP', 'PASS'):
        add('be', 'V', VFORM='base', SUBCAT=frame)
        add('is', 'V', VFORM='fin', AGR='3sg', SUBCAT=frame)
        add('are', 'V', VFORM='fin', AGR='non3sg', SUBCAT=frame)
        add('was', 'V', VFORM='fin', AGR='3sg', SUBCAT=frame)
        add('were', 'V', VFORM='fin', AGR='non3sg', SUBCAT=frame)
    return L


HF = ['0.AGR=1.AGR', '0.VFORM=1.VFORM']      # head features of a VP
BASE_RULES = []
```

```
def _r(lhs, rhs, *cons):
    BASE_RULES.append((lhs, tuple(rhs.split()), list(cons)))


# ---- sentence level ----------------------------------------------------
_r('S', 'NP VP', '1.CASE=nom', '1.AGR=2.AGR', '2.VFORM=fin')                  # declarative
_r('S', 'VP', '1.VFORM=base')                                                 # imperative
_r('S', 'SQ')                                                               # yes/no question
_r('S', 'WhNP VP', '1.AGR=2.AGR', '2.VFORM=fin')                              # subject wh-
question
_r('S', 'WhNP SQ/NP')                                                       # object wh-question (gap)
_r('S', 'WhAdv SQ')                                                         # adjunct wh-question
_r('S', 'S Conj S')                                                         # clause coordination
_r('SQ', 'Aux NP VP', '1.AGR=2.AGR', '2.CASE=nom', '3.VFORM=base')            #
auxiliary inversion
_r('SQ', 'V NP AdjP', '1.SUBCAT=ADJ', '1.VFORM=fin', '1.AGR=2.AGR', '2.CASE=nom')
_r('SQ', 'V NP V', '1.SUBCAT=PASS', '1.VFORM=fin', '1.AGR=2.AGR', '2.CASE=nom',
'3.VFORM=pp', '3.SUBCAT=NP')
_r('SQ', 'V NP PP', '1.SUBCAT=PP', '1.VFORM=fin', '1.AGR=2.AGR', '2.CASE=nom',
'1.PFORM=3.PFORM')
_r('SQ/NP', 'Aux NP VP/NP', '1.AGR=2.AGR', '2.CASE=nom', '3.VFORM=base')       # gap
versions
_r('S/NP', 'NP VP/NP', '1.CASE=nom', '1.AGR=2.AGR', '2.VFORM=fin')
_r('VP/NP', 'V', '1.SUBCAT=NP', *HF)
_r('VP/NP', 'V NP', '1.SUBCAT=NP_NP', '2.CASE=acc', *HF)
_r('VP/NP', 'V PP', '1.SUBCAT=NP_PP', '1.PFORM=2.PFORM', *HF)
_r('VP/NP', 'Aux VP/NP', '2.VFORM=base', '0.AGR=1.AGR', '0.VFORM=1.VFORM')
# ---- noun phrases ------------------------------------------------------
_r('NP', 'Det Nom', '1.AGR=2.AGR', '0.AGR=2.AGR')
_r('NP', 'Nom', '1.AGR=non3sg', '0.AGR=1.AGR')
_r('NP', 'Pron', '0.AGR=1.AGR', '0.CASE=1.CASE')
_r('NP', 'PN', '0.AGR=1.AGR')
_r('NP', 'NP PP', '0.AGR=1.AGR', '0.CASE=1.CASE')
_r('NP', 'NP Conj NP', '0.AGR=2.AGR', '0.CASE=1.CASE')
_r('NP', 'NP RelCl', '0.AGR=1.AGR', '0.CASE=1.CASE', '1.AGR=2.AGR')
_r('Nom', 'N', '0.AGR=1.AGR')
_r('Nom', 'N Nom', '1.AGR=3sg', '0.AGR=2.AGR')
_r('Nom', 'Adj Nom', '0.AGR=2.AGR')
_r('Nom', 'Nom Conj Nom', '0.AGR=2.AGR')
_r('WhNP', 'WhPron', '0.AGR=1.AGR')
_r('WhNP', 'WhDet Nom', '0.AGR=2.AGR')
# ---- verb phrases (one rule per subcategorisation frame) ---------------
_r('VP', 'V', '1.SUBCAT=INTR', *HF)
_r('VP', 'V NP', '1.SUBCAT=NP', '2.CASE=acc', *HF)
_r('VP', 'V NP NP', '1.SUBCAT=NP_NP', '2.CASE=acc', '3.CASE=acc', *HF)
_r('VP', 'V CP', '1.SUBCAT=CP', *HF)
_r('VP', 'V NP CP', '1.SUBCAT=NP_CP', '2.CASE=acc', *HF)
_r('VP', 'V PP', '1.SUBCAT=PP', '1.PFORM=2.PFORM', *HF)
```

```
_r('VP', 'V NP PP', '1.SUBCAT=NP_PP', '2.CASE=acc', '1.PFORM=3.PFORM', *HF)
_r('VP', 'V AdjP', '1.SUBCAT=ADJ', *HF)
_r('VP', 'V V', '1.SUBCAT=PASS', '2.VFORM=pp', '2.SUBCAT=NP', *HF)              #
passive
_r('VP', 'Aux VP', '2.VFORM=base', '0.AGR=1.AGR', '0.VFORM=1.VFORM')
_r('VP', 'VP PP', *HF)                                          # adjunct
_r('VP', 'VP SubCl', *HF)
_r('VP', 'VP Conj VP', '0.AGR=1.AGR', '1.AGR=3.AGR', '0.VFORM=1.VFORM',
'1.VFORM=3.VFORM')
# ---- other phrases -----------------------------------------------------------
_r('PP', 'P NP', '0.PFORM=1.PFORM', '2.CASE=acc')
_r('AdjP', 'Adj')
_r('AdjP', 'Adv Adj')
_r('AdjP', 'Adj PP')
_r('AdjP', 'AdjP Conj AdjP')
_r('RelCl', 'Comp VP', '2.VFORM=fin', '0.AGR=2.AGR')
_r('RelCl', 'Comp S/NP')
_r('CP', 'Comp S')
_r('SubCl', 'Sconj S')


class Rule:
    __slots__ = ('id', 'lhs', 'rhs', 'cons_str', 'cons', 'skel')

    def __init__(self, rid, lhs, rhs, cons_str):
        self.id, self.lhs, self.rhs = rid, lhs, tuple(rhs)
        self.cons_str = list(cons_str)
        self.cons = compile_constraints(cons_str)
        self.skel = (lhs, self.rhs)

    def __repr__(self):
        return '%s -> %s' % (self.lhs, ' '.join(self.rhs))


class Grammar:
    def __init__(self, rules, lexicon, start='S', features=True, name=''):
        self.name = name
        self.start = start
        self.features = features
        self.rules = []
        for i, (lhs, rhs, cons) in enumerate(rules):
            self.rules.append(Rule(i, lhs, rhs, cons if features else []))
        self.lexicon = lexicon
        self.lex = defaultdict(list)
        for w, c, f in lexicon:
            self.lex[w].append((c, f if features else {}))
        self.preterms = set(c for _, c, _ in lexicon)
        self.by_lhs = defaultdict(list)
        for r in self.rules:
            self.by_lhs[r.lhs].append(r)
```

```
        self.nonterms = set(r.lhs for r in self.rules)
        # interface features: those mentioned in any constraint for a category
        self.iface = defaultdict(set)
        for r in self.rules:
            cats = (r.lhs,) + r.rhs
            for a, b in r.cons:
                for t in (a, b):
                    if t[0] == 't':
                        self.iface[cats[t[1]]].add(t[2])
        self._left_corners()

    def restrict(self, cat, feats):
        keep = self.iface.get(cat, ())
        return fkey({k: v for k, v in feats.items() if k in keep})

    def _left_corners(self):
        lc = defaultdict(set)
        for p in self.preterms:
            lc[p].add(p)
        changed = True
        while changed:
            changed = False
            for r in self.rules:
                before = len(lc[r.lhs])
                lc[r.lhs] |= lc[r.rhs[0]]
                if len(lc[r.lhs]) != before:
                    changed = True
        self.lc = lc

    def pos_of(self, word):
        return set(c for c, _ in self.lex.get(word, []))

    def n_rules(self):
        return len(self.rules)

    def n_lex_entries(self):
        return len(set((w, c) for w, c, _ in self.lexicon))


def make_GF():
    return Grammar(BASE_RULES, build_lexicon(), features=True, name='G_F')


def make_G_nofeat():
    """The feature grammar with every constraint ignored (agreement and subcategorisation
    switched off): the over-generating skeleton."""
    return Grammar(BASE_RULES, build_lexicon(), features=False, name='G_skeleton')
```

```
#
================================================================
==========
# 3. GRAMMAR TRANSFORMATION: SPLITTING CATEGORIES (plain CFG variants)
#
================================================================
==========
DOMAINS = {'AGR': ['3sg', 'non3sg'], 'CASE': ['nom', 'acc'],
        'VFORM': ['fin', 'base', 'pp'], 'SUBCAT': list(SUBCAT_NAMES.keys()),
        'PFORM': ['in', 'on', 'near', 'for', 'to', 'with', 'at', 'by', 'from', 'after', 'before']}
_SHORT = {'3sg': 'sg', 'non3sg': 'pl'}


def split_name(base, vals):
   """Readable name of a split category, e.g. NP-sg, V-trans, V-trans-pl-fin."""
   parts = []
   if 'SUBCAT' in vals:
      parts.append(SUBCAT_NAMES[vals['SUBCAT']])
      base = ''
   for f in ('AGR', 'CASE', 'VFORM', 'PFORM'):
      if f in vals:
         parts.append(_SHORT.get(vals[f], vals[f]))
   if base and parts:
      return base + '-' + '-'.join(parts)
   if base:
      return base
   return '-'.join(parts) if parts[0].startswith('V') and len(parts) == 1 else parts[0] + ('-' + '-
'.join(parts[1:]) if len(parts) > 1 else '')


def expand_grammar(rules, lexicon, features):
   """Return (rules, lexicon) of the plain CFG obtained by multiplying the categories
   that carry the given features (e.g. NP -> NP-sg, NP-pl)."""
   features = list(features)
   carriers = {f: set() for f in features}
   for lhs, rhs, cons in rules:
      cats = (lhs,) + tuple(rhs)
      for a, b in compile_constraints(cons):
         for t in (a, b):
            if t[0] == 't' and t[2] in features:
               carriers[t[2]].add(cats[t[1]])

   def cat_name(c, assign):
      vals = {f: assign[f] for f in features if c in carriers[f] and f in assign}
      return split_name(c, vals) if vals else c

   new_rules, seen = [], set()
   for lhs, rhs, cons in rules:
      cats = (lhs,) + tuple(rhs)
      cc = compile_constraints(cons)
```

```
        # classes of (position, feature) terms
        terms = [(i, f) for i, c in enumerate(cats) for f in features if c in carriers[f]]
        parent = {t: t for t in terms}

        def find(t):
            while parent[t] != t:
                t = parent[t]
            return t
        consts = {}
        for a, b in cc:
            if a[0] == 't' and b[0] == 't' and a[2] in features and (a[1], a[2]) in parent and (b[1],
b[2]) in parent:
                ra, rb = find((a[1], a[2])), find((b[1], b[2]))
                if ra != rb:
                    parent[ra] = rb
        for a, b in cc:
            if a[0] == 't' and b[0] == 'c' and a[2] in features and (a[1], a[2]) in parent:
                consts[find((a[1], a[2]))] = b[1]
            if b[0] == 't' and a[0] == 'c' and b[2] in features and (b[1], b[2]) in parent:
                consts[find((b[1], b[2]))] = a[1]
        classes = sorted(set(find(t) for t in terms))
        choices = []
        for cl in classes:
            choices.append([consts[cl]] if cl in consts else DOMAINS[cl[1]])
        for combo in itertools.product(*choices):
            cval = dict(zip(classes, combo))
            names = []
            for i, c in enumerate(cats):
                assign = {f: cval[find((i, f))] for f in features if (i, f) in parent}
                names.append(cat_name(c, assign))
            key = (names[0], tuple(names[1:]))
            if key not in seen:
                seen.add(key)
                new_rules.append((names[0], tuple(names[1:]), []))
    carr_pre = {f: set(c for c in carriers[f]) for f in features}
    new_lex, seenl = [], set()
    for w, c, fts in lexicon:
        opts = []
        for f in features:
            if c in carr_pre[f]:
                opts.append([fts[f]] if f in fts else DOMAINS[f])
            else:
                opts.append([None])
        for combo in itertools.product(*opts):
            assign = {f: v for f, v in zip(features, combo) if v is not None}
            nm = cat_name(c, assign)
            if (w, nm) not in seenl:
                seenl.add((w, nm))
                new_lex.append((w, nm, {}))
    return new_rules, new_lex
```

```
#
============================================================================
===========
# 4. TREE UTILITIES
#
============================================================================
===========
class Node:
    __slots__ = ('label', 'kids', 'feats', 'word', 'rid')

    def __init__(self, label, kids=None, feats=None, word=None, rid=None):
        self.label, self.kids = label, kids or []
        self.feats, self.word, self.rid = feats or {}, word, rid

    def is_pre(self):
        return self.word is not None


def bracket(n):
    if n.is_pre():
        return '(%s %s)' % (n.label, n.word)
    return '(%s %s)' % (n.label, ' '.join(bracket(k) for k in n.kids))


def tree_stats(n):
    """height (edges, including the word below each preterminal), nodes (incl. words),
    rule applications (internal nodes + preterminals)."""
    def h(x):
        return 1 if x.is_pre() else 1 + max(h(k) for k in x.kids)

    def cnt(x):
        if x.is_pre():
            return 2, 1                 # preterminal node + word ; one lexical rule
        a = b = 0
        for k in x.kids:
            c1, c2 = cnt(k)
            a, b = a + c1, b + c2
        return a + 1, b + 1
    nodes, rules = cnt(n)
    return {'height': h(n), 'nodes': nodes, 'rules': rules}


def relabel_plain(n):
    """Rename V preterminals to their frame category (V-trans, ...) = tree in plain CFG G."""
    if n.is_pre():
        lab = SUBCAT_NAMES[n.feats['SUBCAT']] if n.label == 'V' and 'SUBCAT' in n.feats
else n.label
        return Node(lab, word=n.word, feats=n.feats)
```

```
    return Node(n.label, [relabel_plain(k) for k in n.kids], n.feats, rid=n.rid)


def tree_rules(n, out=None):
    """List of rules (lhs, rhs-tuple) used, in preorder (leftmost derivation order)."""
    if out is None:
        out = []
    if n.is_pre():
        out.append((n.label, (n.word,)))
        return out
    out.append((n.label, tuple(k.label for k in n.kids)))
    for k in n.kids:
        tree_rules(k, out)
    return out


def derivation(n, rightmost=False):
    """Sentential forms of the leftmost (or rightmost) derivation."""
    forms = [[n.label]]
    # a form is a list of items: Node (unexpanded) or str (terminal)
    cur = [n]
    forms_txt = [[n.label]]
    while True:
        idx = None
        rng = range(len(cur) - 1, -1, -1) if rightmost else range(len(cur))
        for i in rng:
            if isinstance(cur[i], Node):
                idx = i
                break
        if idx is None:
            break
        x = cur[idx]
        rep = [x.word] if x.is_pre() else list(x.kids)
        cur = cur[:idx] + rep + cur[idx + 1:]
        forms_txt.append([c.label if isinstance(c, Node) else c for c in cur])
    return forms_txt


#
============================================================================
===========
# 5. TOP-DOWN PARSER (backtracking recursive descent)
#
============================================================================
===========
class BudgetExceeded(Exception):
    pass


class TopDown:
```

```
"""Goal-driven backtracking parser.

filtered=False : textbook parser (expands every rule of the goal category).
filtered=True  : adds (a) a left-corner filter (the next word must be able to start
                 the rule's first daughter) and (b) a length filter (the pending
                 daughters must fit into the remaining words; the grammar is
                 epsilon-free).  Together they make left-recursive rules safe.
"""

def __init__(self, G, filtered=True, budget=2_000_000, use_features=True):
    self.G, self.filtered, self.budget = G, filtered, budget
    self.use_features = use_features

def parse(self, words, first_only=False):
    from collections import Counter
    self.comp = Counter()
    G, n = self.G, len(words)
    self.steps = 0
    self.lex_tests = 0
    trees = []
    posset = [G.pos_of(w) for w in words]
    t0 = time.perf_counter()
    status = 'ok'

    def tick():
        self.steps += 1
        if self.steps > self.budget:
            raise BudgetExceeded()

    def parse_cat(cat, i, need):
        if cat in G.preterms:
            tick()
            if i < n:
                for c, f in G.lex.get(words[i], []):
                    if c == cat:
                        yield Node(cat, feats=dict(f), word=words[i]), i + 1
            return
        for r in G.by_lhs[cat]:
            tick()
            if self.filtered:
                if i >= n or not (G.lc[r.rhs[0]] & posset[i]):
                    continue
                if n - i < len(r.rhs) + need:
                    continue
            yield from parse_rule(r, i, need)

    def parse_rule(r, i, need):
        m = len(r.rhs)

        def rec(k, pos, kids, cf):
```

```
            if k == m:
                feats = evaluate(r.cons, cf) if self.use_features else {}
                if feats is None:
                    return
                self.comp[(r.lhs, i, pos)] += 1
                yield Node(r.lhs, kids, dict(G.restrict(r.lhs, feats)), rid=r.id), pos
                return
            for child, j in parse_cat(r.rhs[k], pos, need + (m - k - 1)):
                ncf = cf + [dict(G.restrict(child.label, child.feats))]
                if self.use_features and r.cons and \
                        evaluate(r.cons, ncf + [None] * (m - k - 1)) is None:
                    continue
                yield from rec(k + 1, j, kids + [child], ncf)
        yield from rec(0, i, [], [])

    try:
        for node, j in parse_cat(G.start, 0, 0):
            if j == n:
                trees.append(node)
                if first_only:
                    break
    except BudgetExceeded:
        status = 'budget'
    except RecursionError:
        status = 'recursion'
    self.time = time.perf_counter() - t0
    return trees, {'steps': self.steps, 'status': status, 'time': self.time}


#
==========================================================================
===========
# 6. EARLEY PARSER (with back-pointers and unification)
#
==========================================================================
===========
class State:
    __slots__ = ('rid', 'lhs', 'rhs', 'dot', 'origin', 'end', 'cf', 'feats', 'bps', 'op',
                 'idx', 'key', 'lex', 'src')

    def complete(self):
        return self.dot == len(self.rhs)


class Earley:
    def __init__(self, G, use_features=True):
        self.G = G
        self.use_features = use_features

    def parse(self, words, keep_chart=True):
```

```
G, n = self.G, len(words)
chart = [[] for _ in range(n + 1)]
index = [dict() for _ in range(n + 1)]
attempts = [0]
t0 = time.perf_counter()
uf = self.use_features

def add(k, key, lhs, rhs, rid, dot, origin, cf, feats, bp, op, lex=None, src=''):
   attempts[0] += 1
   s = index[k].get(key)
   if s is None:
      s = State()
      s.rid, s.lhs, s.rhs, s.dot, s.origin, s.end = rid, lhs, rhs, dot, origin, k
      s.cf, s.feats, s.bps, s.op, s.key, s.lex = cf, feats, set(), op, key, lex
      s.src = src
      s.idx = len(chart[k])
      chart[k].append(s)
      index[k][key] = s
   if bp is not None:
      s.bps.add(bp)
   return s

# dummy start state  gamma -> . S
g = add(0, ('gamma',), 'gamma', (G.start,), -1, 0, 0, (), {}, None, 'INIT')
for k in range(n + 1):
   i = 0
   while i < len(chart[k]):
      s = chart[k][i]
      i += 1
      if not s.complete():
         sym = s.rhs[s.dot]
         if sym in G.preterms:                    # SCANNER
            if k < n:
               for c, f in G.lex.get(words[k], []):
                  if c == sym:
                     fk = G.restrict(c, f) if uf else ()
                     key = (('L', c, words[k]), 1, k, fk)
                     add(k + 1, key, c, (words[k],), -2, 1, k, fk, dict(f), None,
                        'SCANNER', lex=words[k], src='S[%d].%d' % (k, s.idx))
         else:                                    # PREDICTOR
            for r in G.by_lhs[sym]:
               add(k, (r.id, 0, k, ()), r.lhs, r.rhs, r.id, 0, k, (), {}, None,
                  'PREDICTOR', src='S[%d].%d' % (k, s.idx))
      else:                                       # COMPLETER
         if s.rid == -1:
            continue
         o = s.origin
         childfeats = dict(s.feats)
         lhs = s.lhs
         j = 0
```

```
                while j < len(chart[o]):
                    p = chart[o][j]
                    j += 1
                    if p.complete() or p.rhs[p.dot] != lhs:
                        continue
                    r = G.rules[p.rid]
                    rf = G.restrict(lhs, childfeats) if uf else ()
                    ncf = p.cf + (rf,)
                    feats = {}
                    if uf and r.cons:
                        m = len(r.rhs)
                        kids = [dict(x) for x in ncf] + [None] * (m - len(ncf))
                        feats = evaluate(r.cons, kids)
                        if feats is None:
                            continue
                    nd = p.dot + 1
                    key = (r.id, nd, p.origin, ncf)
                    lf = dict(G.restrict(r.lhs, feats)) if (uf and nd == len(r.rhs)) else {}
                    add(k, key, r.lhs, r.rhs, r.id, nd, p.origin, ncf, lf,
                        (p if p.dot > 0 else None, s), 'COMPLETER',
                        src='S[%d].%d + S[%d].%d' % (o, p.idx, k, s.idx))
        self.time = time.perf_counter() - t0
        self.chart, self.index, self.n = chart, index, n
        self.words = words
        roots = [s for s in chart[n] if s.complete() and s.lhs == G.start and s.origin == 0
                 and s.rid >= 0]
        n_states = sum(len(c) for c in chart)
        stats = {'states': n_states, 'attempts': attempts[0], 'time': self.time,
                 'chart_sizes': [len(c) for c in chart]}
        self.roots = roots
        return roots, stats

    # -- forest utilities ----------------------------------------------------
    def count(self, root):
        """Number of trees packed under a state (dynamic programming over back-pointers)."""
        memo = {}

        def c(s):
            if id(s) in memo:
                return memo[id(s)]
            if s.lex is not None:
                v = 1
            else:
                v = 0
                for prev, child in s.bps:
                    v += (c(prev) if prev is not None else 1) * c(child)
            memo[id(s)] = v
            return v
        return c(root)
```

```
    def trees(self, root=None, limit=1000):
        """Enumerate parse trees from the packed forest."""
        def nodes(s):
            if s.lex is not None:
                yield Node(s.lhs, feats=dict(s.feats), word=s.lex)
                return
            for kl in kidlists(s):
                yield Node(s.lhs, kl, feats=dict(s.feats), rid=s.rid)

        def kidlists(s):
            for prev, child in sorted(s.bps, key=lambda b: (b[0].idx if b[0] else -1, b[1].idx)):
                lefts = [[]] if prev is None else kidlists(prev)
                for l in lefts:
                    for c in nodes(child):
                        yield l + [c]

        out = []
        for r in (roots_sorted(self.roots) if root is None else [root]):
            for t in nodes(r):
                out.append(t)
                if len(out) >= limit:
                    return out
        return out

    def fragments(self):
        """All completed constituents (category, i, j) in the chart (for fall-back parsing)."""
        out = set()
        for k, col in enumerate(self.chart):
            for s in col:
                if s.complete() and s.rid != -1:
                    out.add((s.lhs, s.origin, k))
        return out

    def longest_prefix(self):
        """Largest k such that words[:k] is a viable prefix (chart column k non-empty)."""
        best = 0
        for k in range(self.n + 1):
            if self.chart[k]:
                best = k
        return best


def roots_sorted(roots):
    return sorted(roots, key=lambda s: s.idx)


#
==========================================================================
===========
# 7. PCFG: maximum-likelihood estimation with add-alpha smoothing
```

```
#
============================================================================
===========
class PCFG:
    def __init__(self, G):
        self.G = G
        self.rule_count = defaultdict(float)
        self.lex_count = defaultdict(float)
        self.prob = {}

    def count_tree(self, t):
        if t.is_pre():
            self.lex_count[(t.label, t.word)] += 1
            return
        self.rule_count[t.rid] += 1
        for k in t.kids:
            self.count_tree(k)

    def estimate(self, trees, alpha=0.1):
        self.alpha = alpha
        self.rule_count.clear()
        self.lex_count.clear()
        for t in trees:
            self.count_tree(t)
        G = self.G
        lhs_total = defaultdict(float)
        for rid, c in self.rule_count.items():
            lhs_total[G.rules[rid].lhs] += c
        pre_total = defaultdict(float)
        for (cat, w), c in self.lex_count.items():
            pre_total[cat] += c
        pre_words = defaultdict(set)
        for w, c, _ in G.lexicon:
            pre_words[c].add(w)
        self.prob = {}
        self.denom = {}
        for lhs, rs in G.by_lhs.items():
            K = len(rs)
            tot = lhs_total[lhs] + alpha * K
            self.denom[lhs] = (lhs_total[lhs], K)
            for r in rs:
                self.prob[r.id] = (self.rule_count[r.id] + alpha) / tot
        for cat, ws in pre_words.items():
            K = len(ws)
            tot = pre_total[cat] + alpha * K
            self.denom[cat] = (pre_total[cat], K)
            for w in ws:
                self.prob[(cat, w)] = (self.lex_count[(cat, w)] + alpha) / tot
        return self.prob
```

```
    def tree_prob(self, t):
        if t.is_pre():
            return self.prob[(t.label, t.word)]
        p = self.prob[t.rid]
        for k in t.kids:
            p *= self.tree_prob(k)
        return p

    def check_sums(self):
        sums = defaultdict(float)
        for r in self.G.rules:
            sums[r.lhs] += self.prob[r.id]
        for cat in self.G.preterms:
            sums[cat] += sum(p for k, p in self.prob.items() if isinstance(k, tuple) and k[0] == cat)
        return sums


#
============================================================================
===========
# 8. PROBABILISTIC CKY (binarised grammar with unary closure)
#
============================================================================
===========
class Item:
    __slots__ = ('best', 'inside', 'count', 'bp')

    def __init__(self):
        self.best, self.inside, self.count, self.bp = 0.0, 0.0, 0, None


class PCKY:
    """Viterbi / inside CKY for the binarised, unit-closed form of the PCFG.
    Features are carried in the chart items (item key = (symbol, interface features)),
    so that agreement and subcategorisation are checked during parsing."""

    def __init__(self, G, pcfg, use_features=True):
        self.G, self.pcfg, self.uf = G, pcfg, use_features
        self.unary = defaultdict(list)
        self.pair = defaultdict(list)
        for r in G.rules:
            m = len(r.rhs)
            if m == 1:
                self.unary[r.rhs[0]].append(r)
                continue

            def sym(j):          # symbol standing for daughters j..m (1-based, j>=2)
                return r.rhs[m - 1] if j == m else ('P', r.id, j)
            self.pair[(r.rhs[0], sym(2))].append((r.lhs, 'top', r.id, 1))
            for j in range(2, m):
```

```
            self.pair[(r.rhs[j - 1], sym(j + 1))].append((('P', r.id, j), 'mid', r.id, j))
        # topological order of categories w.r.t. unary rules (children first)
        order, seen = [], set()

        def visit(c):
            if c in seen:
                return
            seen.add(c)
            for rr in G.rules:
                if len(rr.rhs) == 1 and rr.lhs == c:
                    visit(rr.rhs[0])
            order.append(c)
        for c in list(G.nonterms) + list(G.preterms):
            visit(c)
        self.topo = order

    def parse(self, words):
        G, P, n = self.G, self.pcfg.prob, len(words)
        uf = self.uf
        cells = {}
        combos = 0
        t0 = time.perf_counter()

        def put(cell, key, prob, cnt, bp):
            it = cell.get(key)
            if it is None:
                it = cell[key] = Item()
            it.inside += prob
            it.count += cnt
            if prob > it.best:
                it.best, it.bp = prob, bp

        for i in range(n):
            cell = {}
            for c, f in G.lex.get(words[i], []):
                fk = G.restrict(c, f) if uf else ()
                put(cell, (c, fk), P[(c, words[i])], 1, ('lex', words[i], dict(f)))
            self._unary(cell, i, i + 1, put)
            cells[(i, i + 1)] = cell
        for span in range(2, n + 1):
            for i in range(0, n - span + 1):
                j = i + span
                cell = {}
                for k in range(i + 1, j):
                    left, right = cells[(i, k)], cells[(k, j)]
                    if not left or not right:
                        continue
                    for (y, fy), iy in left.items():
                        for (z, fz), iz in right.items():
                            for target, kind, rid, jp in self.pair.get((y, z), ()):
```

```
                        combos += 1
                        r = G.rules[rid]
                        m = len(r.rhs)
                        suffix = list(fz) if isinstance(z, tuple) else [fz]
                        if kind == 'top':
                            if uf and r.cons:
                                kids = [dict(fy)] + [dict(x) for x in suffix]
                                feats = evaluate(r.cons, kids)
                                if feats is None:
                                    continue
                                fk = G.restrict(r.lhs, feats)
                            else:
                                fk = ()
                            prob = iy.best * iz.best * P[rid]
                            ins = iy.inside * iz.inside * P[rid]
                            key = (target, fk)
                        else:
                            sufkeys = [fy] + suffix
                            if uf and r.cons:
                                kids = [None] * (jp - 1) + [dict(x) for x in sufkeys]
                                kids += [None] * (m - len(kids))
                                if evaluate(r.cons, kids) is None:
                                    continue
                            prob = iy.best * iz.best
                            ins = iy.inside * iz.inside
                            key = (target, tuple(sufkeys))
                        bp = ('bin', (i, k, (y, fy)), (k, j, (z, fz)), rid, kind)
                        itm = cell.get(key)
                        if itm is None:
                            itm = cell[key] = Item()
                        itm.inside += ins
                        itm.count += iy.count * iz.count
                        if prob > itm.best:
                            itm.best, itm.bp = prob, bp
            self._unary(cell, i, j, put)
            cells[(i, j)] = cell
    self.cells, self.n, self.words = cells, n, words
    top = [(key, it) for key, it in cells[(0, n)].items() if key[0] == G.start]
    self.time = time.perf_counter() - t0
    n_items = sum(len(c) for c in cells.values())
    stats = {'items': n_items, 'combos': combos, 'time': self.time}
    if top:
        best_key, best_it = max(top, key=lambda x: x[1].best)
        stats.update({'viterbi': best_it.best, 'inside': sum(it.inside for _, it in top),
                      'count': sum(it.count for _, it in top)})
        self.best_key = best_key
        tree = self.build(0, n, best_key)
        return tree, stats
    stats.update({'viterbi': 0.0, 'inside': 0.0, 'count': 0})
    return None, stats
```

```
def _unary(self, cell, i, j, put):
    G, P = self.G, self.pcfg.prob
    for cat in self.topo:
        srcs = [(key, it) for key, it in list(cell.items()) if key[0] == cat]
        if not srcs:
            continue
        for r in self.unary.get(cat, ()):
            for (c, fk), it in srcs:
                if self.uf and r.cons:
                    feats = evaluate(r.cons, [dict(fk)])
                    if feats is None:
                        continue
                    nfk = G.restrict(r.lhs, feats)
                else:
                    nfk = ()
                tgt = (r.lhs, nfk)
                nit = cell.get(tgt)
                if nit is None:
                    nit = cell[tgt] = Item()
                nit.inside += it.inside * P[r.id]
                nit.count += it.count
                if it.best * P[r.id] > nit.best:
                    nit.best = it.best * P[r.id]
                    nit.bp = ('unary', (i, j, (c, fk)), r.id)

def build(self, i, j, key):
    it = self.cells[(i, j)][key]
    sym = key[0]
    bp = it.bp
    if bp[0] == 'lex':
        return Node(sym, feats=bp[2], word=bp[1])
    if bp[0] == 'unary':
        _, ck, rid = bp
        return Node(sym, [self.build(*ck)], dict(key[1]) if False else {}, rid=rid)
    _, lk, rk, rid, kind = bp
    left = self.build(*lk)
    right = self._suffix(*rk)
    return Node(sym, [left] + right, {}, rid=rid)

def _suffix(self, i, j, key):
    sym = key[0]
    if not (isinstance(sym, tuple) and sym[0] == 'P'):
        return [self.build(i, j, key)]
    it = self.cells[(i, j)][key]
    _, lk, rk, rid, kind = it.bp
    return [self.build(*lk)] + self._suffix(*rk)
```

```
#
================================================================================
==========
# helpers
#
================================================================================
==========
def tokenize(s):
    """Lower-case, strip punctuation, split on blanks (the lexicon is lower-case)."""
    return s.lower().replace(',', ' ').replace('?', ' ').replace('.', ' ').split()
```

A.2 trace_tools.py - hand-trace generator for G_mini

```
import helpdesk_parsers as H
MINI_RULES=[('S','NP VP'),('NP','Det N'),('NP','NP PP'),('NP','PN'),('VP','V NP'),('VP','VP
PP'),('PP','P NP')]
MINI_LEX=[('the','Det'),('student','N'),('form','N'),('notice','N'),('submitted','V'),('on','P'),('with
','P'),('friday','PN')]
def mini_grammar(rules=None):
    rules=rules or MINI_RULES
    return H.Grammar([(a,tuple(b.split()),[]) for a,b in rules],[(w,c,{}) for w,c in
MINI_LEX],features=False,name='G_mini')
def td_trace(rules,words,maxsteps=500,find_all=False,prune=False):
    G=mini_grammar(rules)
    byl={}
    for a,b in rules: byl.setdefault(a,[]).append(b.split())
    pos={w:set() for w,_ in MINI_LEX}
    for w,c in MINI_LEX: pos[w].add(c)
    n=len(words); stack=['S']; i=0; choices=[]; log=[]; parses=0
    def L(act): log.append((len(log)+1,' '.join(stack) if stack else '(empty)',' '.join(words[i:]) if
i<n else '(empty)',act))
    failed=False
    while len(log)<maxsteps:
        if not stack and i==n:
            L('SUCCESS: parse found'); parses+=1
            if not find_all: break
            failed=True
        elif not stack:
            L('FAIL: input left over'); failed=True
        else:
            top=stack[0]
            if top in byl:
                rhs=byl[top][0]; choices.append((list(stack),i,top,1))
                ns=rhs+stack[1:]
                if prune and len(ns)>n-i:
                    L('expand %s -> %s, PRUNE: %d symbols pending > %d words left'%(top,'
'.join(rhs),len(ns),n-i)); failed=True
                else:
                    L('expand %s -> %s'%(top,' '.join(rhs))); stack=ns
            else:
                if i<n and top in pos.get(words[i],()):
```

```
                L("match %s = '%s'"%(top,words[i])); stack=stack[1:]; i+=1
            else:
                L("FAIL: %s cannot match '%s'"%(top,words[i] if i<n else 'end of input'));
failed=True
        if failed:
            failed=False; done=True
            while choices:
                st,ii,nt,a=choices.pop()
                if a<len(byl[nt]):
                    rhs=byl[nt][a]; stack=list(st); i=ii
                    ns=rhs+stack[1:]
                    choices.append((list(st),ii,nt,a+1))
                    if prune and len(ns)>n-i:
                        L('BACKTRACK to %s: try %s -> %s, PRUNE: %d symbols pending > %d
words left'%(nt,nt,' '.join(rhs),len(ns),n-i))
                        continue
                    L('BACKTRACK to %s: try %s -> %s'%(nt,nt,' '.join(rhs))); stack=ns;
done=False; break
            if done:
                L('no alternatives left: search ends'); break
    return log,parses
if __name__=='__main__':
    w="the student submitted the form".split()
    log,p=td_trace(MINI_RULES,w); print(len(log),p)
    for l in log[:12]: print(l)
    log,p=td_trace(MINI_RULES,"the student submitted the form on
friday".split(),find_all=True,maxsteps=2000); print(len(log),p)
    rr=[('S','NP VP'),('NP','NP PP'),('NP','Det N'),('NP','PN'),('VP','V NP'),('VP','VP
PP'),('PP','P NP')]
    log,p=td_trace(rr,w,maxsteps=10)
    for l in log: print(l)
```

A.3 data_sets.py - query set, treebank and gold-tree predicates

```
"""Query set, treebank sentences and gold-attachment predicates (hand-written)."""
QUERIES = [
 ("Q1","D","The student submitted the form."),
 ("Q2","D","The portal crashed."),
 ("Q3","D","The exams start on Friday."),
 ("Q4","D","The students paid the fee."),
 ("Q5","D","The office sent me the receipt."),
 ("Q6","D","The notice says that the fee is due."),
 ("Q7","D","The fee is due on Friday."),
 ("Q8","D","They submitted the forms and the fees."),
 ("Q9","D","The exams that were postponed will be held on Monday."),
 ("Q10","I","Submit the form."),
 ("Q11","I","Send me the receipt."),
 ("Q12","I","Apply for a hostel room."),
 ("Q13","I","Tell me that the fee is paid."),
 ("Q14","I","Check the timetable and renew the books."),
 ("Q15","I","Submit the form if the fee is paid."),
```

```
 ("Q16","I","Show the students with the late fee notice."),
 ("Q17","I","Book the room near the library for the seminar."),
 ("Q18","I","Send the form to the office in the library."),
 ("Q19","I","Check the fees and hostel charges."),
 ("Q20","I","Collect the receipt for the fee on Friday."),
 ("Q21","Y","Did the student submit the form?"),
 ("Q22","Y","Can you send me the receipt?"),
 ("Q23","Y","Is the fee due?"),
 ("Q24","Y","Were the exams postponed?"),
 ("Q25","Y","Will the results be published on Friday?"),
 ("Q26","W","Who submitted the form?"),
 ("Q27","W","Which students paid the fee?"),
 ("Q28","W","What did you submit?"),
 ("Q29","W","When does the library close?"),
 ("Q30","W","Which form should I submit?"),
]
# treebank: (id, sentence, attachment preference for ambiguous ones)
# 'vp' = prefer VP-attachment, 'np' = prefer NP-attachment, 'comp' = prefer frame
(complement) analysis
TREEBANK = [
 ("T1","The student submitted the form on Friday.","vp"),
 ("T2","Submit the form to the office.","comp"),
 ("T3","The office sent the student the receipt.",None),
 ("T4","The library closes.",None),
 ("T5","Check the timetable.",None),
 ("T6","The hostel fee is due.",None),
 ("T7","Pay the fee in the office.","vp"),
 ("T8","The students registered for the seminar.",None),
 ("T9","Did the office publish the results?",None),
 ("T10","Who collected the certificates?",None),
 ("T11","When will the exams start?",None),
 ("T12","What did the student send?",None),
 ("T13","The results were published on Monday.","vp"),
 ("T14","Show me the notice with the late fee.","np"),
 ("T15","The students that paid the fee collected the receipts.",None),
 ("T16","Is the portal available?",None),
 ("T17","The staff checked the hostel charges and the fees.",None),
 ("T18","Tell me that the exam is postponed.",None),
 ("T19","The student in the library booked the room.","np"),
 ("T20","Collect the receipt for the fee.","np"),
]
# gold-tree predicates on the bracketed string for ambiguous test queries
GOLD = {
 "Q7":  lambda b: "(AdjP (Adj due) (PP" in b,
 "Q9":  lambda b: "(VP (Aux will) (VP (VP (V be) (V held)) (PP (P on)" in b,
 "Q14": lambda b: b.startswith("(S (VP (VP"),
 "Q16": lambda b: "(NP (NP (Det the) (Nom (N students))) (PP (P with)" in b,
 "Q17": lambda b: "(VP (VP (V book) (NP (NP (Det the) (Nom (N room))) (PP (P near) (NP
(Det the) (Nom (N library)))))) (PP (P for)" in b,
```

```
 "Q18": lambda b: "(VP (V send) (NP (Det the) (Nom (N form))) (PP (P to) (NP (NP (Det
the) (Nom (N office))) (PP (P in)" in b,
 "Q19": lambda b: "(NP (NP (Det the) (Nom (N fees))) (Conj and) (NP" in b,
 "Q20": lambda b: "(VP (VP (V collect) (NP (NP (Det the) (Nom (N receipt))) (PP (P for)
(NP (Det the) (Nom (N fee)))))) (PP (P on)" in b,
 }
def treebank_pref_ok(pref, b):
    vpa = "(VP (VP" in b
    npa = "(NP (NP" in b
    if pref == 'vp':
        return vpa and not npa
    if pref == 'np':
        return npa and not vpa
    if pref == 'comp':
        return not vpa and not npa
    return True
```

A.4 exp_c.py - test driver for Task 10

```
import helpdesk_parsers as H, data_sets as D, json, math, time, collections, statistics
import matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
plt.rcParams['font.family']='Liberation Serif'
R={}
G=H.make_GF()
gold=[]
E=H.Earley(G)
for tid,s,pref in D.TREEBANK:
    w=H.tokenize(s); e=H.Earley(G); roots,st=e.parse(w); ts=e.trees(limit=500)
    sel=[t for t in ts if D.treebank_pref_ok(pref,H.bracket(t))] if pref else ts
    gold.append(sel[0])
P=H.PCFG(G); P.estimate(gold,alpha=0.1)
def run_all(s,tdbudget=3_000_000,naive=True):
    w=H.tokenize(s); out={'s':s,'n':len(w)}
    if naive:
        tn=H.TopDown(G,filtered=False,budget=200_000); tr,st=tn.parse(w,first_only=True)
        out['naive_first']=(len(tr)>0,st['status'],st['steps'])
        tn=H.TopDown(G,filtered=False,budget=200_000); tr,st=tn.parse(w)
        out['naive_all']=(len(tr),st['status'],st['steps'],st['time'])
    td=H.TopDown(G,filtered=True,budget=tdbudget); tr,st=td.parse(w)
    out['td']=(len(tr),st['status'],st['steps'],st['time'])
    e=H.Earley(G); roots,est=e.parse(w); out['earley']=(sum(e.count(r) for r in
roots),est['states'],est['attempts'],est['time'])
    ck=H.PCKY(G,P); bt,cst=ck.parse(w);
out['cky']=(cst['count'],cst['items'],cst['combos'],cst['time'],cst['viterbi'],cst['inside'])
    return out
rows=[]
for qid,typ,s in D.QUERIES:
    r=run_all(s); r['id']=qid; r['type']=typ; rows.append(r)
    print(qid,r['n'],r['naive_first'],r['naive_all'][:3],r['td'][:3],r['earley'][:2],r['cky'][:2],flush=Tru
e)
R['rows']=rows
```

```
# ungrammatical and special tests
ungr=["The students submits the form.","The student submit the form.","These student
submitted the form.","A students paid the fees.","Him submitted the form.","Send I the
receipt.","Submit.","The portal crashed the form.","Apply in a hostel room."]
R['ungr']=[]
for s in ungr:
    r=run_all(s); R['ungr'].append(r); print(s,r['td'][:3],r['earley'][:2],r['cky'][:2],flush=True)
# left recursion cases
lr=["The form in the office near the library crashed.","Submit the form in the office near the
library."]
R['lr']=[]
for s in lr:
    r=run_all(s); R['lr'].append(r);
print(s,r['naive_first'],r['naive_all'][:3],r['td'][:3],r['earley'][:2],flush=True)
# long sentences family
preps=['near','in','at','on','for']; nouns=['library','hostel','office','portal','seminar']
fam=[]
for k in range(0,6):
    s="Book the room"+".join(' %s the %s'%(preps[i%5],nouns[i%5]) for i in range(k))+'.'
    r=run_all(s,tdbudget=6_000_000,naive=False); r['k']=k; fam.append(r)
    print('fam',k,r['n'],r['td'][:3],r['earley'][:2],r['cky'][:2],flush=True)
R['fam']=fam
json.dump(R,open('res_c.json','w'),indent=1,default=str)
# plot
ns=[r['n'] for r in fam]
fig,ax=plt.subplots(1,2,figsize=(6.5,3.0))
ax[0].plot(ns,[r['earley'][1] for r in fam],'k-o',label='Earley states',markersize=4)
ax[0].plot(ns,[r['cky'][1] for r in fam],'k--s',label='Viterbi-CKY
items',markersize=4,markerfacecolor='white')
ax[0].plot(ns,[r['td'][2] for r in fam],'k:^',label='Top-down steps',markersize=4)
ax[0].set_yscale('log'); ax[0].set_xlabel('Sentence length (words)',fontsize=11);
ax[0].set_ylabel('Count (log scale)',fontsize=11); ax[0].legend(fontsize=8);
ax[0].grid(True,color='0.8',lw=0.5)
ax[1].plot(ns,[r['earley'][0] for r in fam],'k-o',markersize=4); ax[1].set_yscale('log');
ax[1].set_xlabel('Sentence length (words)',fontsize=11); ax[1].set_ylabel('Number of parses
(log scale)',fontsize=11); ax[1].grid(True,color='0.8',lw=0.5)
for a in ax:
    for t in a.get_xticklabels()+a.get_yticklabels(): t.set_fontsize(9)
plt.tight_layout(); plt.savefig('figs/t10_plot.png',dpi=200); plt.close()
fig,ax=plt.subplots(figsize=(4.2,3.0))
ax.plot(ns,[r['earley'][3]*1000 for r in fam],'k-o',label='Earley',markersize=4)
ax.plot(ns,[r['cky'][3]*1000 for r in fam],'k--s',label='Viterbi-
CKY',markersize=4,markerfacecolor='white')
ax.plot(ns,[r['td'][3]*1000 for r in fam],'k:^',label='Top-down',markersize=4)
ax.set_yscale('log'); ax.set_xlabel('Sentence length (words)',fontsize=11); ax.set_ylabel('Run-
time (ms, log scale)',fontsize=11); ax.legend(fontsize=8); ax.grid(True,color='0.8',lw=0.5)
plt.tight_layout(); plt.savefig('figs/t10_time.png',dpi=200); plt.close()
```

Appendix B - Query Set, Treebank and Effort Log

B.1 The 30-sentence query set

Table B.1 - Query set with sentence type and number of parses

Id	Type	Sentence	Words	Parses
Q1	declarative	The student submitted the form.	5	1
Q2	declarative	The portal crashed.	3	1
Q3	declarative	The exams start on Friday.	5	1
Q4	declarative	The students paid the fee.	5	1
Q5	declarative	The office sent me the receipt.	6	1
Q6	declarative	The notice says that the fee is due.	8	1
Q7	declarative	The fee is due on Friday.	6	2
Q8	declarative	They submitted the forms and the fees.	7	1
Q9	declarative	The exams that were postponed will be held on Monday.	10	2
Q10	imperative	Submit the form.	3	1
Q11	imperative	Send me the receipt.	4	1
Q12	imperative	Apply for a hostel room.	5	1
Q13	imperative	Tell me that the fee is paid.	7	1
Q14	imperative	Check the timetable and renew the books.	7	2
Q15	imperative	Submit the form if the fee is paid.	8	1
Q16	imperative	Show the students with the late fee notice.	8	2
Q17	imperative	Book the room near the library for the seminar.	9	5
Q18	imperative	Send the form to the office in the library.	9	7
Q19	imperative	Check the fees and hostel charges.	6	2
Q20	imperative	Collect the receipt for the fee on Friday.	8	5
Q21	yes/no question	Did the student submit the form?	6	1
Q22	yes/no question	Can you send me the receipt?	6	1
Q23	yes/no question	Is the fee due?	4	1
Q24	yes/no question	Were the exams postponed?	4	1
Q25	yes/no question	Will the results be published on Friday?	7	1
Q26	wh-question	Who submitted the form?	4	1
Q27	wh-question	Which students paid the fee?	5	1
Q28	wh-question	What did you submit?	4	1
Q29	wh-question	When does the library close?	5	1
Q30	wh-question	Which form should I submit?	5	1

The set has 10 declaratives (Q1-Q9 and one coordinated imperative group), 11 imperatives, 5 yes/no questions and 5 wh-questions (the type column gives the exact assignment: 9 declaratives, 11 imperatives, 5 yes/no questions, 5 wh-questions). 8 queries are structurally ambiguous (two or more parses): Q7 (2), Q9 (2), Q14 (2), Q16 (2), Q17 (5), Q18 (7), Q19 (2), Q20 (5). The brief asks for at least five. Gold analyses of the eight ambiguous queries used for evaluation in Tasks 9 and 11 were fixed before the experiments: Q7 adjective complement (due + PP); Q9 PP attached to be held; Q14 coordination of the verb phrases; Q16 PP attached to the students; Q17 PP near the library attached to the room and for the seminar attached to the verb phrase; Q18 PP to the office as complement of send with in the library attached to the office; Q19 coordination of the noun phrases [the fees] and [hostel charges]; Q20 for the fee attached to the receipt and on Friday attached to the verb phrase.

B.2 The mini-treebank (20 annotated sentences)

Each tree was selected from the candidates of the Earley parser. The bracketing uses the categories of G_F (single V; the frame is a feature).

T1. The student submitted the form on Friday.

(S (NP (Det the) (Nom (N student))) (VP (VP (V submitted) (NP (Det the) (Nom (N form)))) (PP (P on) (NP (PN friday)))))

T2. Submit the form to the office.

(S (VP (V submit) (NP (Det the) (Nom (N form))) (PP (P to) (NP (Det the) (Nom (N office))))))

T3. The office sent the student the receipt.

(S (NP (Det the) (Nom (N office))) (VP (V sent) (NP (Det the) (Nom (N student))) (NP (Det the) (Nom (N receipt)))))

T4. The library closes.

(S (NP (Det the) (Nom (N library))) (VP (V closes)))

T5. Check the timetable.

(S (VP (V check) (NP (Det the) (Nom (N timetable)))))

T6. The hostel fee is due.

(S (NP (Det the) (Nom (N hostel) (Nom (N fee)))) (VP (V is) (AdjP (Adj due))))

T7. Pay the fee in the office.

(S (VP (VP (V pay) (NP (Det the) (Nom (N fee)))) (PP (P in) (NP (Det the) (Nom (N office))))))

T8. The students registered for the seminar.

(S (NP (Det the) (Nom (N students))) (VP (V registered) (PP (P for) (NP (Det the) (Nom (N seminar))))))

T9. Did the office publish the results?

(S (SQ (Aux did) (NP (Det the) (Nom (N office))) (VP (V publish) (NP (Det the) (Nom (N results))))))

T10. Who collected the certificates?

(S (WhNP (WhPron who)) (VP (V collected) (NP (Det the) (Nom (N certificates)))))

T11. When will the exams start?

(S (WhAdv when) (SQ (Aux will) (NP (Det the) (Nom (N exams))) (VP (V start))))

T12. What did the student send?

(S (WhNP (WhPron what)) (SQ/NP (Aux did) (NP (Det the) (Nom (N student))) (VP/NP (V send))))

T13. The results were published on Monday.

(S (NP (Det the) (Nom (N results))) (VP (VP (V were) (V published)) (PP (P on) (NP (PN monday)))))

T14. Show me the notice with the late fee.

(S (VP (V show) (NP (Pron me)) (NP (NP (Det the) (Nom (N notice))) (PP (P with) (NP (Det the) (Nom (Adj late) (Nom (N fee))))))))

T15. The students that paid the fee collected the receipts.

(S (NP (NP (Det the) (Nom (N students))) (RelCl (Comp that) (VP (V paid) (NP (Det the) (Nom (N fee)))))) (VP (V collected) (NP (Det the) (Nom (N receipts)))))

T16. Is the portal available?

(S (SQ (V is) (NP (Det the) (Nom (N portal))) (AdjP (Adj available))))

T17. The staff checked the hostel charges and the fees.

(S (NP (Det the) (Nom (N staff))) (VP (V checked) (NP (NP (Det the) (Nom (N hostel) (Nom (N charges)))) (Conj and) (NP (Det the) (Nom (N fees)))))))

T18. Tell me that the exam is postponed.

(S (VP (V tell) (NP (Pron me)) (CP (Comp that) (S (NP (Det the) (Nom (N exam))) (VP (V is) (V postponed)))))))

T19. The student in the library booked the room.

(S (NP (NP (Det the) (Nom (N student))) (PP (P in) (NP (Det the) (Nom (N library))))) (VP (V booked) (NP (Det the) (Nom (N room))))))

T20. Collect the receipt for the fee.

(S (VP (V collect) (NP (NP (Det the) (Nom (N receipt))) (PP (P for) (NP (Det the) (Nom (N fee))))))))

B.3 Additional trees for Task 12

Table B.3 - Ten sentences annotated for the extended grammar

Id	Sentence	Selected tree
X1	Can you tell me when the exams will be held?	(S (SQ (Aux can) (NP (Pron you)) (VP (V tell) (NP (Pron me)) (CP (WhAdv when) (S (NP (Det the) (Nom (N exams))) (VP (Aux will) (VP (V be) (V held))))))))
X2	Tell me whether the fee is due.	(S (VP (V tell) (NP (Pron me)) (CP (Whether whether) (S (NP (Det the) (Nom (N fee))) (VP (V is) (AdjP (Adj due)))))))
X3	Do you know who collected the certificates?	(S (SQ (Aux do) (NP (Pron you)) (VP (V know) (CP (WhNP (WhPron who)) (VP (V collected) (NP (Det the) (Nom (N certificates))))))))
X4	Please send me the receipt.	(S (Intj please) (S (VP (V send) (NP (Pron me)) (NP (Det the) (Nom (N receipt))))))
X5	Hi can you tell me where I can find the book?	(S (Intj hi) (S (SQ (Aux can) (NP (Pron you)) (VP (V tell) (NP (Pron me)) (CP (WhAdv where) (S (NP (Pron i)) (VP (Aux can) (VP (V find) (NP (Det the) (Nom (N book))))))))))
X6	Tell me what you submitted.	(S (VP (V tell) (NP (Pron me)) (CP (WhNP (WhPron what)) (S/NP (NP (Pron you)) (VP/NP (V submitted))))))
X7	Hello the portal crashed.	(S (Intj hello) (S (NP (Det the) (Nom (N portal))) (VP (V crashed))))
X8	Tell me when the library closes and whether the portal is open.	(S (VP (V tell) (NP (Pron me)) (CP (CP (WhAdv when) (S (NP (Det the) (Nom (N library))) (VP (V closes)))) (Conj and) (CP (Whether whether) (S (NP (Det the) (Nom (N portal))) (VP (V is) (AdjP (Adj open))))))))
X9	Do you know if the results were published?	(S (SQ (Aux do) (NP (Pron you)) (VP (V know) (CP (Whether if) (S (NP (Det the) (Nom (N results))) (VP (V were) (V published)))))))
X10	Thanks the office sent me the receipt.	(S (Intj thanks) (S (NP (Det the) (Nom (N office))) (VP (V sent) (NP (Pron me)) (NP (Det the) (Nom (N receipt))))))

B.4 Time log

The effort log with unit times and costs is Table 11.3 (Tasks 1 to 11, about 50 hours) and the Task 12 effort is itemised in Part C of Task 12 (about 13.5 hours). The annotation work was supported by the parser: the annotator saw the candidate trees and selected one, which took about 8 minutes per sentence including the check of ambiguous sentences; the rule writing, the lexicon, the query set and the tests took the remaining time.

References

Chomsky, N. (1957). Syntactic Structures. The Hague: Mouton.

Earley, J. (1970). An efficient context-free parsing algorithm. Communications of the ACM, 13(2), 94-102.

Gazdar, G., Klein, E., Pullum, G. K. and Sag, I. A. (1985). Generalized Phrase Structure Grammar. Oxford: Blackwell.

Jurafsky, D. and Martin, J. H. Speech and Language Processing (3rd edition draft), chapters on constituency grammars, constituency parsing and statistical constituency parsing. Stanford University.

Kasami, T. (1965). An efficient recognition and syntax-analysis algorithm for context-free languages. Technical report, Air Force Cambridge Research Laboratory.

Manning, C. D. and Schutze, H. (1999). Foundations of Statistical Natural Language Processing. Cambridge, MA: MIT Press.

Shieber, S. M. (1985). Evidence against the context-freeness of natural language. Linguistics and Philosophy, 8, 333-343.

Shieber, S. M. (1986). An Introduction to Unification-Based Approaches to Grammar. Stanford: CSLI.

Younger, D. H. (1967). Recognition and parsing of context-free languages in time n^3. Information and Control, 10(2), 189-208.

Booth, T. L. and Thompson, R. A. (1973). Applying probability measures to abstract languages. IEEE Transactions on Computers, C-22(5), 442-450.

Charniak, E. (1996). Tree-bank grammars. Proceedings of the 13th National Conference on Artificial Intelligence, 1031-1036.